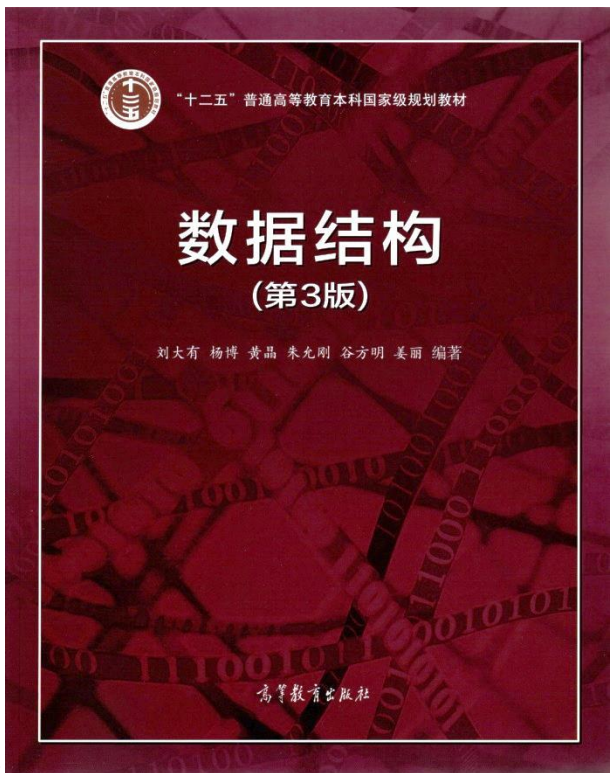




栈及其应用

- 栈的实现
- 进制转换
- 括号匹配
- 表达式求值
- 栈扩容
- 栈混洗
- 递归与回溯

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

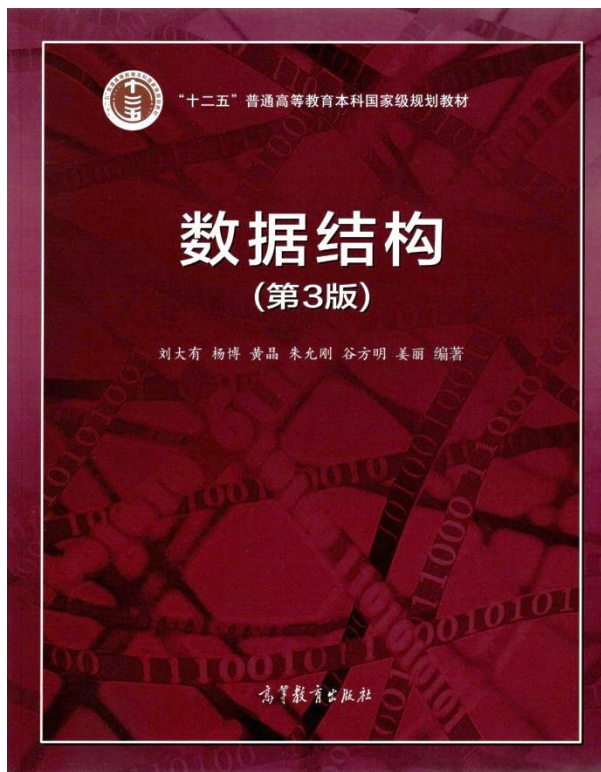


流水不争先，争滔滔不绝。



栈及其应用

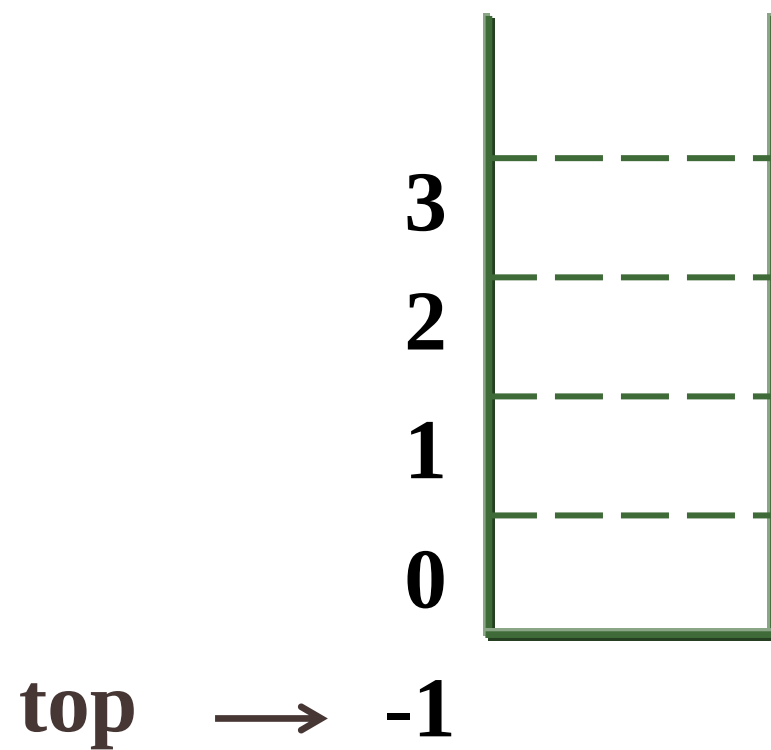
- 栈的实现
- 进制转换
- 括号匹配
- 表达式求值
- 栈混洗
- 栈扩容
- 递归与回溯



数据之法
结构之美
算法之道

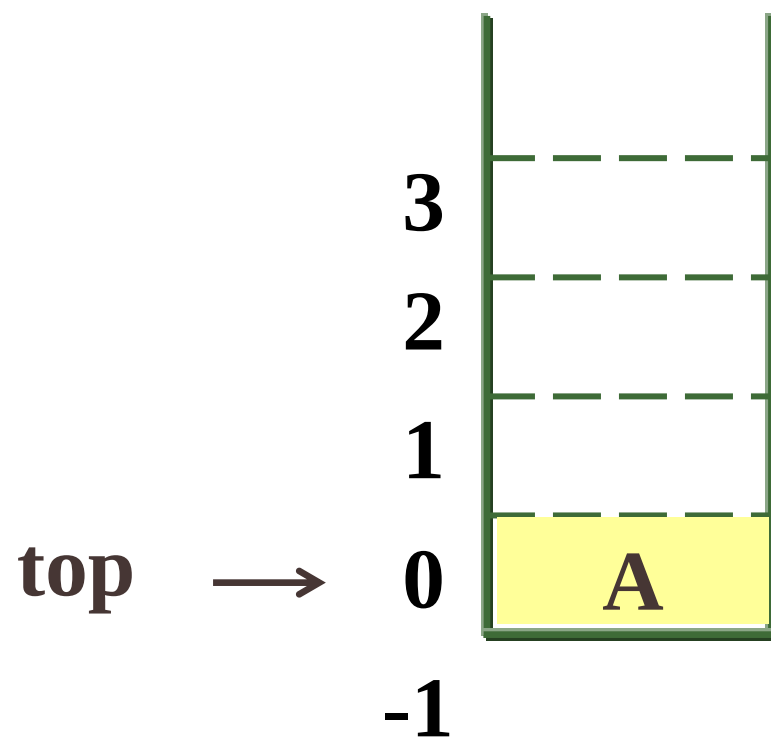
zhuyungang@jlu.edu.cn

栈



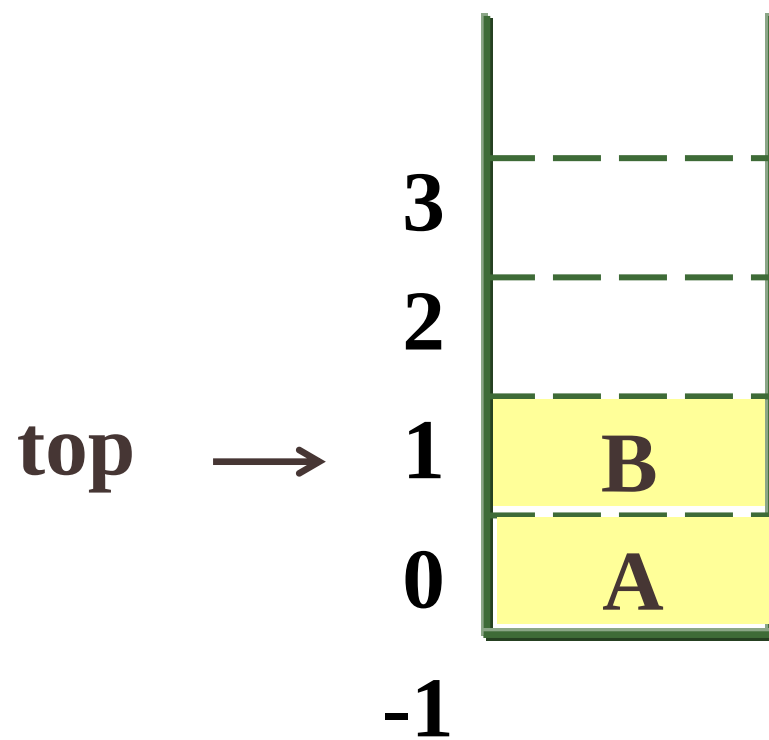
空栈

栈



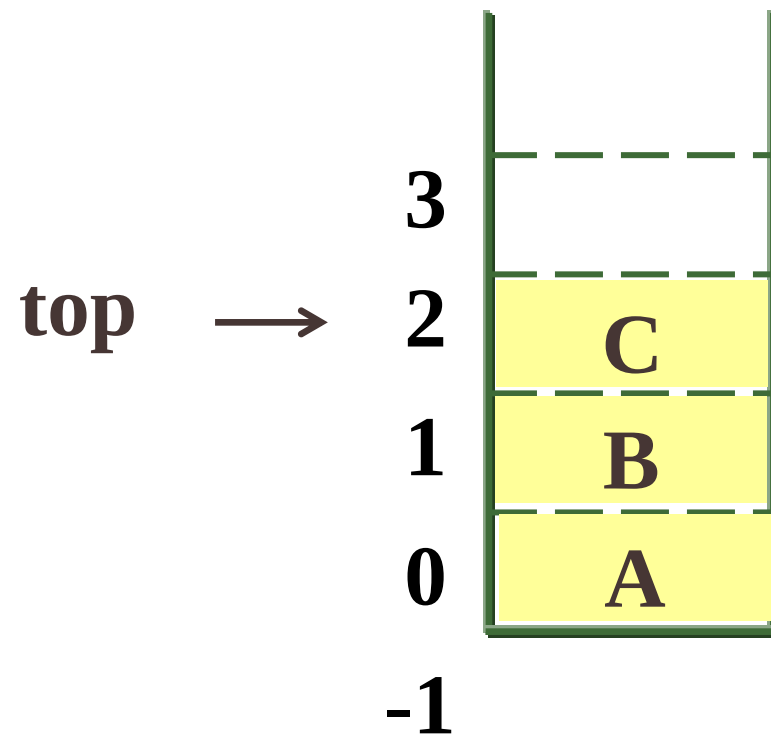
压栈A

栈



压栈B

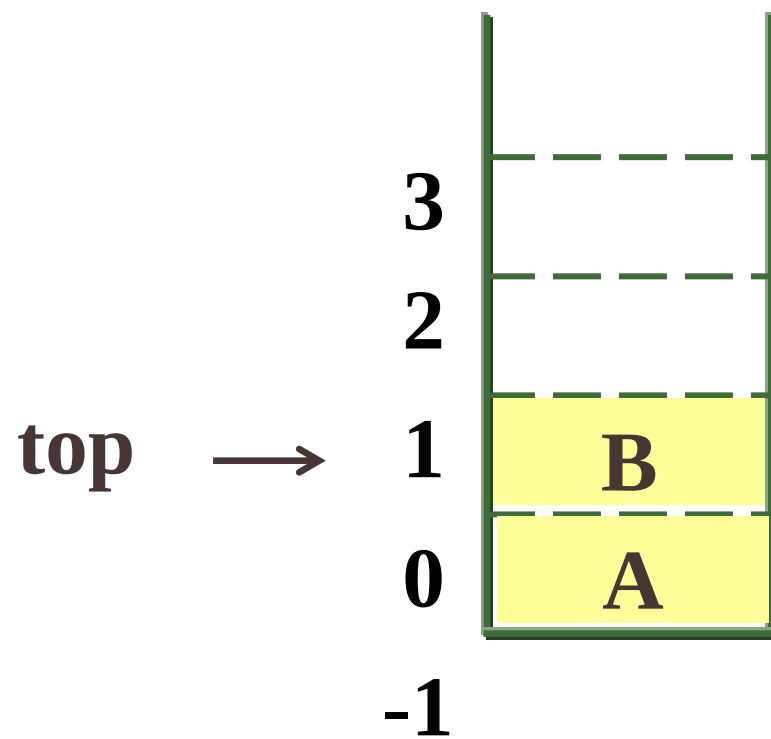
栈



压栈C

A yellow rectangular box with a blue border containing the text '压栈C' (Push C onto the stack). A blue arrow points from this box to the stack diagram, specifically to the cell at index 2.

栈

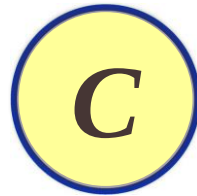


弹栈

后进先出

栈的数组实现

```
const int maxsize = 1e4 + 10;
template <class T>
class Stack {
public:
    bool empty() { return top == -1; }
    bool full() { return top == maxsize - 1; }
    void push(T K) { assert(!full()); A[++top] = K; }
    T pop() { assert(!empty()); return A[top--]; }
    T peek() { assert(!empty()); return A[top]; }
private:
    T A[maxsize];
    int top = -1;
};
```



防御式编程——assert()断言

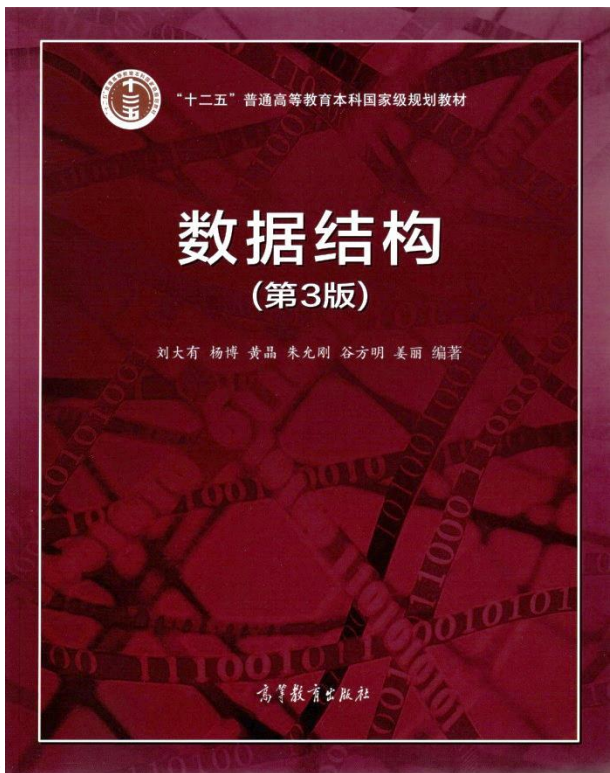
```
void assert(int expression);
```

- assert宏定义在<assert.h>中，功能是先计算表达式expression，如果其值为真（即为非0）则程序继续执行，若其值为假（即为0）则调用abort()函数终止程序运行。
- 作用：使用断言来声明代码中所做的假设，使程序在运行时进行自查，如果断言为真则表明程序运行正常，如果断言为假意味着在代码中检测到一个意外的错误。从而便于排查出各种意外情况，定位出错点。
- 用于检查某段代码执行前必须具备的状态，或某段代码执行后必须具备的状态。



栈及其应用

- 栈的实现
- **进制转换**
- 括号匹配
- 表达式求值
- 栈混洗
- 栈扩容
- 递归与回溯



数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

进制转换

例：十进制转换成七进制： $(66)_{10} = (123)_7$

$$66/7 = 9 \text{ 余 } 3$$

$$9/7 = 1 \text{ 余 } 2$$

$$1/7 = 0 \text{ 余 } 1$$

结果为余数的逆序
123

- 不断除以7取余数。后生成的余数先输出，先生成的余数后输出，正好符合栈的**后进先出**性质。
- 把余数按生成的顺序压栈，最后再依次弹栈。



进制转换

给定一个整数 n ，将其转化为 7 进制，并以字符串形式输出，
 $-10^7 \leq n \leq 10^7$ 。【[LeetCode 504](#)】

进制转换

给定一个整数 n ，将其转化为 7 进制，并以字符串形式输出， $-10^7 \leq n \leq 10^7$ 。【[LeetCode 504](#)】

```
string convertToBase7(int n) {  
    if(n==0) return "0";  
    Stack<int> s;  
    string ans=(n>0)?"":"-";  
    n=abs(n);  
    while(n>0){  
        s.push(n%7);  
        n/=7;  
    }  
    while(!s.empty())  
        ans += s.pop()+'0';  
    return ans;  
}
```

```
string convertToBase7(int n) {  
    if(n==0) return "0";  
    int stack[110], top=-1;  
    string ans=(n>0)?"":"-";  
    n=abs(n);  
    while(n>0){  
        stack[++top]=n%7;  
        n/=7;  
    }  
    while(top!=-1)  
        ans += stack[top--]+'0';  
    return ans;  
}
```

进制转换

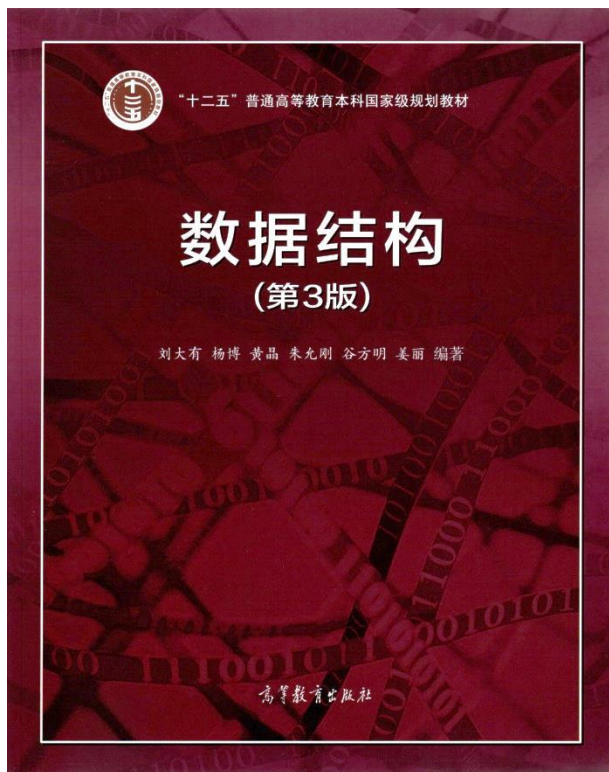
➤ 十进制转 K 进制 ($K > 10$)，以 $K=16$ 为例：

```
char digit[] = { '0', '1', '2', '3', '4', '5', '6',  
                 '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F', };  
while(n > 0) {  
    s.push(digit[n % K]);  
    n /= K;  
}
```




栈及其应用

- 栈的实现
- 进制转换
- **括号匹配**
- 表达式求值
- 栈混洗
- 栈扩容
- 递归与回溯



数据之法
结构之美
算法之道

括号匹配

➤ 给定一个字符串，判断其中的括号是否匹配。匹配：“(”与“)”匹配、“[”与“]”匹配、“{”与“}”匹配。【大厂面试题 [LeetCode 20](#)】

➤ 用途：编译器语法检查

输入	是否匹配
}	×
[(]	×
{ (}	×
{a=(b*c)+func() }	×
({ })	✓

算法思路——只有一种括号

➤ 若仅需考虑一种括号，只需一个计数器，初值为0，从左往右扫描字符串。

- ✓ 遇到左括号：计数器加1
- ✓ 遇到右括号：计数器减1
- ✓ 扫描过程中计数器的值始终 ≥ 0
- ✓ 扫描结束后，计数器的值归0

计数器的值：当前左括号比右括号多了多少个

➤ 可否推广至多种括号的情况？
[()]

算法思路——多种括号

- 根据匹配规则：后遇到的左括号先匹配，考虑采用栈
- 从左向右扫描字符串
 - ✓ 若遇到左括号：压栈
 - ✓ 若遇到右括号：弹出栈顶的左括号与之匹配。
- 栈只存左括号

算法思路

- 根据匹配规则：后遇到的左括号先匹配，考虑栈
- 从左向右扫描字符串
 - ✓ 若遇到左括号：压栈
 - ✓ 若遇到右括号：弹出栈顶的左括号与之匹配。
- 栈只存左括号

([{ })



左括号栈



动作

“(”压栈

算法思路

- 根据匹配规则：后遇到的左括号先匹配，考虑栈
- 从左向右扫描字符串
 - ✓ 若遇到左括号：压栈
 - ✓ 若遇到右括号：弹出栈顶的左括号与之匹配。
- 栈只存左括号

([{ })



左括号栈

([

动作

“ [” 压栈

算法思路

- 根据匹配规则：后遇到的左括号先匹配，考虑栈
- 从左向右扫描字符串
 - ✓ 若遇到左括号：压栈
 - ✓ 若遇到右括号：弹出栈顶的左括号与之匹配。
- 栈只存左括号

([{ }



左括号栈

([{

动作

“{”压栈

算法思路

- 根据匹配规则：后遇到的左括号先匹配，考虑栈
- 从左向右扫描字符串
 - ✓ 若遇到左括号：压栈
 - ✓ 若遇到右括号：弹出栈顶的左括号与之匹配。
- 栈只存左括号

([{ })



左括号栈



动作

“{”弹栈，并与当前扫描到的“}”匹配

算法思路

- 根据匹配规则：后遇到的左括号先匹配，考虑栈
- 从左向右扫描字符串
 - ✓ 若遇到左括号：压栈
 - ✓ 若遇到右括号：弹出栈顶的左括号与之匹配。
- 栈只存左括号

([{ })

左括号栈

([

动作

“[” 弹栈，并与当前扫描到的 “)” 匹配

```
bool isValid(string s) {  
    int n=s.size();  
    Stack<char> bracket;  
    char pair[150]={0};  
    pair['(']=')';    pair['[']=']';    pair['{']='}';  
    for(int i=0; i<n; i++){           //扫描字符串  
        if(s[i]=='(' || s[i]=='[' || s[i]=='{')  
            bracket.push(s[i]);       //遇左括号, 压栈  
        else if(s[i]==')' || s[i]==']' || s[i]=='}'){//遇右括号  
            if(bracket.empty()) return false;  
            char t=bracket.pop();      //弹出左括号  
            if(pair[t]!=s[i]) return false;  
        }  
    }  
    return bracket.empty();  
}
```

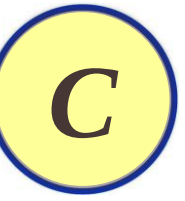
*Talk is cheap,
show me the code*



课下思考

编写程序检查给定字符串中包含的括号是否正确匹配，本题中的括号有{ }、[]、()、< >四种。**另外再加上一个新的约束条件：当有多种括号嵌套时，嵌套的顺序应为{ → [→ (→ <**，例如{[(O)]},{(O)},{{} } 为合法的表达式，而 ([{}]),{([])},[{<>}] 都是非法的。【中山大学考研复试机试题，吉林大学2024年保研夏令营机试题，[AcWing 3703](#)】





问题

若只考虑一种括号， n 对括号共有多少种可能的合法匹配序列？

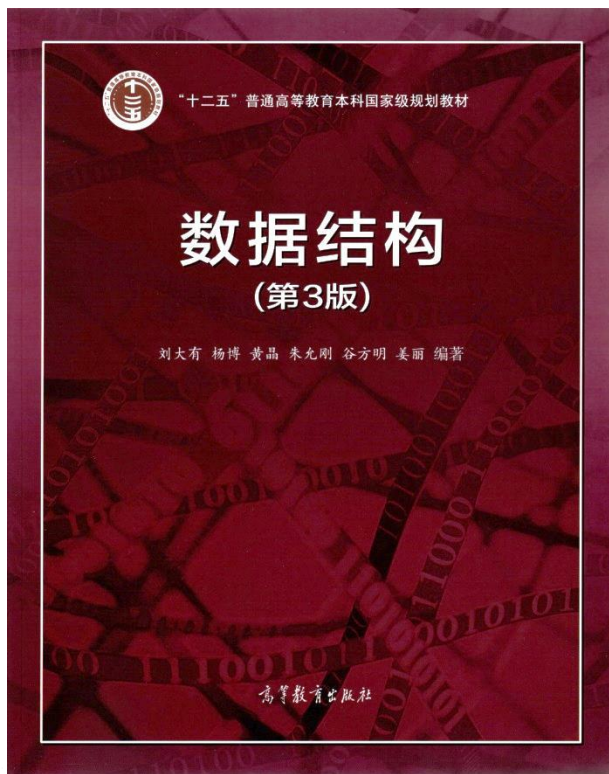
例如 $n=3$

$((()))$

$()()()$

$(())()$

.....



栈及其应用

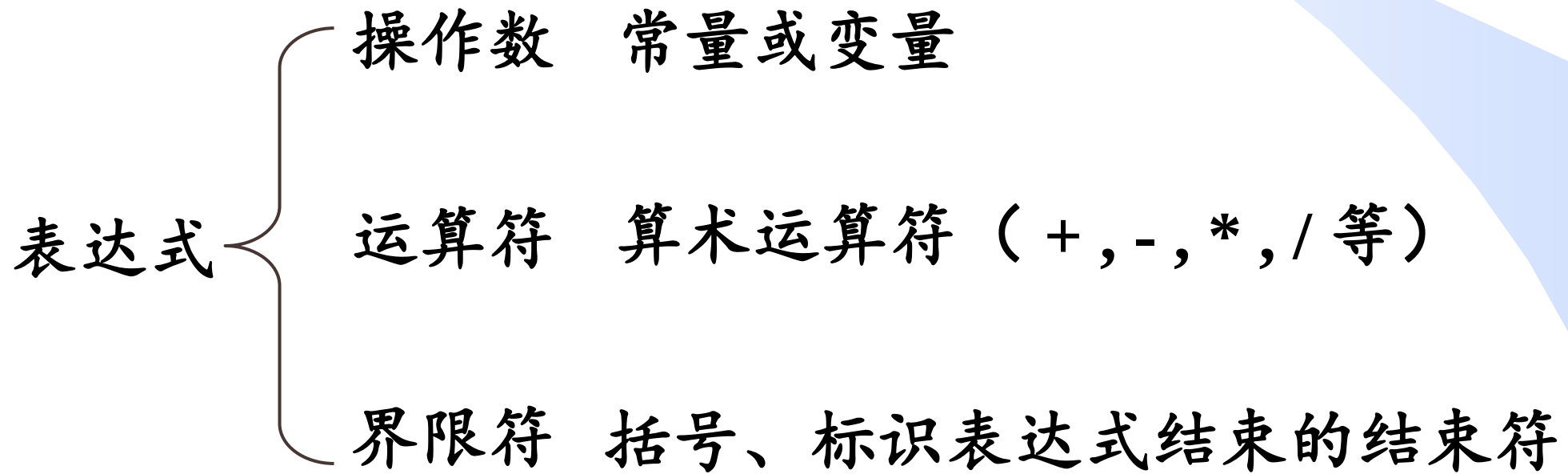
- 栈的实现
- 进制转换
- 括号匹配
- **表达式求值**
- 栈混洗
- 栈扩容
- 递归与回溯

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

算术表达式求值

算术表达式求值是高级语言在编译过程中的一个基本问题。它的实现是栈的一个典型应用。



算术表达式的表示方法

- **中缀表达式**：运算符在运算对象（操作数或子表达式）之间，
如 $(a + b \times c) / d - e$

运算规则：

- ✓ 先计算括号内，后计算括号外；
 - ✓ 在无括号或同层括号内，先进行乘除运算，后进行加减运算，即乘除运算的优先级高于加减运算的优先级；
 - ✓ 同一优先级运算，从左向右依次进行。
- 一般来说，往往把**中缀表达式**转换成另外一种表示方法，即**后缀表达式**，然后对后缀表达式进行处理，后缀表达式也称为逆波兰表达式。

后缀表达式的概念

1929 年，由波兰数学家Lukasiewicz提出。

➤定义：运算符紧跟在运算对象之后的表达式。

➤优点：① 没有括号

② 不存在优先级的差别

③ 计算过程完全按照运算符出现的先后次序进行



中缀表达式	后缀表达式
$a + b$	$a b +$
$1 + 2 \times 3$	$1 2 3 \times +$
$(a + b) \times c$	$a b + c \times$
$a \times b \times c + c \times d$	$a b \times c \times c d \times +$
$(a + b) \times ((c - d) \times e + f)$	$a b + c d - e \times f + \times$
$a + (b - c / d) \times e$	$a b c d / - e \times +$

后缀表达式求值的算法

设置一个栈，存放操作数

从左到右依次读入后缀表达式的每一个操作数/运算符/结束符：

- ①若读到的是操作数，将它压入栈。
- ②若读到的是运算符，就从操作数栈中连续弹出两个元素（操作数），进行相应的运算，并将结果压入栈中。
- ③读入结束符时，栈顶元素就是计算结果。

中缀表达式	后缀表达式
$1 + 2 \times 3$	$1\ 2\ 3\ \times\ +$



6 2 3 × / 8 + 2 4 × -



操作数栈

6

操作

6入栈

6 2 3 × / 8 + 2 4 × -



操作数栈

6 2

操作

2入栈

6 2 3 × / 8 + 2 4 × -



操作数栈

6 2 3

操作

3入栈

6 2 3 × / 8 + 2 4 × -



操作数栈

6 2 3

操作

3和2弹栈
计算 $2 \times 3 = 6$
并压栈6

6 2 3 × / 8 + 2 4 × -



操作数栈

6 6

操作

3和2弹栈
计算 $2 \times 3 = 6$
并压栈6

6 2 3 × / 8 + 2 4 × -



操作数栈

6 6

操作

6和6弹栈
计算 $6/6=1$
并压栈1

6 2 3 × / 8 + 2 4 × -



操作数栈

1

操作

6和6弹栈
计算 $6/6=1$
并压栈1

6 2 3 × / 8 + 2 4 × -



操作数栈

1 8

操作

8入栈

6 2 3 × / 8 + 2 4 × -



操作数栈

1 8

操作

8和1弹栈
计算 $1+8=9$
并压栈9

6 2 3 × / 8 + 2 4 × -



操作数栈

9

操作

8和1弹栈
计算 $1+8=9$
并压栈9

6 2 3 × / 8 + 2 4 × -



操作数栈

9 2

操作

2入栈

6 2 3 × / 8 + 2 4 × -



操作数栈

9 2 4

操作

4入栈

6 2 3 × / 8 + 2 4 × -



操作数栈

9 2 4

操作

4和2弹栈
计算 $2 \times 4 = 8$
并压栈8

6 2 3 × / 8 + 2 4 × -



操作数栈

9 8

操作

4和2弹栈
计算 $2 \times 4 = 8$
并压栈8

6 2 3 × / 8 + 2 4 × -



操作数栈

9 8

操作

8和9弹栈
计算 $9-8=1$
并压栈1

6 2 3 × / 8 + 2 4 × -



操作数栈

1

操作

8和9弹栈
计算 $9-8=1$
并压栈1

6 2 3 × / 8 + 2 4 × -



操作数栈

1

操作
结束

后缀表达式求值

算法 $\text{Calculate}(P, n)$ // P 存储长度为 n 的后缀表达式

CREATES (S).

FOR $i=1$ **TO** n **DO**

IF $P[i]$ 是操作数 **THEN** $S \leftarrow P[i]$.

ELSE // $P[i]$ 是运算符

($y \leftarrow S$. $x \leftarrow S$.

$S \leftarrow \text{Operation}(x, P[i], y)$

)

运算符 $P[i]$ 作用于
操作数 x 和 y 的结果

ADL语言中的栈操作

CREATES (S): 建立一个栈 S ;

$S \leftarrow x$: 元素 x 进栈;

$x \leftarrow S$: 元素 x 出栈;

算法 $\text{Operation}(x, t, y)$

//假定只有 $+-\times/$ 运算且除数不为 0

IF $t = '+'$ **THEN RETURN** $x+y$.

IF $t = '-'$ **THEN RETURN** $x-y$.

IF $t = '\times'$ **THEN RETURN** $x*y$.

IF $t = '/'$ **THEN RETURN** x/y .

中缀表达式转后缀表达式算法

中缀表达式 $A - (B + C / D) \times E$ 的后缀形式是 (**D**)

A. $A B - C + D / E \times$

B. $A B C + D / - E \times$

C. $A B C D / E \times + -$

D. $A B C D / + E \times -$

后缀表达式与中缀表达式操作数出现顺序一样，运算符出现顺序就是实际的计算顺序。

手工方法:

① 将表达式充分加括号，最终使每一对括号里仅有一个运算符:

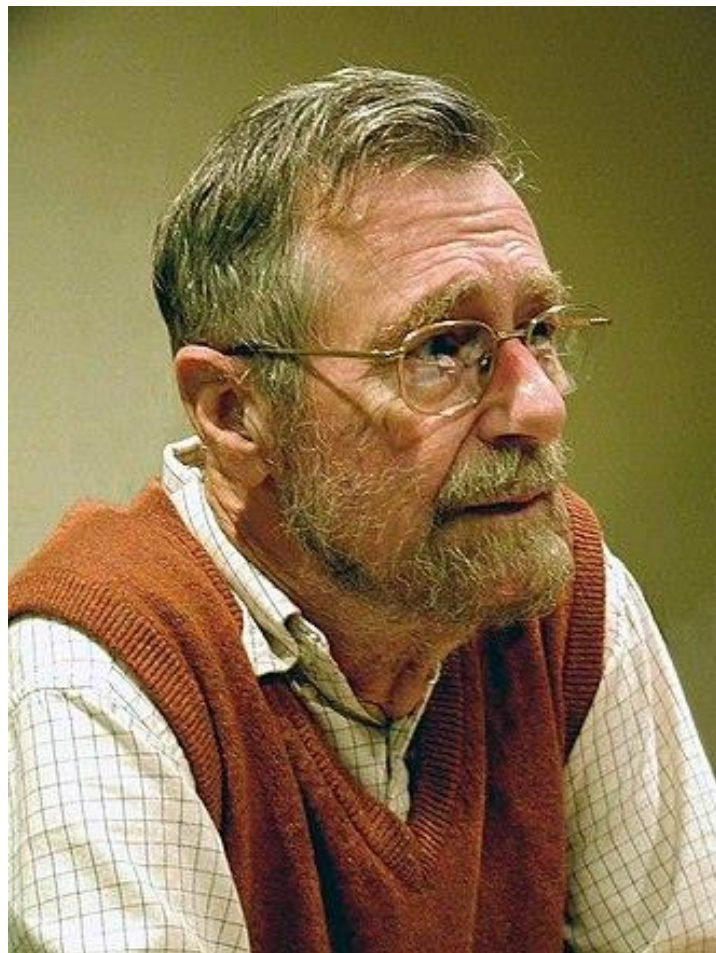
$(A - ((B + (C / D)) \times E))$

② 移动运算符使之放在与之对应的右括号后面:

$(A ((B (C D) /) + E) \times) -$

③ 去掉所有括号: $A B C D / + E \times -$

中缀表达式转后缀表达式算法——调度场算法



Edsger Dijkstra (1930-2002)

德州大学奥斯汀分校教授

图灵奖获得者（首位非英美籍获奖者）

荷兰皇家科学院院士



中缀表达式转后缀表达式算法：大致思路

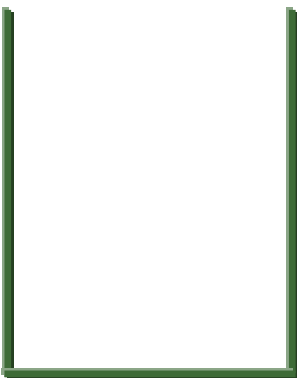
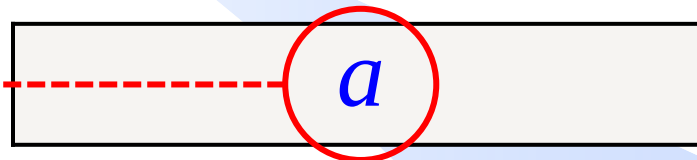
设置一个栈，存放运算符和括号

从左到右依次读入中缀表达式的每一个元素：

后缀表达式



中缀表达式



运算符栈

若遇到操作数
则直接放入后缀
表达式

中缀表达式转后缀表达式算法：大致思路

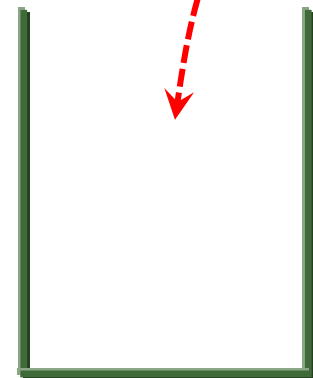
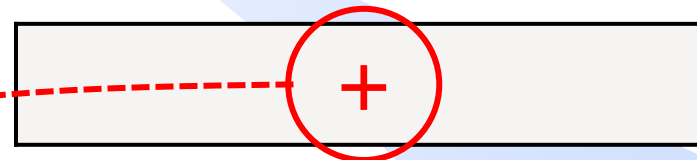
设置一个栈，存放运算符和括号

从左到右依次读入中缀表达式的每一个元素：

后缀表达式



中缀表达式



运算符栈

若遇到运算符
选择1：直接压栈

中缀表达式转后缀表达式算法：大致思路

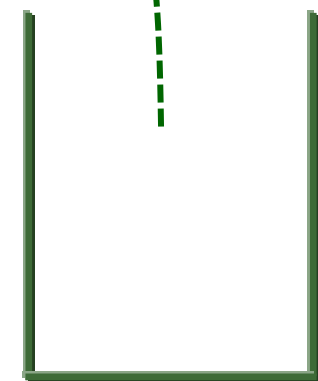
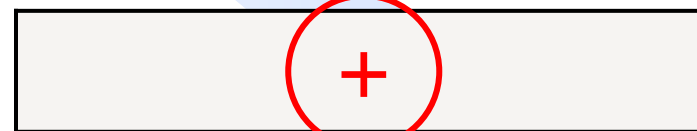
设置一个栈，存放运算符和括号

从左到右依次读入中缀表达式的每一个元素：

后缀表达式



中缀表达式



运算符栈

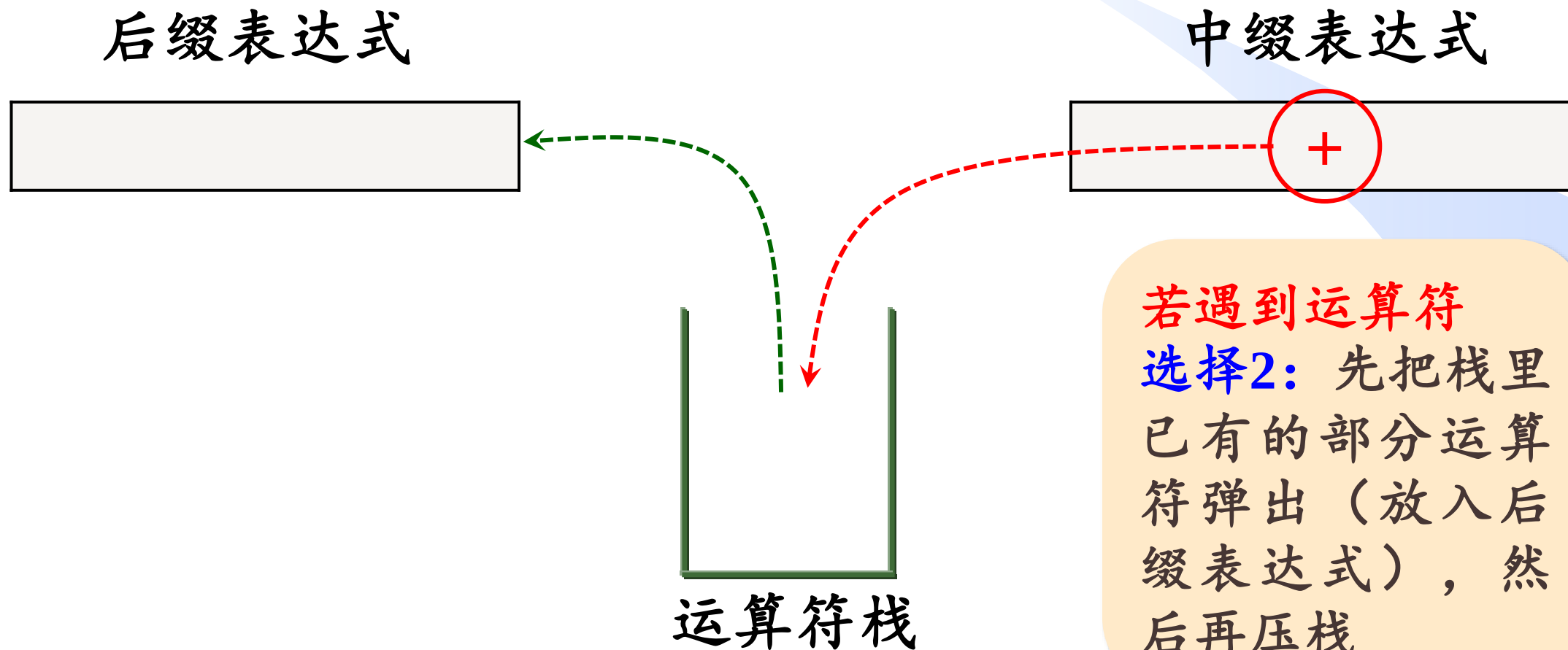
若遇到运算符

选择2：先把栈里已有的部分运算符弹出（放入后缀表达式），然后再压栈

中缀表达式转后缀表达式算法：大致思路

设置一个栈，存放运算符和括号

从左到右依次读入中缀表达式的每一个元素：



中缀表达式转后缀表达式——调度场算法

设置一个栈，存放运算符

从左到右依次读入中缀表达式的每一个元素：

➤ **操作数规则**：直接放入后缀表达式

➤ **运算符规则**：

(1) **栈空或栈顶是左括号**：压栈

(2) 当前运算符优先级 > 栈顶运算符：压栈

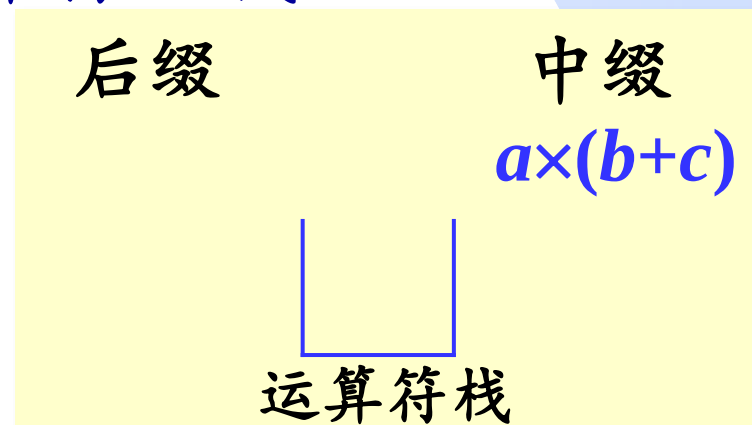
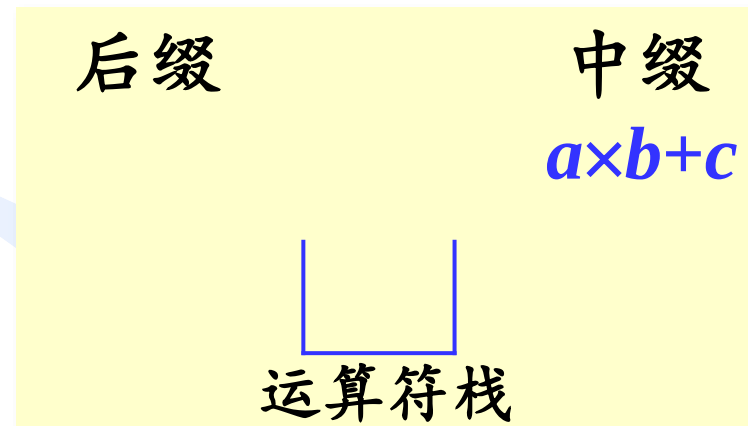
(3) 当前运算符优先级 ≤ 栈顶运算符：弹栈直至**当前运算符优先级 > 栈顶或栈空或栈顶为左括号**（期间弹出的运算符顺序依次放入后缀表达式），再把当前运算符压栈

➤ **括号规则**：

(1) 遇到左括号：压栈

(2) 遇到右括号：弹栈直至左括号

➤ **结束符规则**：弹栈至栈空



中缀表达式转后缀表达式——举例

后缀表达式

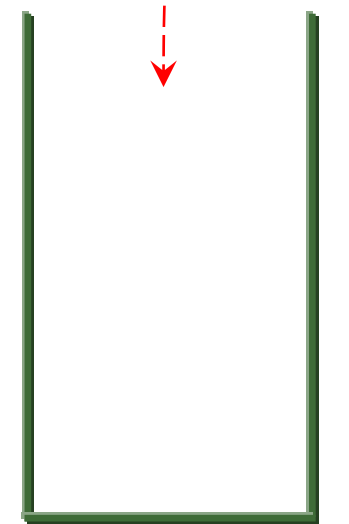
中缀表达式

$(A+B) \times ((C-D) \times E + F)$



括号：

- (1) 遇左括号：压栈
- (2) 遇右括号：弹栈
直至左括号



运算符栈

动作

(入栈

中缀表达式转后缀表达式——举例

后缀表达式

中缀表达式

← $A+B) \times ((C-D) \times E+F)$



操作数：
直接放入后缀表达式

(
运算符栈

动作

A放入
后缀表达式

中缀表达式转后缀表达式——举例

后缀表达式

A

运算符:

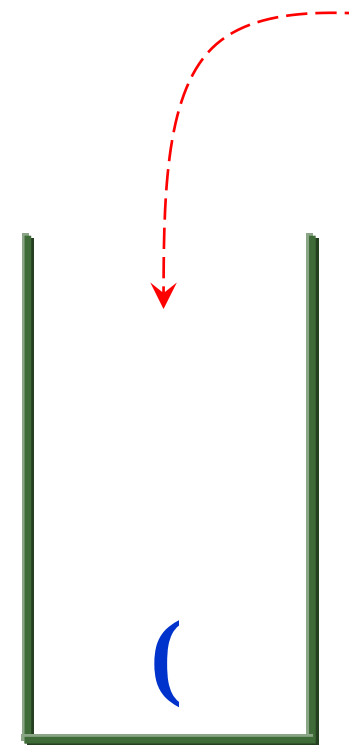
(1) 栈空或栈顶是左括号:

压栈

(2) 当前运算符优先级 > 栈顶运算符: 压栈

(3) 当前运算符优先级 ≤ 栈顶运算符: 弹栈直至当前运算符优先级 > 栈顶或栈空或栈顶为左括号, 再把当前运算符压栈

中缀表达式

 $+B) \times ((C-D) \times E + F)$ 

运算符栈

动作

+入栈

中缀表达式转后缀表达式——举例

后缀表达式

A

中缀表达式

 $B) \times ((C - D) \times E + F)$

操作数：
直接放入后缀表达式

+

(

运算符栈

动作

B放入
后缀表达式

中缀表达式转后缀表达式——举例

后缀表达式

AB

中缀表达式

)×((C-D)×E+F)



动作

+和 (弹栈

括号:

- (1) 遇左括号: 压栈
- (2) 遇右括号: 弹栈
直至左括号

+
(

运算符栈

中缀表达式转后缀表达式——举例

后缀表达式

AB+

运算符:

(1) 栈空或栈顶是左括号:
压栈

(2) 当前运算符优先级 > 栈
顶运算符: 压栈

(3) 当前运算符优先级 ≤ 栈
顶运算符: 弹栈直至当前运
算符优先级 > 栈顶或栈空或
栈顶为左括号, 再把当前运
算符压栈

运算符栈

中缀表达式

$\times((C-D)\times E+F)$

动作

×入栈

中缀表达式转后缀表达式——举例

后缀表达式

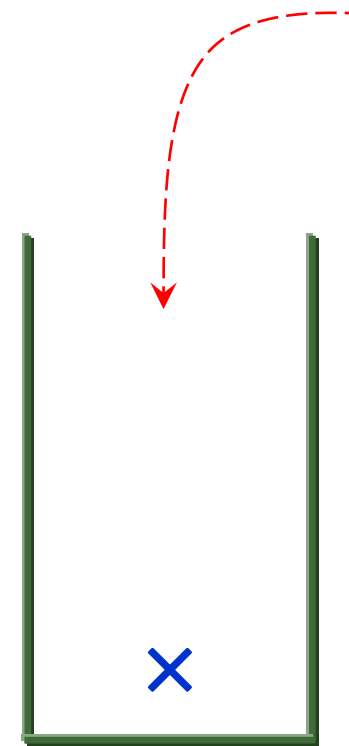
AB+

中缀表达式

$((C-D) \times E + F)$

括号：

- (1) 遇左括号：压栈
- (2) 遇右括号：弹栈
直至左括号



运算符栈

动作

(入栈

中缀表达式转后缀表达式——举例

后缀表达式

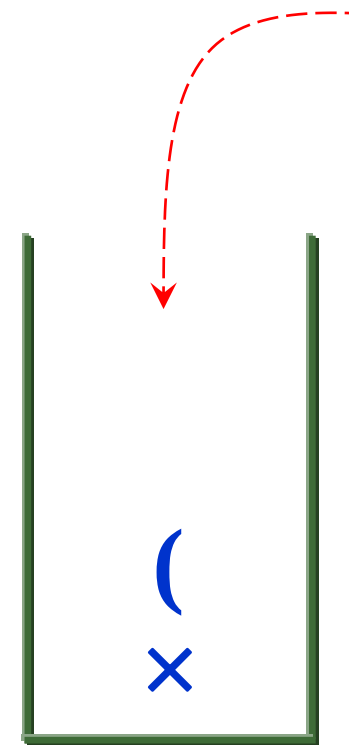
AB+

中缀表达式

$(C-D) \times E + F$

括号：

- (1) 遇左括号：压栈
- (2) 遇右括号：弹栈
直至左括号



运算符栈

动作

(入栈

中缀表达式转后缀表达式——举例

后缀表达式

AB+

中缀表达式

C-D)×E+F)

操作数：
直接放入后缀表达式

(
(
×
运算符栈

动作
C 入
后缀表达式

中缀表达式转后缀表达式——举例

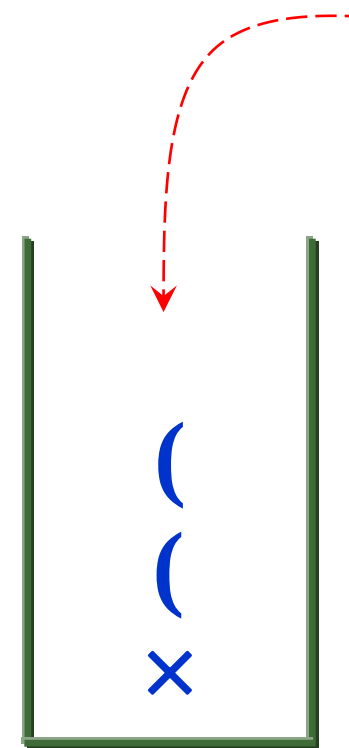
后缀表达式

 $AB+C$

运算符:

(1) 栈空或栈顶是左括号:
压栈(2) 当前运算符优先级 > 栈
顶运算符: 压栈(3) 当前运算符优先级 $\leq$ 栈
顶运算符: 弹栈直至当前运
算符优先级 > 栈顶或栈空或
栈顶为左括号, 再把当前运
算符压栈

中缀表达式

 $-D) \times E + F)$ 

运算符栈

动作

-入栈

中缀表达式转后缀表达式——举例

后缀表达式

 $AB+C$

中缀表达式

 $D) \times E + F)$ 

操作数：
直接放入后缀表达式

—
(
(
×

运算符栈

动作
D 入
后缀表达式

中缀表达式转后缀表达式——举例

后缀表达式

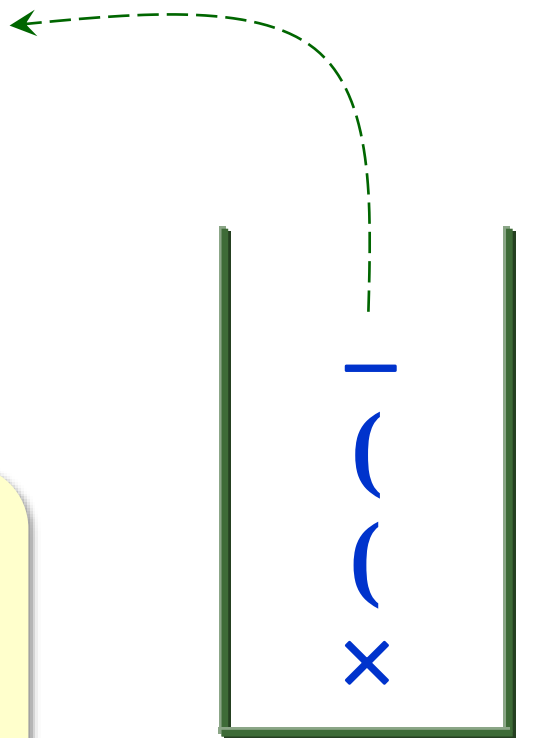
$AB+CD$

中缀表达式

$) \times E + F)$

括号:

- (1) 遇左括号: 压栈
- (2) 遇右括号: 弹栈
直至左括号



运算符栈

动作

-和(弹栈

中缀表达式转后缀表达式——举例

后缀表达式

AB+CD -

中缀表达式

×E+F)

运算符:

(1) 栈空或栈顶是左括号:

压栈

(2) 当前运算符优先级 > 栈

顶运算符: 压栈

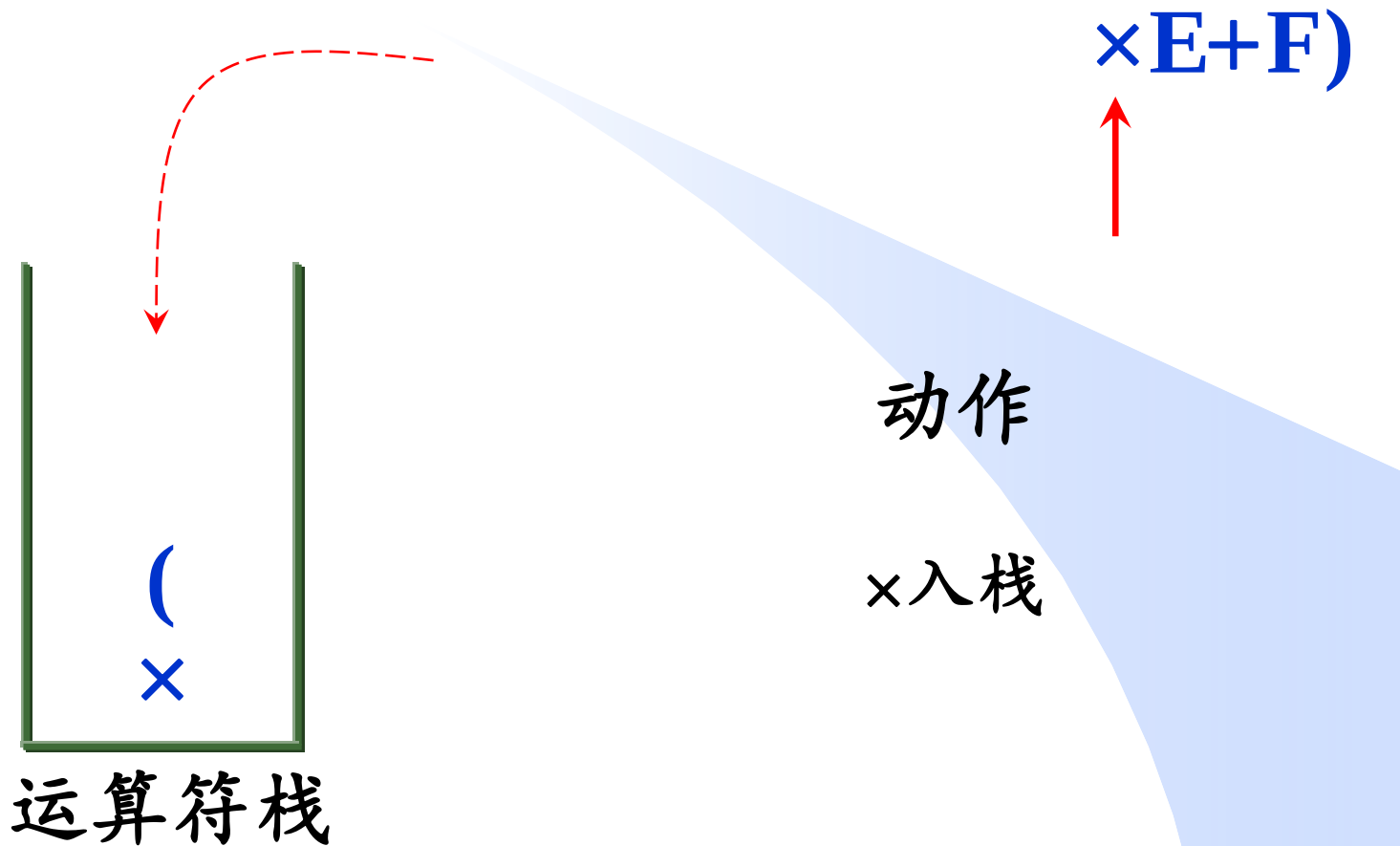
(3) 当前运算符优先级 ≤ 栈

顶运算符: 弹栈直至当前运

算符优先级 > 栈顶或栈空或

栈顶为左括号, 再把当前运

算符压栈



中缀表达式转后缀表达式——举例

后缀表达式

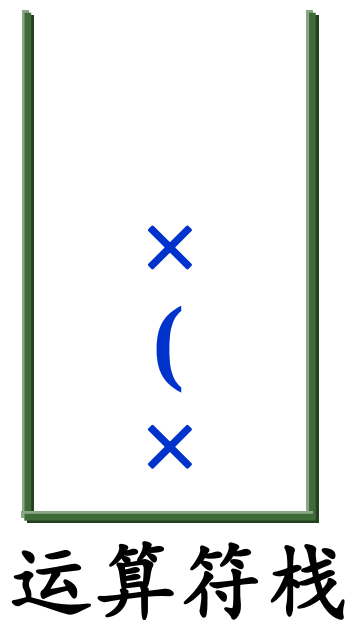
AB+CD-

中缀表达式

E+F)



操作数：
直接放入后缀表达式



动作
E入
后缀表达式

中缀表达式转后缀表达式——举例

后缀表达式

AB+CD-E

中缀表达式

+F)

运算符:

(1) 栈空或栈顶是左括号:

压栈

(2) 当前运算符优先级 > 栈

顶运算符: 压栈

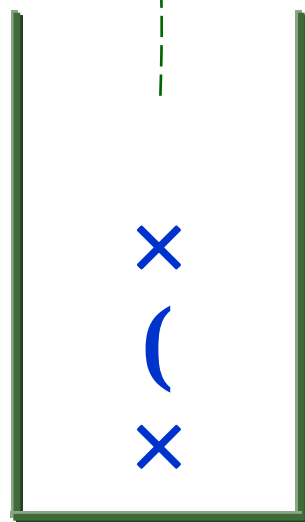
(3) 当前运算符优先级 ≤ 栈

顶运算符: 弹栈直至当前运

算符优先级 > 栈顶或栈空或

栈顶为左括号, 再把当前运

算符压栈



运算符栈

动作

×弹栈

+入栈

中缀表达式转后缀表达式——举例



后缀表达式

$AB+CD-E\times$

中缀表达式

$+F)$

运算符:

(1) 栈空或栈顶是左括号:

压栈

(2) 当前运算符优先级 > 栈

顶运算符: 压栈

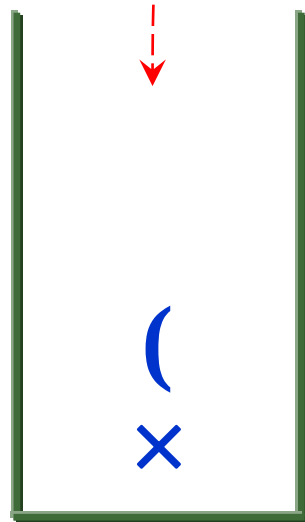
(3) 当前运算符优先级 ≤ 栈

顶运算符: 弹栈直至当前运

算符优先级 > 栈顶或栈空或

栈顶为左括号, 再把当前运

算符压栈



运算符栈

动作

$\times$ 弹栈

$+$ 入栈

中缀表达式转后缀表达式——举例

后缀表达式

AB+CD-E×

中缀表达式

F)



操作数：
直接放入后缀表达式

+
(
×

运算符栈

动作
F入
后缀表达式

中缀表达式转后缀表达式——举例

后缀表达式

$AB+CD-E\times F$

中缀表达式

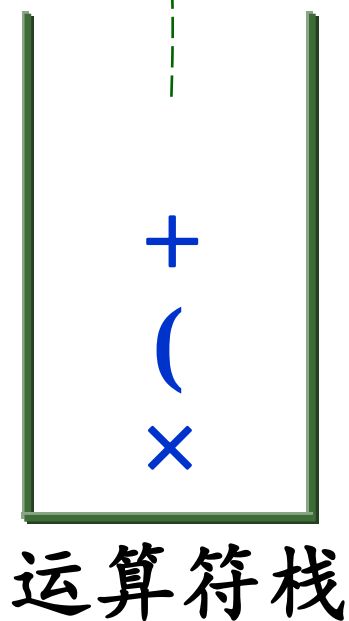
)

动作

+和(弹栈

括号:

- (1) 遇左括号: 压栈
- (2) 遇右括号: 弹栈
直至左括号



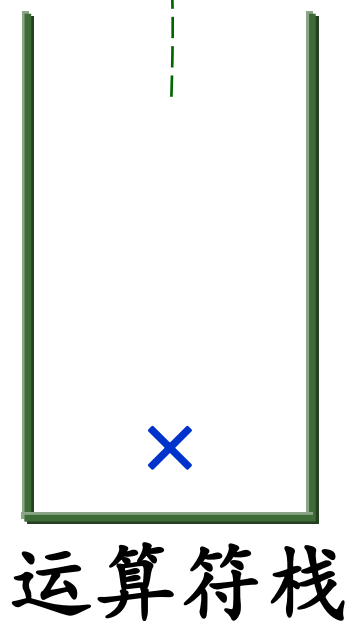
中缀表达式转后缀表达式——举例

后缀表达式

$AB+CD-E\times F+$

中缀表达式

结束符：
弹栈至栈空



动作
 $\times$ 弹栈

中缀表达式转后缀表达式——举例

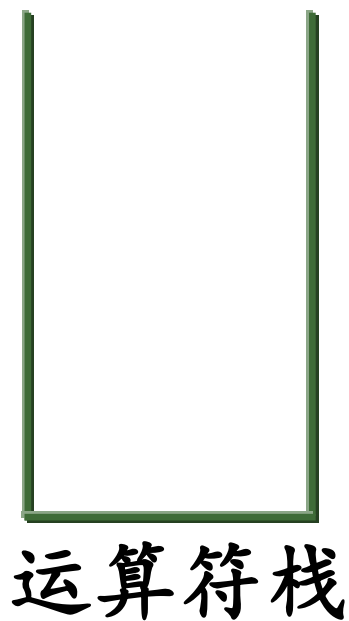


后缀表达式

$AB+CD-E\times F+\times$

中缀表达式

栈
相当于运算符
的调度场



动作
结束



例题

假设栈初始为空，将中缀表达式 $a/b+(c\times d-e\times f)/g$ 转换为等价的后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是____. 【考研题全国卷】

A. $+(\times -$

☒ B. $+(- \times$

C. $/+(\times - \times$

D. $/+ - \times$

后缀表达式

运算符栈

中缀表达式

$a / b + (c \times d - e \times f) / g$

课下思考

将中缀表达式 $a+b-a \times ((c+d)/e-f)+g$ 转换为后缀表达式的过程中，运算符栈中元素最大个数为____. 【考研题全国卷】

☒ A. 5

B. 7

C. 8

D. 11

中缀表达式转换后缀表达式算法的实现

B

- 运算符栈 S : 保存扫描中缀表达式过程中, 暂不能加入后缀表达式的运算符
- 队列 E : 保存后缀表达式

中缀表达式转换后缀表达式

B

算法 **InfixToPostfix**(P, n, S, E) // P 中缀表达式, S 运算符栈, E 后缀表达式队列

FOR $i=1$ **TO** n **DO** // 扫描中缀表达式, n 为表达式长度

($x \leftarrow P[i]$. // 读取中缀表达式的 1 个操作数/运算符/界限符

IF x 是操作数 **THEN** $E \leftarrow x$.

ELSE IF $x = '('$ **THEN** $S \leftarrow x$.

ELSE IF $x = ')'$ **THEN**

($t \leftarrow S$.

WHILE $t \neq '('$ **DO** ($E \leftarrow t$. $t \leftarrow S$.)

)

ELSE // x 是运算符

(**WHILE NOT** **Empty**(S) **AND** $\text{peek}(S) \neq '('$ **AND** x 优先级 $\leq \text{peek}(S)$ **DO**

($t \leftarrow S$. $E \leftarrow t$.)

$S \leftarrow x$.

). // **END IF**

)

WHILE NOT **Empty**(S) **DO** ($t \leftarrow S$. $E \leftarrow t$). ■

运算符规则:

(1) 栈空或栈顶是左括号: 压栈

(2) 当前运算符优先级 $>$ 栈顶运算符: 压栈

(3) 当前运算符优先级 $\leq$ 栈顶运算符: 弹栈

直至当前运算符优先级 $>$ 栈顶或栈空或栈顶为左括号, 再把当前运算符压栈

每个元素最多进栈1次、出栈1次、入队1次
时间复杂度 $O(n)$

中缀表达式求值——方案1

- 先将中缀表达式转换为后缀表达式，再对后缀表达式求值



中缀表达式求值——方案2

中缀转后缀表达式

按后缀表达式的顺序将运算符逐个送给后缀表达式

后缀表达式求值

按后缀表达式顺序扫描表达式，遇到一个运算符时，取出两个操作数进行运算

两个过程合并，中缀转后缀的过程中直接求值

当中缀表达式转后缀表达式时，当需要把一个运算符放入后缀表达式时，不放入后缀表达式了，而是直接取出两个操作数做相应的运算

中缀表达式求值——方案2

算法 **InfixToPostfix**(P, n, S, E) // P 中缀表达式, S 运算符栈, E 后缀表达式队列

FOR $i=1$ **TO** n **DO** // 扫描中缀表达式, n 为表达式长度

($x \leftarrow P[i]$. // 读取中缀表达式的 1 个操作数/运算符/界限符

IF x 是操作数 **THEN** $E \leftarrow x$.

ELSE IF $x = '('$ **THEN** $S \leftarrow x$.

ELSE IF $x = ')'$ **THEN**

 ($t \leftarrow S$.

WHILE $t \neq '('$ **DO** ($E \leftarrow t$. $t \leftarrow S$.)

)

ELSE // x 是运算符

 (**WHILE NOT** **Empty**(S) **AND** $\text{peek}(S) \neq '('$ **AND** x 优先级 $\leq \text{peek}(S)$ **DO**

 ($t \leftarrow S$. $E \leftarrow t$.)

$S \leftarrow x$.

). // **END IF**

)

WHILE NOT **Empty**(S) **DO** ($t \leftarrow S$. $E \leftarrow t$).

中缀表达式求值——方案2

算法 **CalculateInfix(P, n, S, NUM)** //S运算符栈, NUM操作数栈

FOR $i=1$ **TO** n **DO** //扫描中缀表达式, n 为表达式长度

($x \leftarrow P[i]$. //读取中缀表达式的 1 个操作数/运算符/界限符

IF x 是操作数 **THEN** **NUM** $\leftarrow x$.

ELSE IF $x = '('$ **THEN** $S \leftarrow x$.

ELSE IF $x = ')'$ **THEN**

($t \leftarrow S$.

WHILE $t \neq '('$ **DO** ($E \leftarrow t$. $t \leftarrow S$.)

)

ELSE // x 是运算符

(**WHILE NOT** Empty(S) **AND** peek(S) $\neq '('$ **AND** x 优先级 $\leq$ peek(S) **DO**

($t \leftarrow S$. $E \leftarrow t$.)

$S \leftarrow x$.

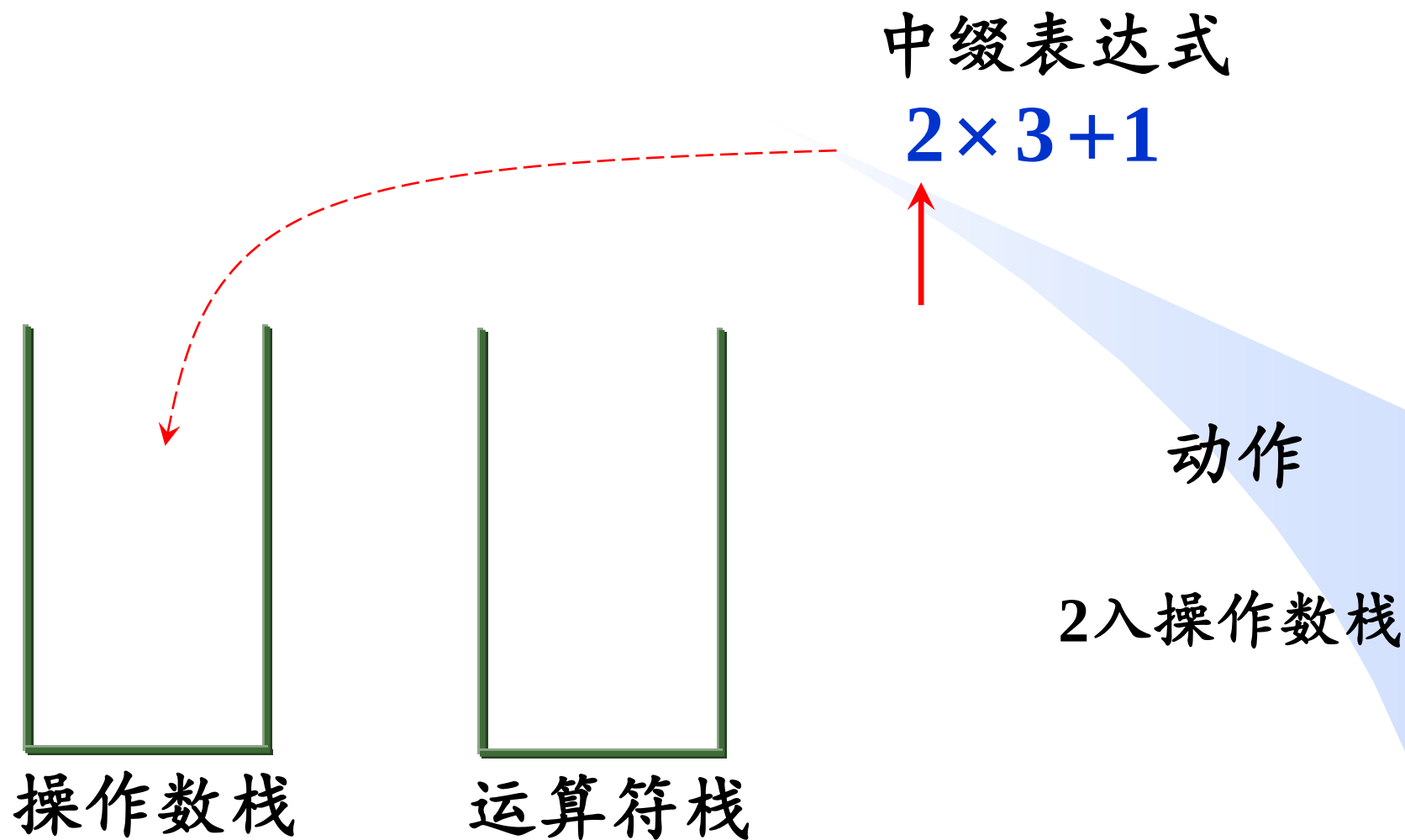
). // **END IF**

)

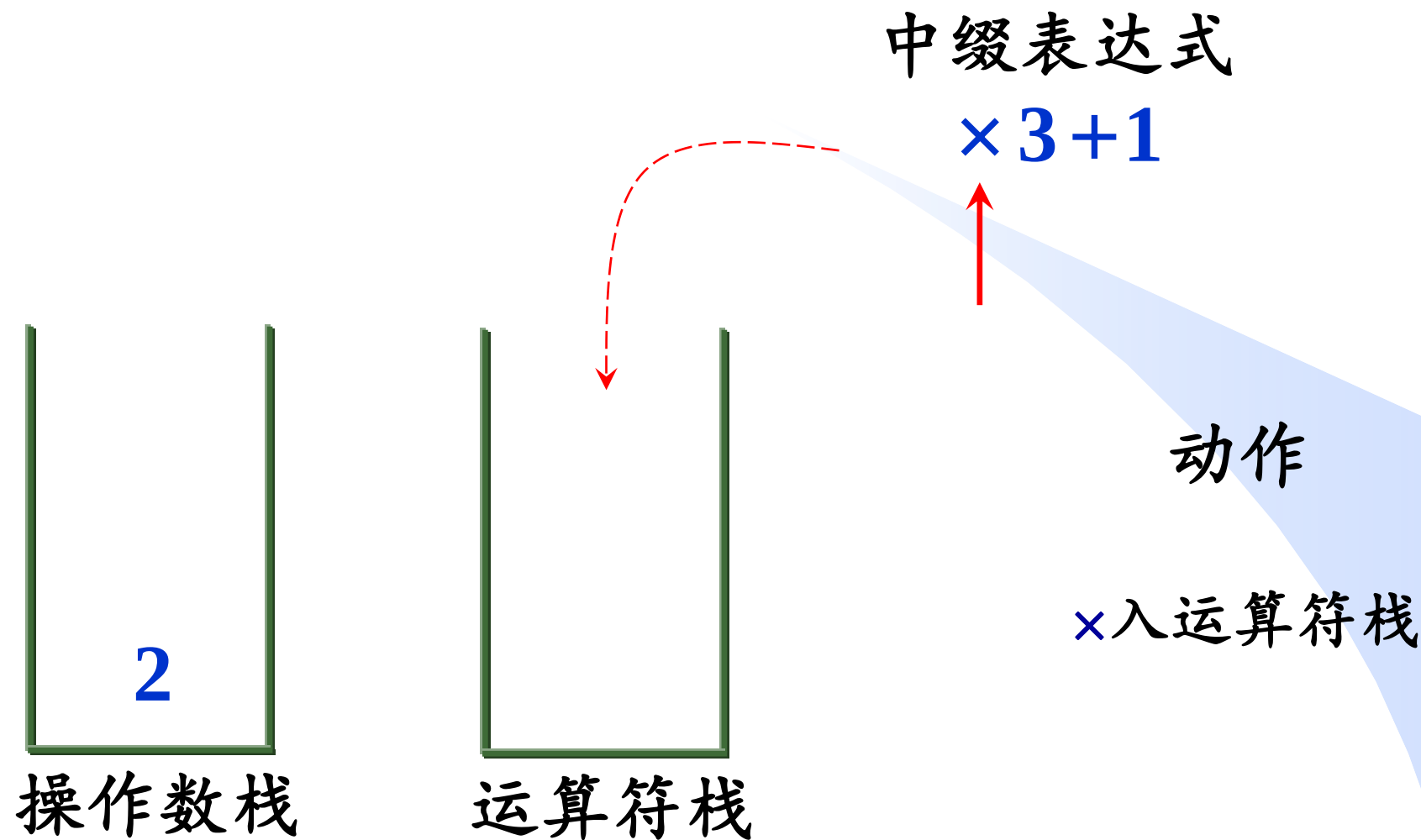
WHILE NOT Empty(S) **DO** ($t \leftarrow S$. $E \leftarrow t$). ■

$b \leftarrow \text{NUM}$. $a \leftarrow \text{NUM}$.
 $\text{NUM} \leftarrow \text{Operation}(a, t, b)$.

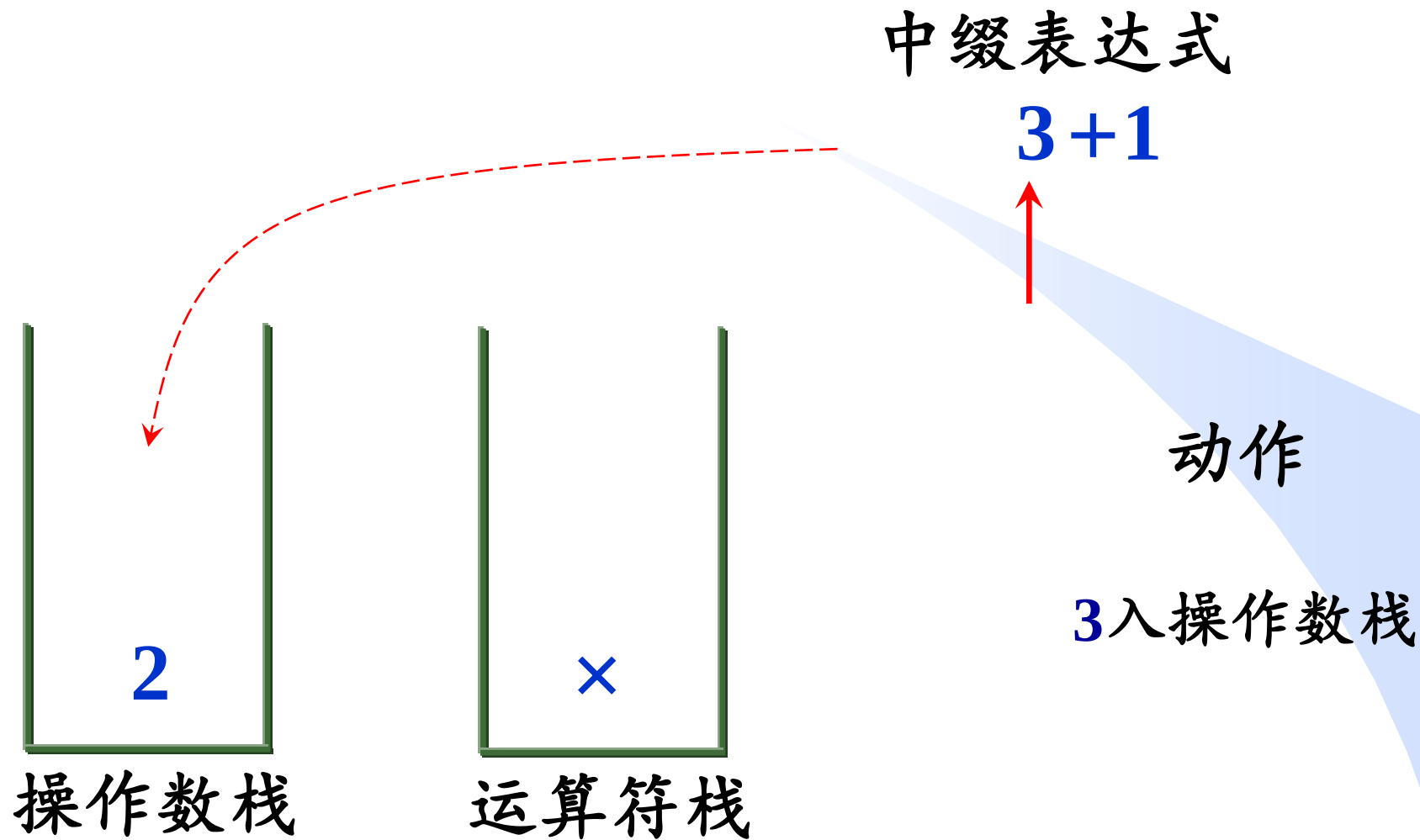
中缀表达式直接求值——举例



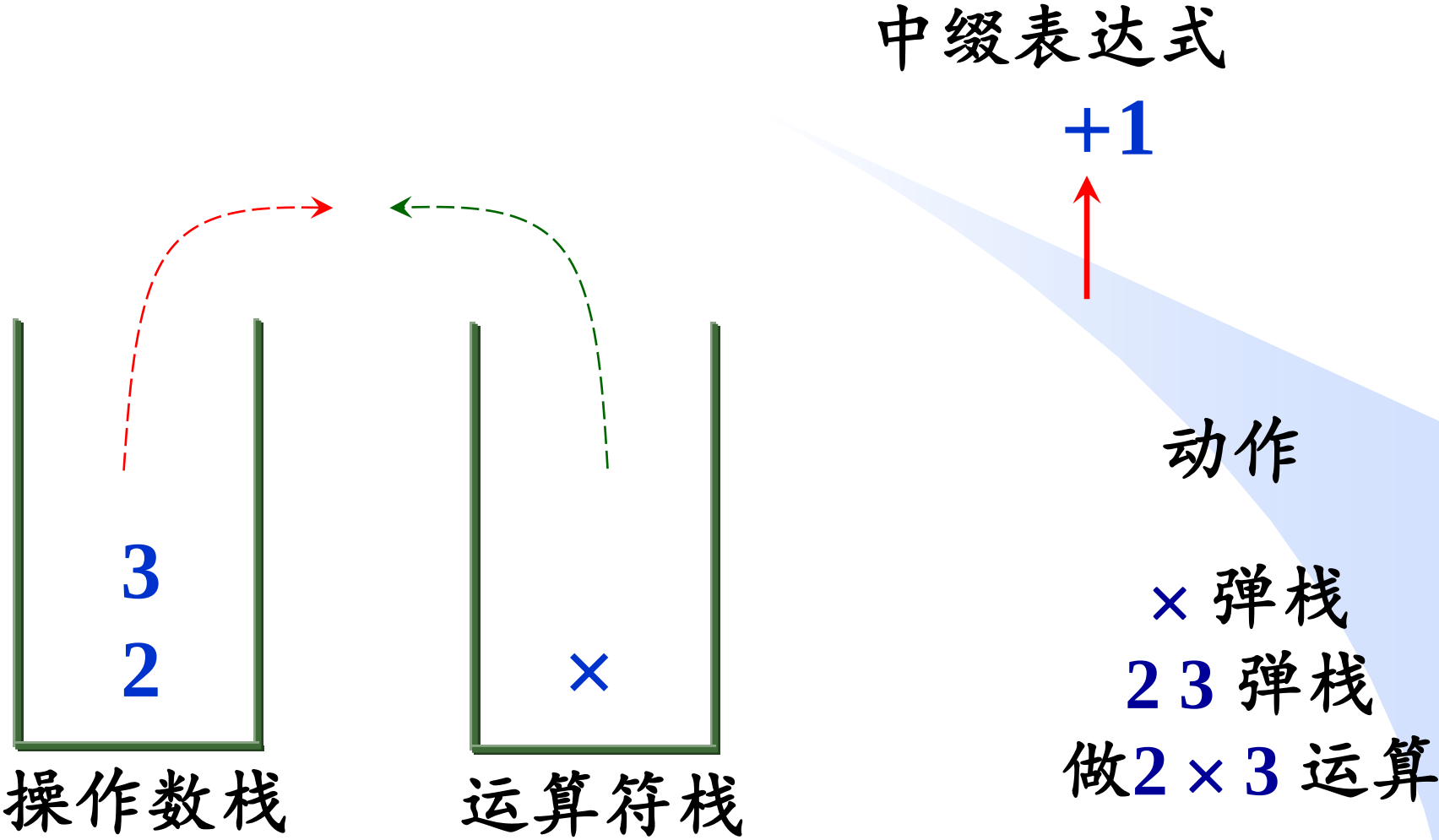
中缀表达式直接求值——举例



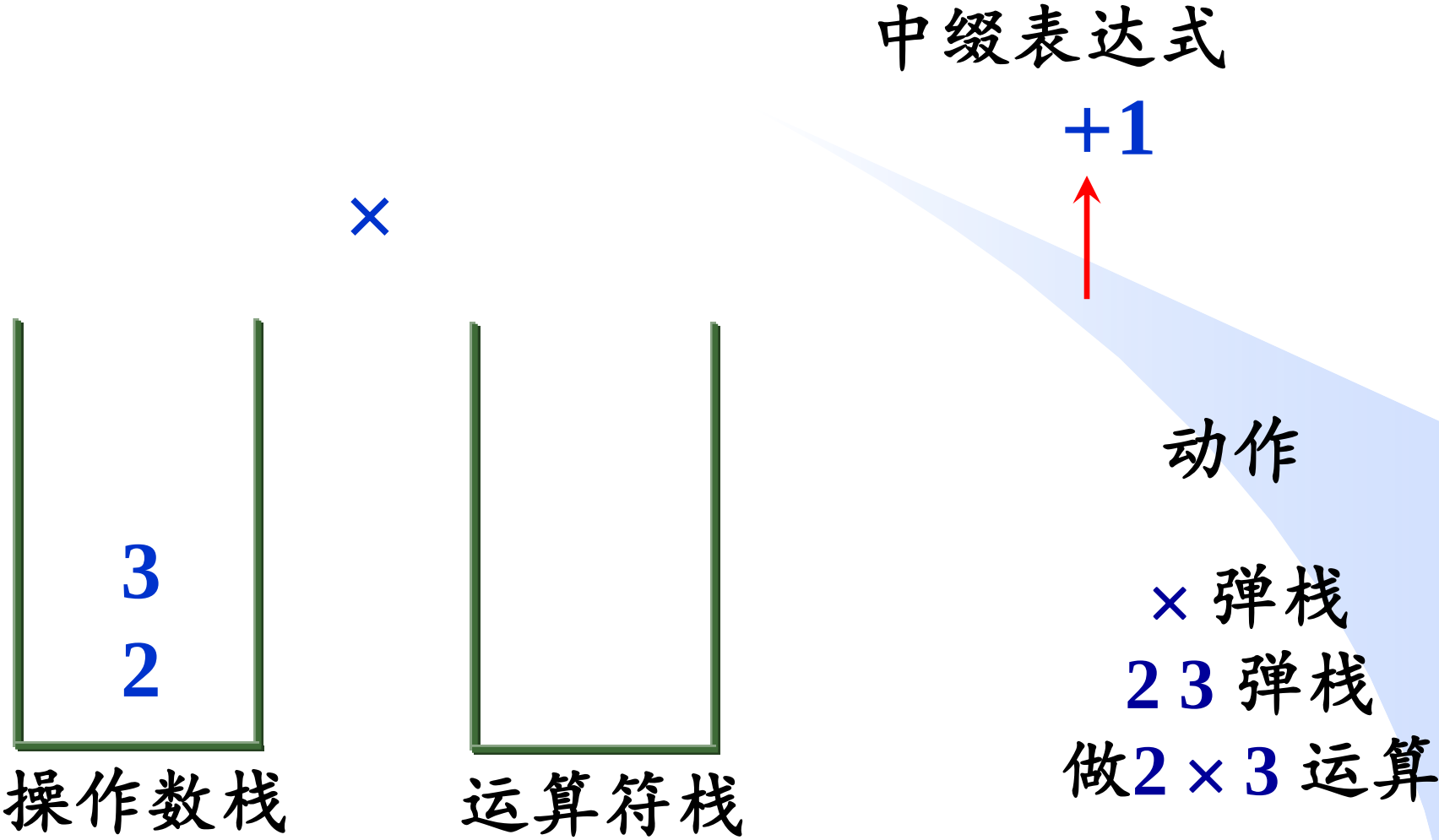
中缀表达式直接求值——举例



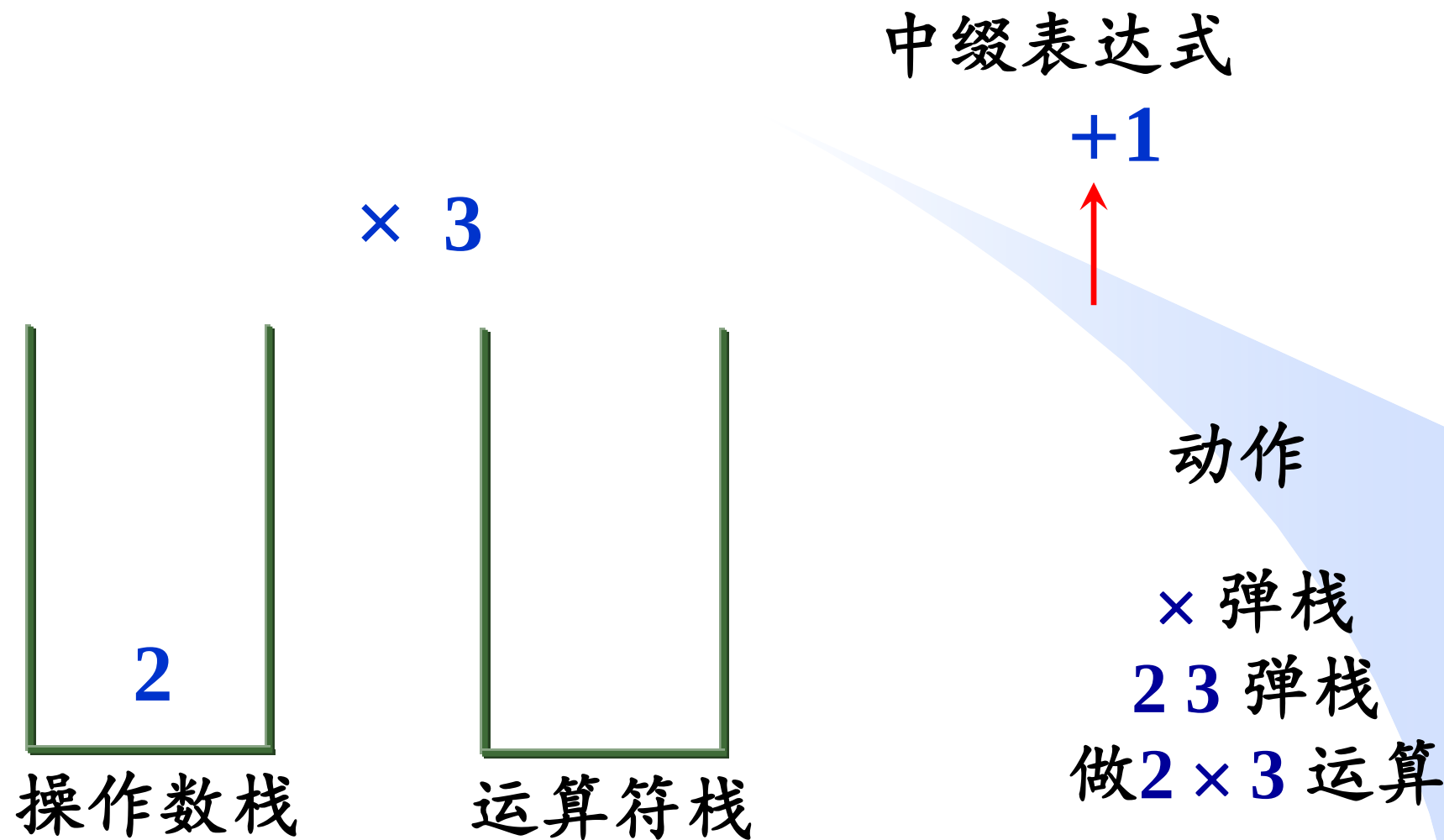
中缀表达式直接求值——举例



中缀表达式直接求值——举例



中缀表达式直接求值——举例



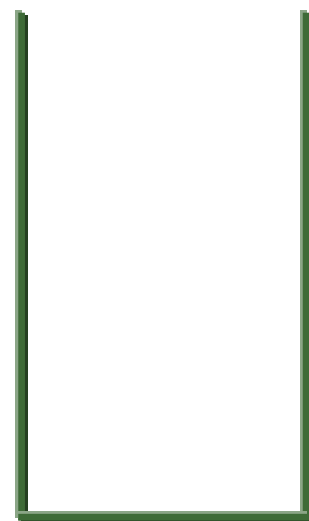
中缀表达式直接求值——举例

中缀表达式

+1



2 × 3 = 6



操作数栈

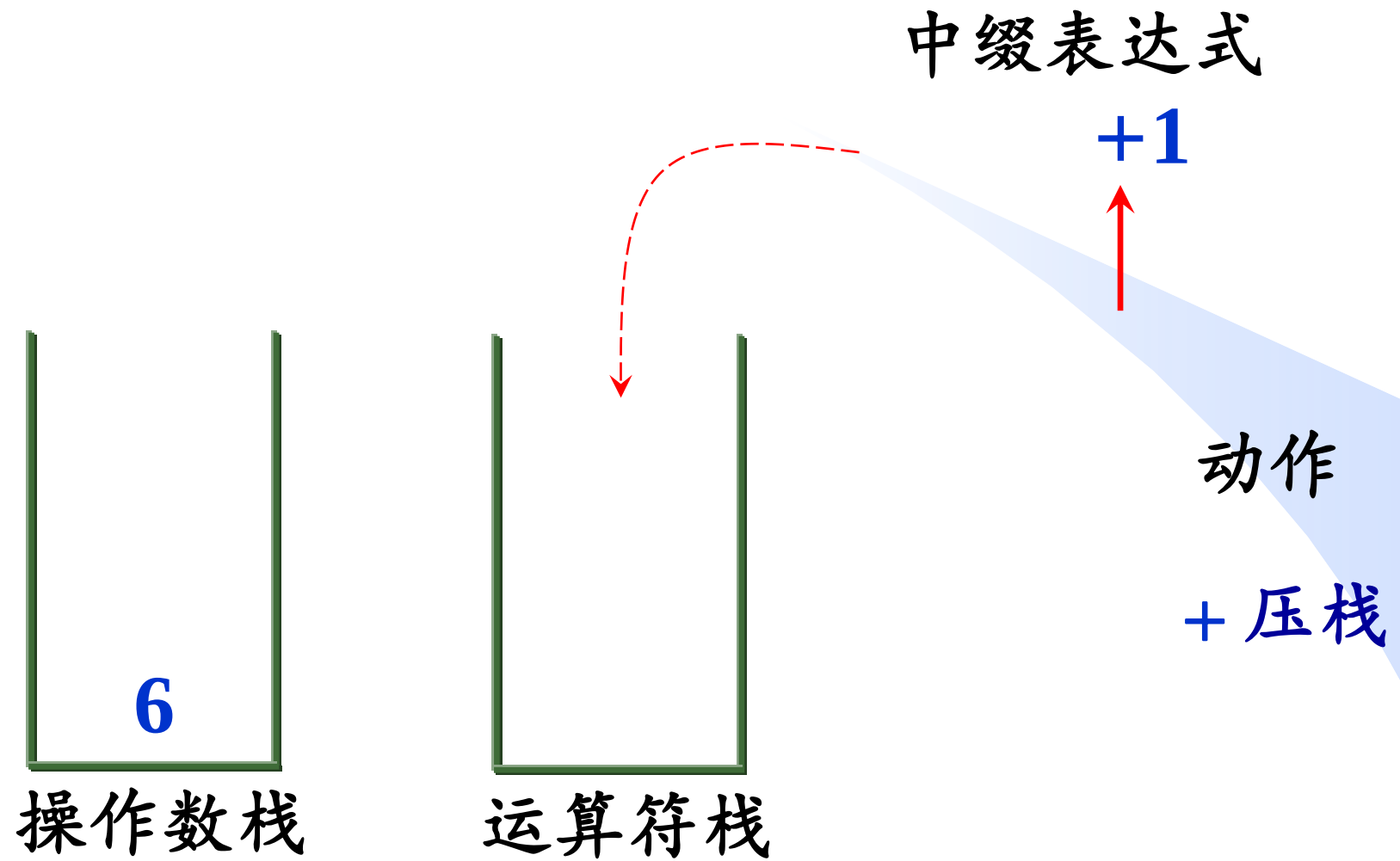


运算符栈

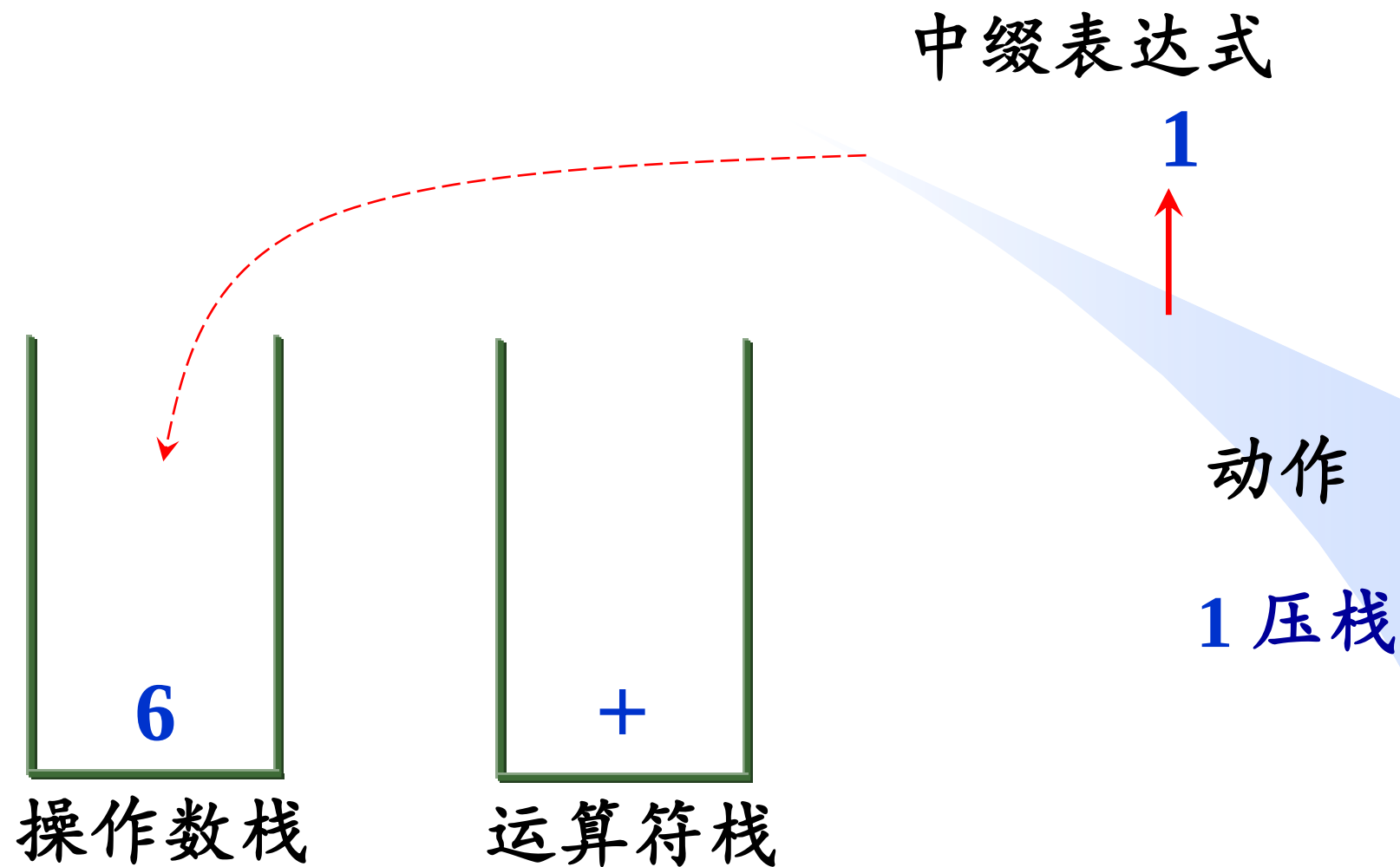
动作

6压栈

中缀表达式直接求值——举例

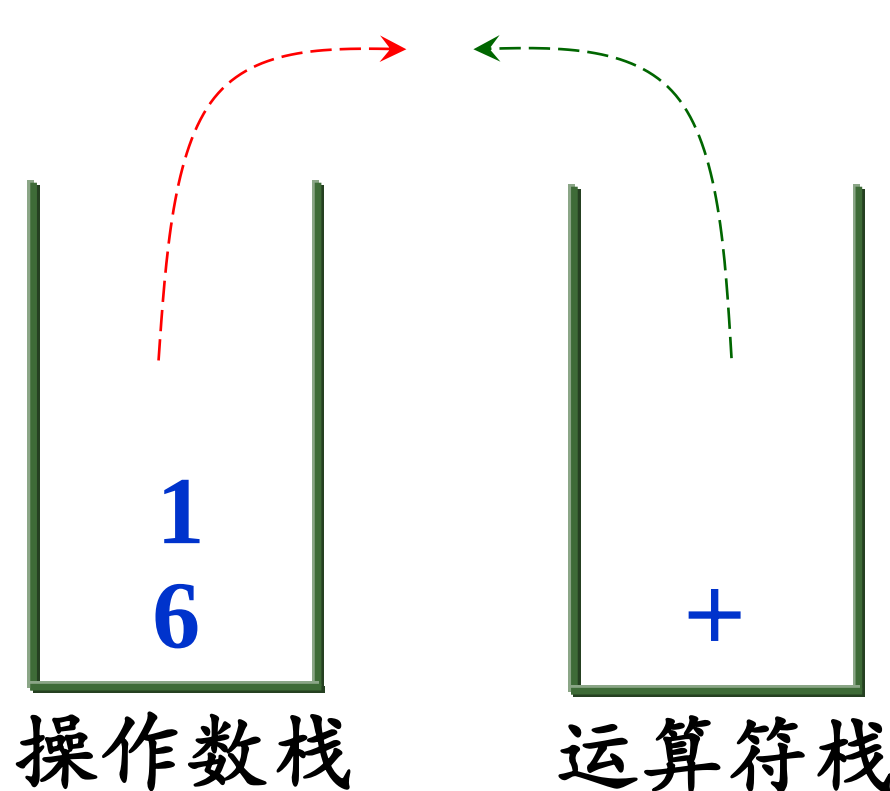


中缀表达式直接求值——举例



中缀表达式直接求值——举例

中缀表达式

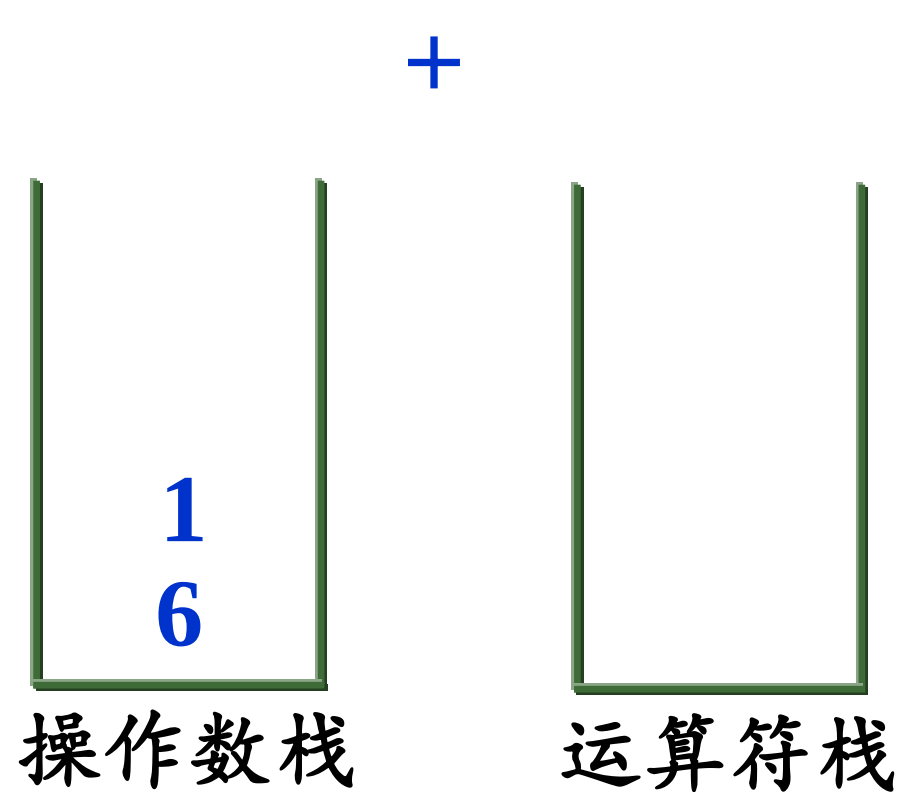


动作

+ 弹栈
6 和 1 弹栈
做6+1运算

中缀表达式直接求值——举例

中缀表达式



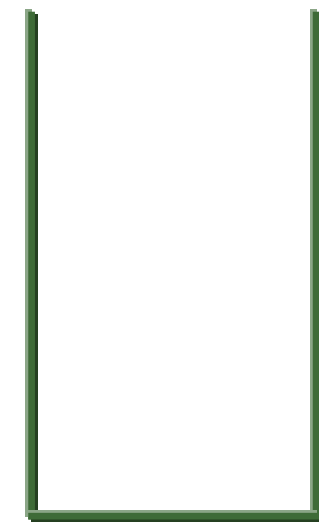
动作

+ 弹栈
6 和 1 弹栈
做6+1运算

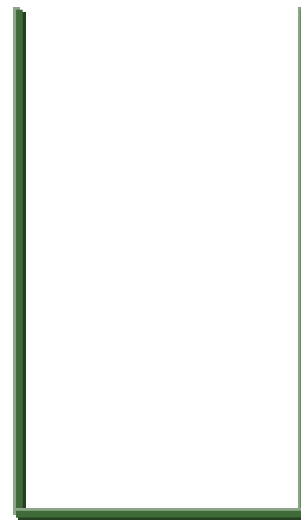
中缀表达式直接求值——举例

中缀表达式

$$6 + 1 = 7$$



操作数栈



运算符栈



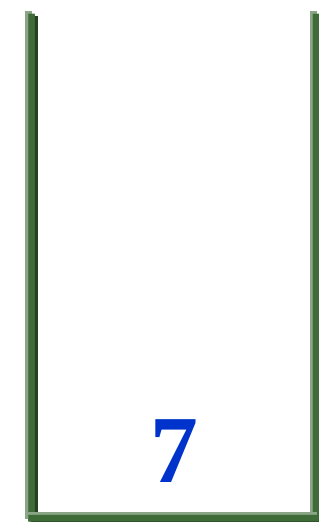
动作

7 入栈

中缀表达式直接求值——举例

算法结束后
操作数栈栈顶即为表达式的值

中缀表达式



操作数栈



运算符栈



动作

7 入栈

中缀表达式直接求值——算法实现

B

实现一个基本的计算器来计算简单的表达式字符串。表达式字符串只包含非负整数，算符 $+$ 、 $-$ 、 $*$ 、 $/$ ，左括号（和右括号）。整数除法需要向下截断。给定的表达式总是有效的。所有的中间结果的范围均满足 $[-2^{31}, 2^{31} - 1]$ 。【字节跳动、华为、谷歌、苹果、微软面试题[LeetCode772](#)】

```
int readnum(string s, int n, int &i){
    int temp = 0;
    while(i < n && s[i] >= '0' && s[i] <= '9'){
        temp = temp * 10 + s[i] - '0';
        i++;
    }
    i--;
    return temp;
}
```

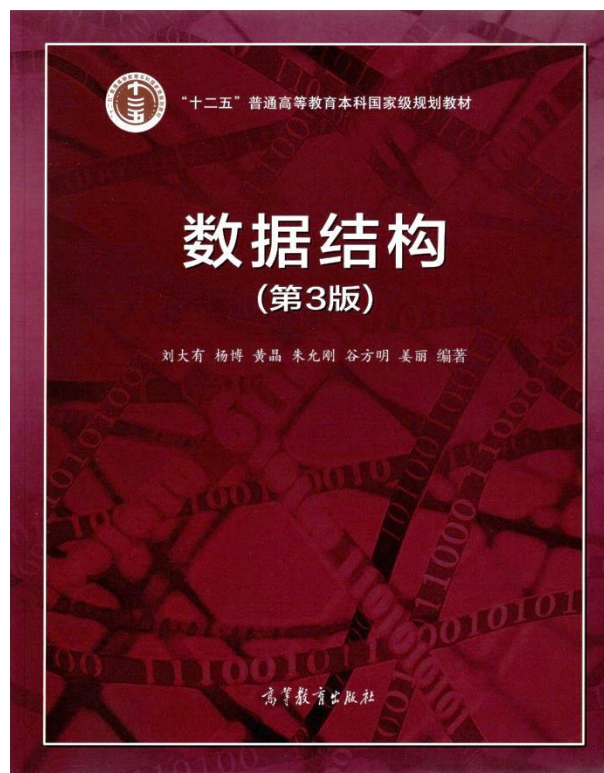
```
void operation(Stack<int> &NUM, Stack<char> &OP){
    char t = OP.pop();
    int b = NUM.pop();
    int a = NUM.pop();
    if (t == '+') NUM.push(a+b);
    else if (t == '-') NUM.push(a-b);
    else if (t == '*') NUM.push(a*b);
    else NUM.push(a/b);
}
```

```
int calculate(string s) { //计算中缀表达式s的值
    Stack<int> NUM;        //操作数栈
    Stack<char> OP;        //运算符栈
    int n = s.size();
    int p[100] = { 0 }; //运算符优先级
    p['+'] = p['-'] = 1;  p['*'] = p['/'] = 2;
    for (int i = 0; i < n; i++) {
        if (s[i]>='0' && s[i]<='9') NUM.push(readnum(s,n,i));
        else if (s[i] == '(') OP.push(s[i]);
        else if (s[i] == ')') {
            while (OP.peek() != '(')
                operation(NUM,OP);
            OP.pop();
        } else {
            while (!OP.empty() && OP.peek() != '(' && p[s[i]]<=p[OP.peek()])
                operation(NUM,OP);
            OP.push(s[i]);
        }
    }
    while (!OP.empty()) operation(NUM,OP);
    return NUM.peek();
}
```




栈及其应用

- 栈的实现
- 进制转换
- 括号匹配
- 表达式求值
- **栈混洗**
- 栈扩容
- 递归与回溯



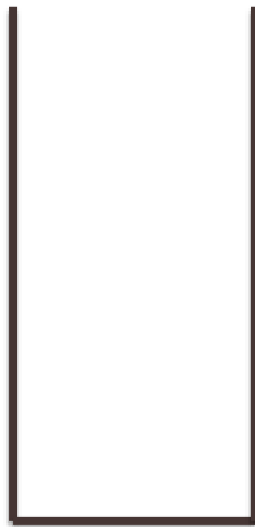
数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

例题

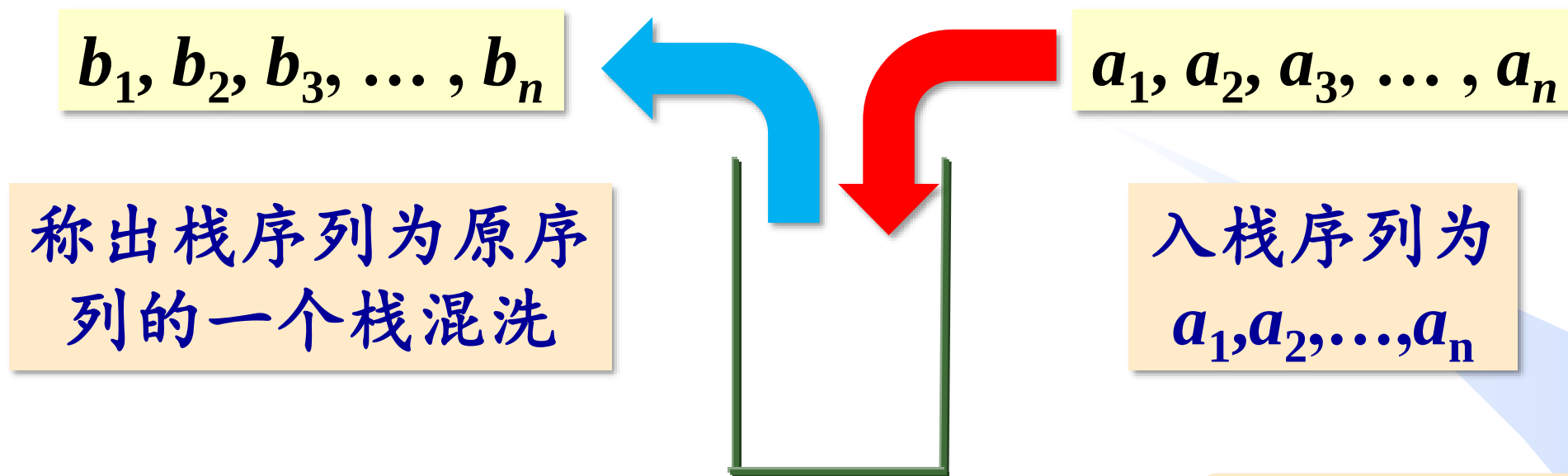
设有四个元素1、2、3、4顺序进栈，在进栈过程中可以出栈，出栈次序错误的排列是？【阿里笔试题】

- A. 1234
- B. 1324
- C. 2341
- D. 4312



并不一定要全部进栈再出栈：进栈的过程中也可以出栈

栈混洗



栈混洗的甄别

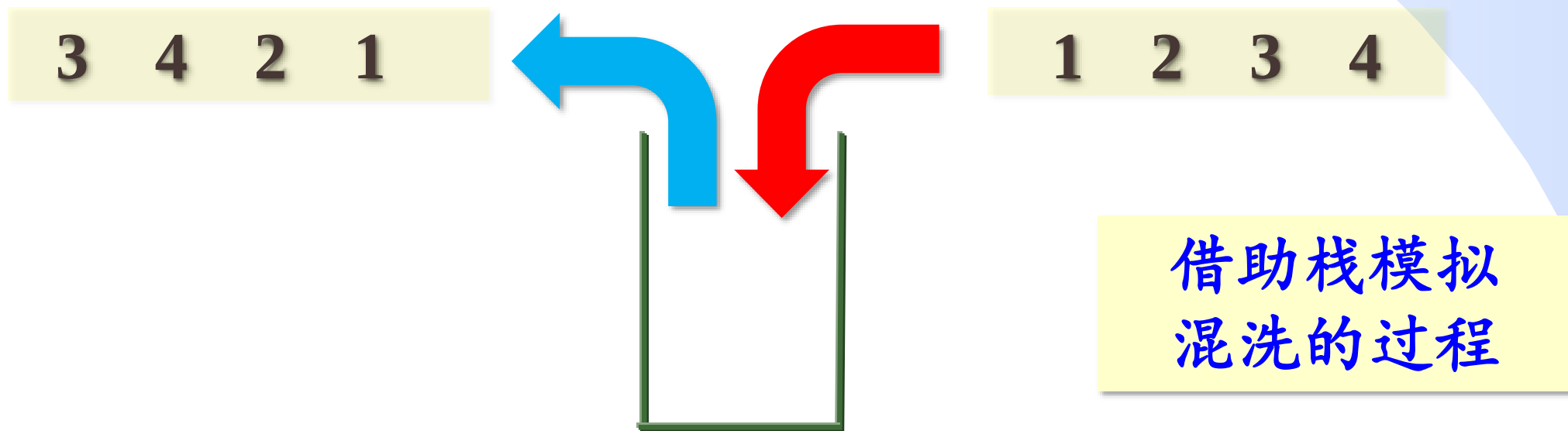
对于输入序列 $a_1, a_2, \dots, a_n$:

- ✓ $a_1, a_2, \dots, a_n$ 的任一排列 $b_1, b_2, \dots, b_n$ 是否为合法的出栈序列?
- ✓ 有多少种可能的出栈序列?

栈混洗的总数

栈混洗的甄别

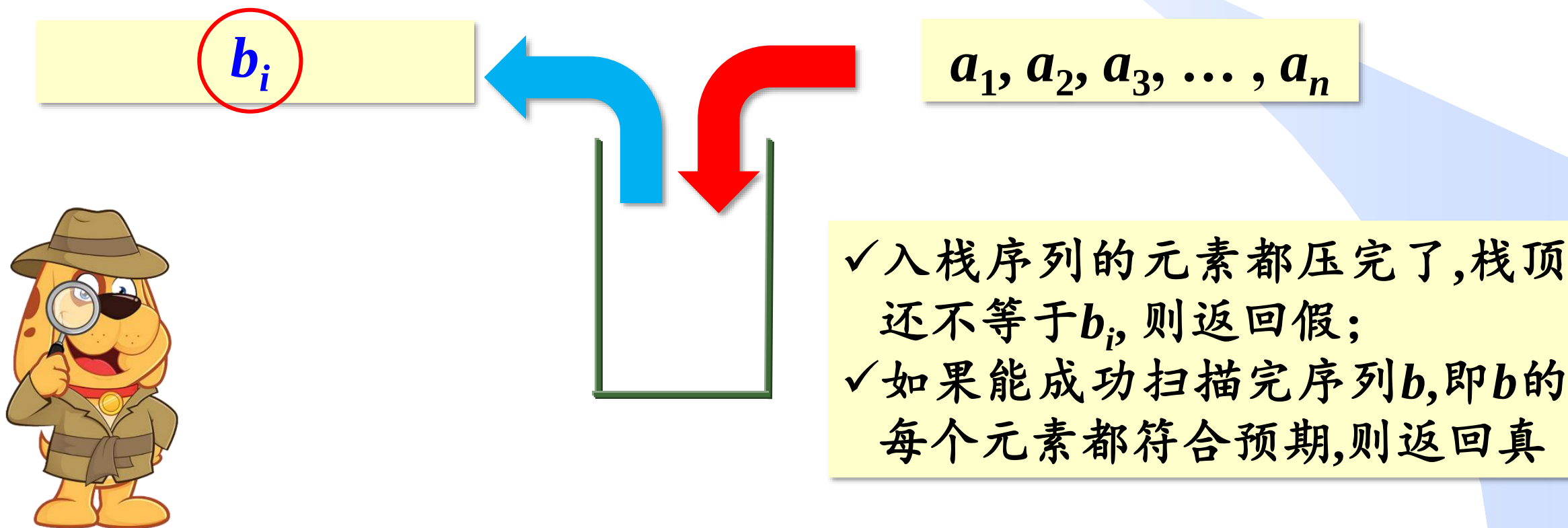
给定入栈序列 $a_0, \dots, a_{n-1}$, 判断序列 $b_0, \dots, b_{n-1}$ 是不是合法的出栈序列 【大厂面试题、北京大学2023年保研机试[LeetCode946](https://leetcode.com/problems/validate-stack-sequences/)、[openjudgeP0450](https://openjudge.pku.edu.cn/problems/p0450/)】



栈混洗的甄别——大致思路

扫描出栈序列，对当前扫描到的元素 b_i ：

- ✓ 若当前栈顶元素等于 b_i ，符合预期，继续扫描序列 b
- ✓ 否则，把入栈序列的元素逐个压栈，直至栈顶等于 b_i

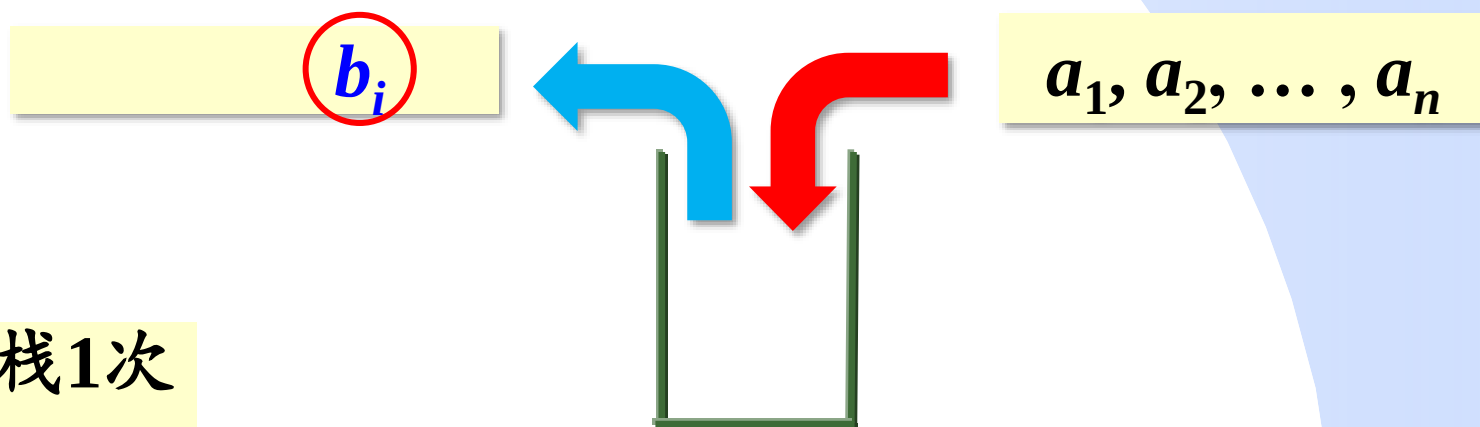


栈混洗的甄别

```

bool check(int a[], int b[], int n){
    //入栈序列 $a_0, \dots, a_{n-1}$ , 判断出栈序列 $b_0, \dots, b_{n-1}$ 是否合法
    Stack<int> s; int k=0; //通过k扫描入栈序列
    for(int i=0; i<n; i++){ //通过i扫描出栈序列
        while(s.empty() || s.peek() != b[i]) //  $b_i$ 当前预期的出栈元素
            if(k<n) s.push(a[k++]);
            else return false;
        s.pop();
    }
    return true;
}

```



每个元素最多进栈1次，出栈1次
时间复杂度 $O(n)$

课下思考

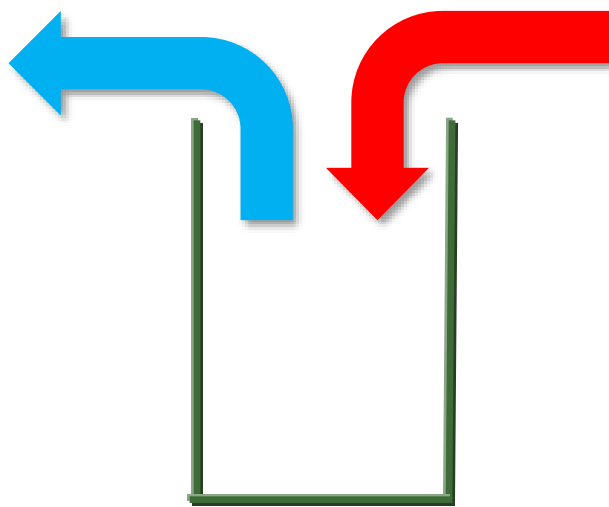
元素1, 2, 3, 4, 5, 6 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，则不可能得到的出栈序列是：【2018年考研题全国卷】

A. 1 2 5 6 4 3

B. 2 3 4 5 6 1

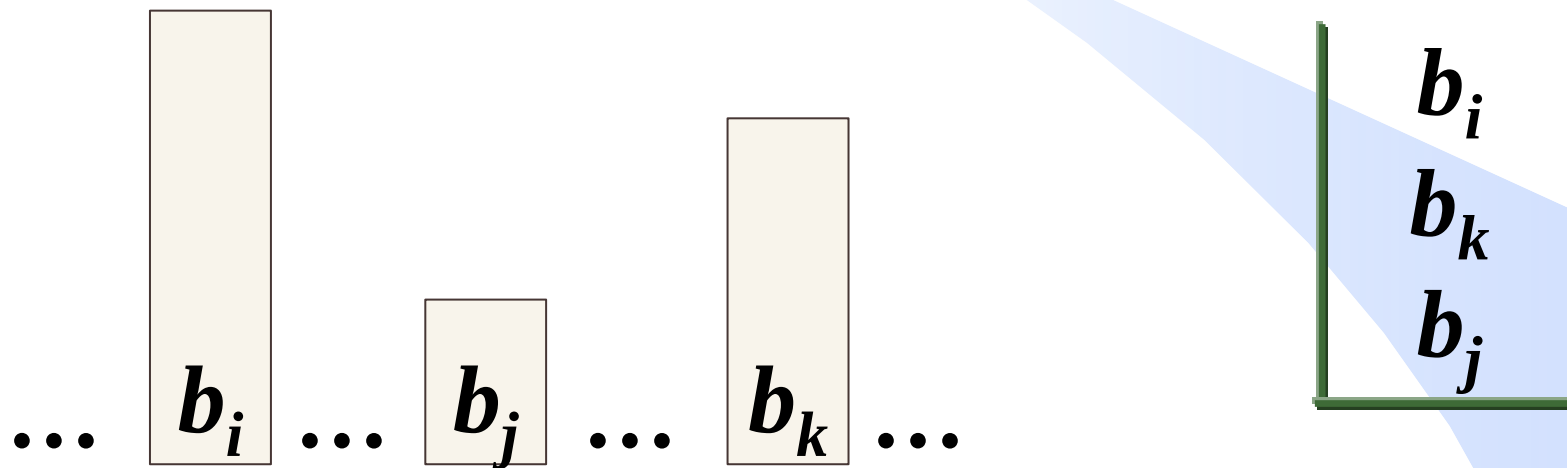
C. 3 4 5 6 1 2

D. 6 5 4 3 2 1



栈混洗的甄别

- 若入栈序列为 $1, 2, \dots, n$ ，对于出栈序列中 $1 \leq i < j < k \leq n$ 位置的3个元素



- 如果 $b_i > b_k > b_j$ ，则必为非法出栈序列。【312模式】
- 充要条件。

栈混洗总数

- 每一个栈混洗，都对应于栈的 n 次push和 n 次pop操作构成的一个序列
- $1243 \rightarrow \text{push, pop, push, pop, push, push, pop, pop}$
- $4321 \rightarrow \text{push, push, push, push, pop, pop, pop, pop}$

n 个元素的栈混洗

n 对括号的合法匹配

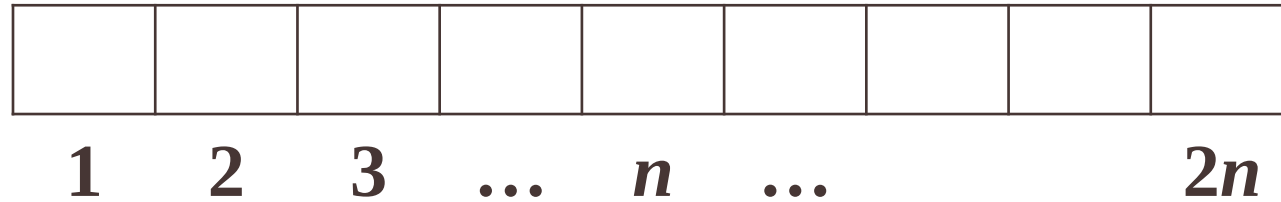
n 个元素有多少种可能的合法出栈序列

n 对括号有多少种可能的合法匹配序列

栈混洗总数

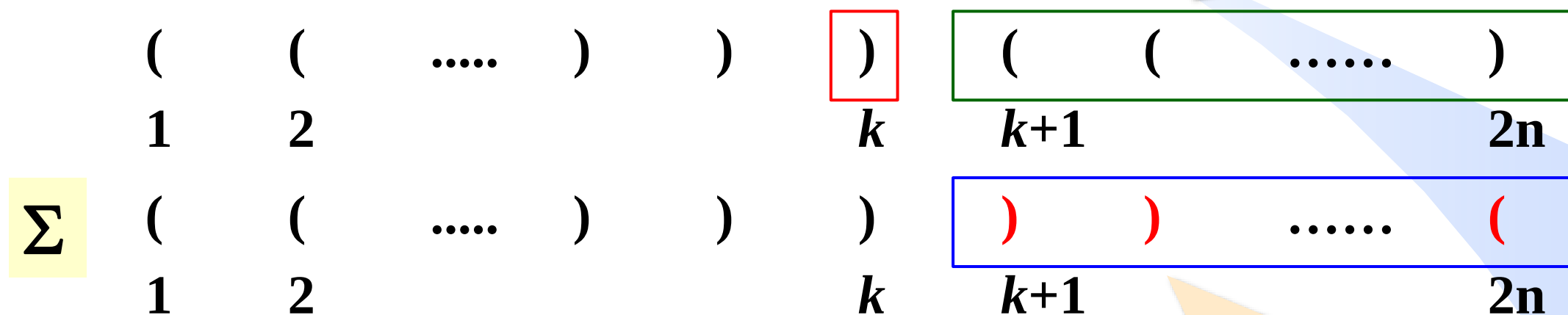
➤ n 对括号的总序列数:

n 个“(”和 n 个“)”的总排列数: C_{2n}^n



栈混洗总数

➤ n 对括号的不合法序列数:



给定一个非法序列，若其第1个匹配失败的位置为 k ，对 k 之后的序列取“反”，即：左括号变成右括号，右括号变成左括号。进而得到1个新序列（不妨称为 Σ 序列）。

栈混洗总数

➤ n 对括号的不合法序列数:

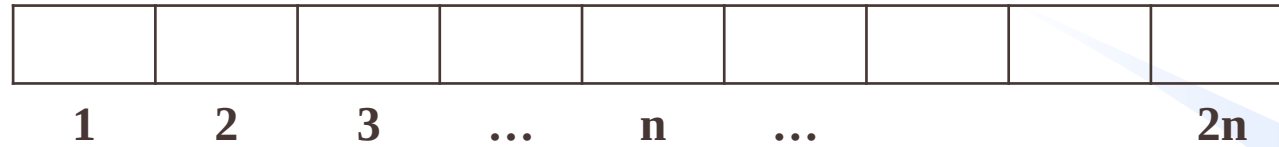
((.....)) ((.....)
 1 2 k k+1 2n

Σ ((.....)))) (
 1 2 k k+1 2n

- Σ 序列右括号比左括号多2个，即有 $n-1$ 个左括号和 $n+1$ 个右括号。现在已将一个非法序列对应到唯一一个 Σ 序列。
- 对于一个 Σ 序列，找第1个匹配失败的位置 k ， k 之后的序列左括号变成右括号，右括号变成左括号，得到一个 n 个左括号和 n 个右括号组成的非法序列。即 Σ 序列可以唯一对应一个非法序列。
- 上述说明： n 对括号的非法序列和 Σ 序列（ $n-1$ 个左括号 $n+1$ 个右括号组成的序列）**一一对应**。

栈混洗总数

➤ $n-1$ 个左括号 $n+1$ 个右括号组成的序列数



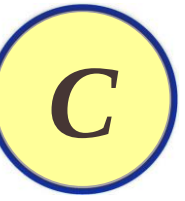
$$C_{2n}^{n-1}$$

➤ n 对括号的总序列数: C_{2n}^n

➤ n 对括号的合法序列数:

$$\begin{aligned} C_{2n}^n - C_{2n}^{n-1} &= \frac{(2n)!}{n!n!} - \frac{(2n)!}{(n-1)!(n+1)!} \\ &= \frac{(2n)!(n+1)}{n!(n+1)!} - \frac{(2n)!n}{n!(n+1)!} = \frac{(2n)!}{n!(n+1)!} \end{aligned}$$

$$\text{Catalan}(n) = \frac{1}{n+1} C_{2n}^n$$



栈混洗总数

- n 个元素的栈混洗总数（可能的合法出栈序列数）
- n 对括号的合法匹配序列数：

$$Catalan(n) = \frac{1}{n+1} C_{2n}^n$$

卡特兰（明安图）数

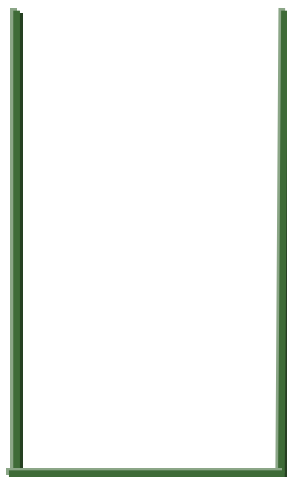
- 以比利时数学家欧仁·卡特兰（Eugène Catalan, 1814-1894）的名字命名。
- 1730年我国清代蒙古族数学家明安图（1692-1763）在发现三角函数幂级数的过程中，更早使用了卡特兰数，见《割圆密率捷法》，后来他的学生在1774年将其完成发表。



练习题

元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是_____。【考研题全国卷，快手校园招聘笔试题】

- A. 3
- B. 4
- C. 5
- D. 6



课下思考

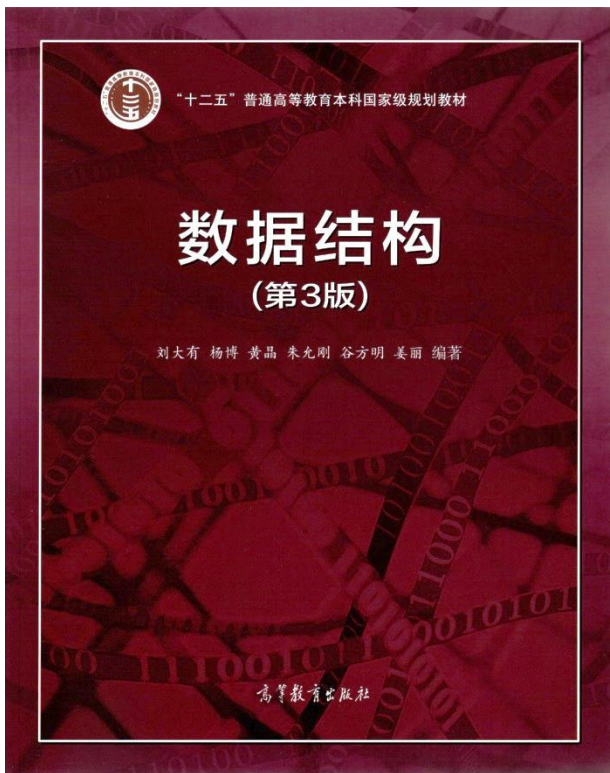
一个栈的入栈序列为 $1, 2, 3, \dots, n$, 其出栈序列是 $p_1, p_2, p_3, \dots, p_n$ 。若 $p_2=3$, 则 p_3 可能取值的个数是_____。【考研题全国卷, 寒武纪招聘笔试题】

- A. $n-3$
- B. $n-2$
- C. $n-1$
- D. 无法确定



栈及其应用

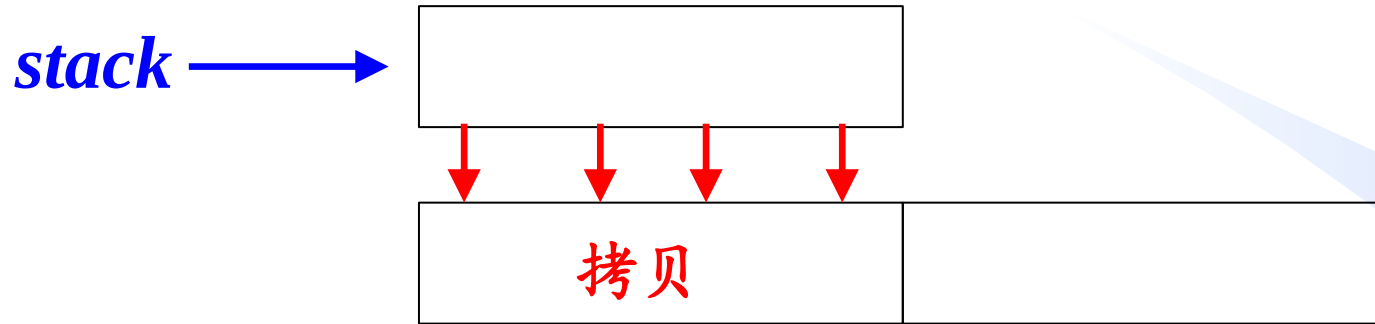
- 栈的实现
- 进制转换
- 括号匹配
- 表达式求值
- 栈混洗
- **栈扩容**
- 递归与回溯



数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

栈扩容



栈满后：

- (1) 申请一个更大的数组
- (2) 将原栈内容拷贝至新数组
- (3) 释放原来的数组
- (4) 将新的数组作为栈空间

栈扩容



栈满后：

- (1) 申请一个更大的数组
- (2) 将原栈内容拷贝至新数组
- (3) 释放原来的数组
- (4) 将新的数组作为栈空间

栈扩容

```
template <class T>
class Stack{
public:
    bool full(){return top==size-1;}
    void push(T K);
    .....
private:
    T *A;
    int size;
    int top = -1;
};
```

一次PUSH操作：最坏情况时间复杂度 $O(n)$ ，但最坏情况不是每次都可能发生，不可能每次压栈都赶上栈满

基本运算：栈元素的赋值

容量倍增策略

```
template <class T>
void Stack<T>::push(T K){
    if(full()) {
        T *B = new T[2*size];
        for(int i=0; i<size; i++)
            B[i] = A[i];
        delete A;
        A = B;
        size *= 2;
    }
    A[++top] = K;
}
```

$T *B = \text{new int}[\text{size}+d]$
容量递增策略

元素拷贝

实际的压栈

容量倍增策略

PUSH次数

1

1

2

4

.....

2^k

共 $n=2^{k+1}$ 次操作

元素拷贝
所需赋值次数 实际压栈
所需赋值次数

0

1

1

1

2

2

4

4

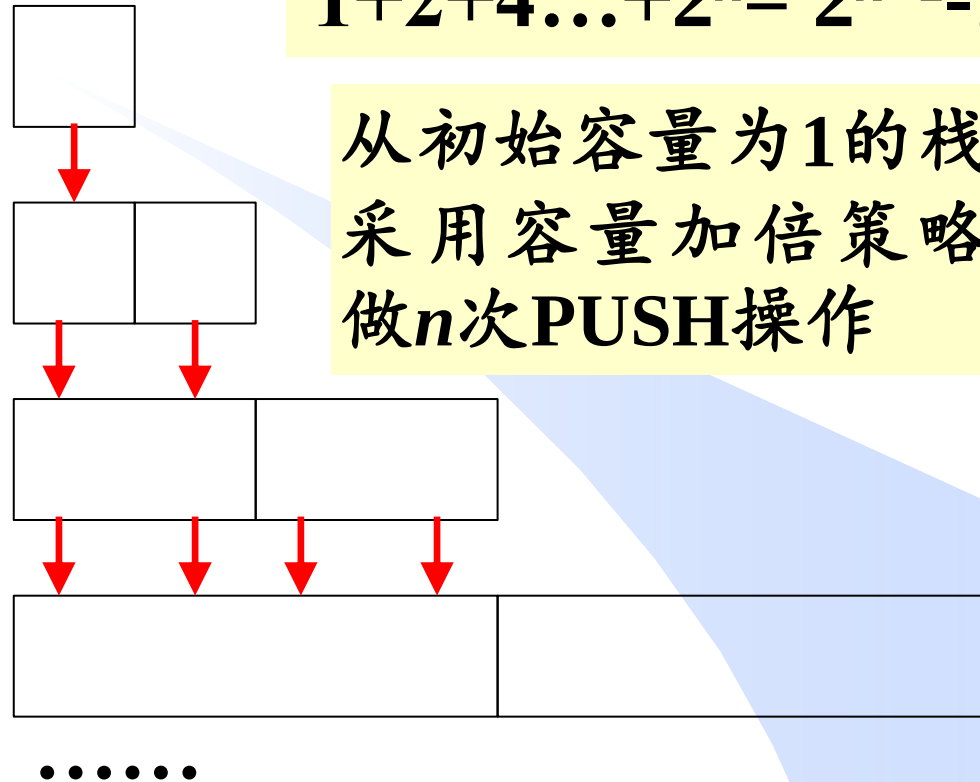
.....

.....

2^k

2^k

总时间 $2^{k+2}-1=O(n)$



n 次Push操作总时间 $O(n)$
每次Push操作均摊时间 $O(1)$

容量递增策略

PUSH次数

d

d

d

d

.....

d

共 $n=kd$ 次操作

元素拷贝
所需赋值次数

0

d

$2d$

$3d$

.....

$(k-1)d$

总时间 $O(n^2)$

实际压栈
所需赋值次数

d

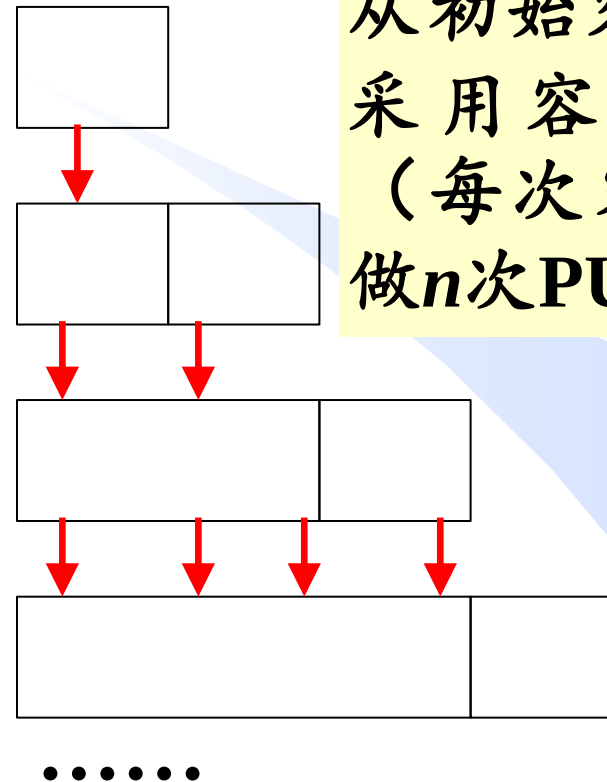
d

d

d

.....

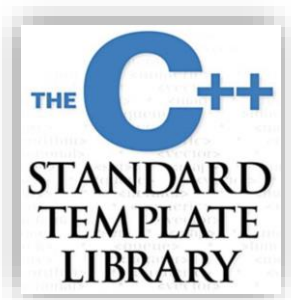
d



从初始容量为 d 的栈,
采用容量递增策略
(每次容量增加 d)
做 n 次PUSH操作

n 次Push操作总时间 $O(n^2)$
每次Push操作均摊时间 $O(n)$

数组扩容



C++ STL: *vector*

容量倍增
策略



每次以1.5倍扩容

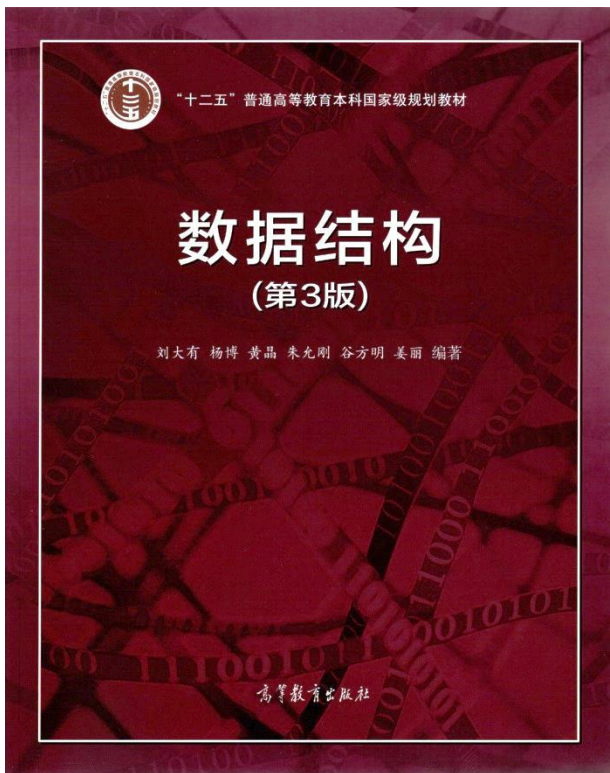


每次以2倍扩容



栈及其应用

- 栈的实现
- 进制转换
- 括号匹配
- 表达式求值
- 栈混洗
- 栈扩容
- **递归与回溯**



数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

递归——栈的经典应用

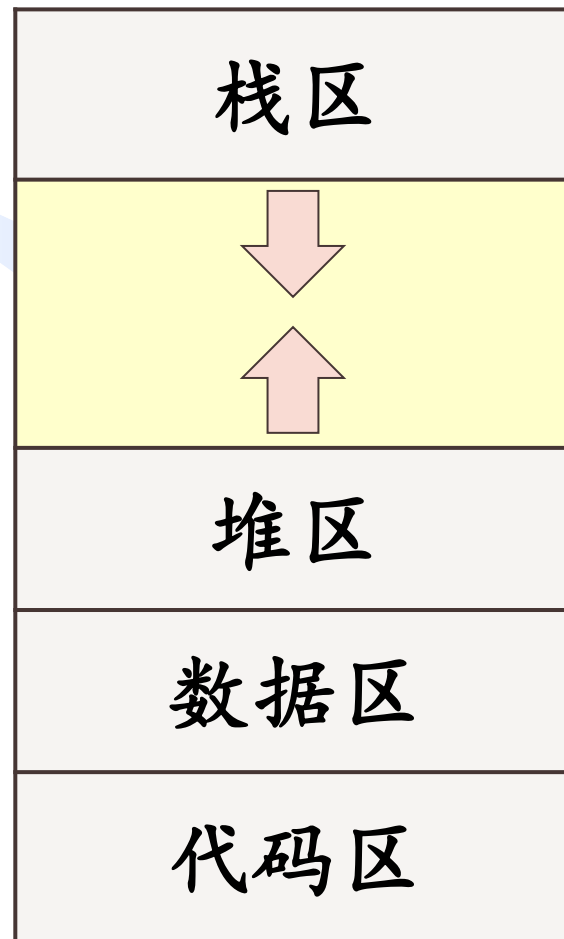
- 操作系统是如何实现函数调用的？
- 如何记录调用与被调用函数间的关系？
- 如何实现函数调用的返回？
- 如何维护同时活跃的所有函数实例？
- 递归函数调用是一种特殊的函数调用形式，因此也通过**栈**实现。

栈

程序在内存里是什么样

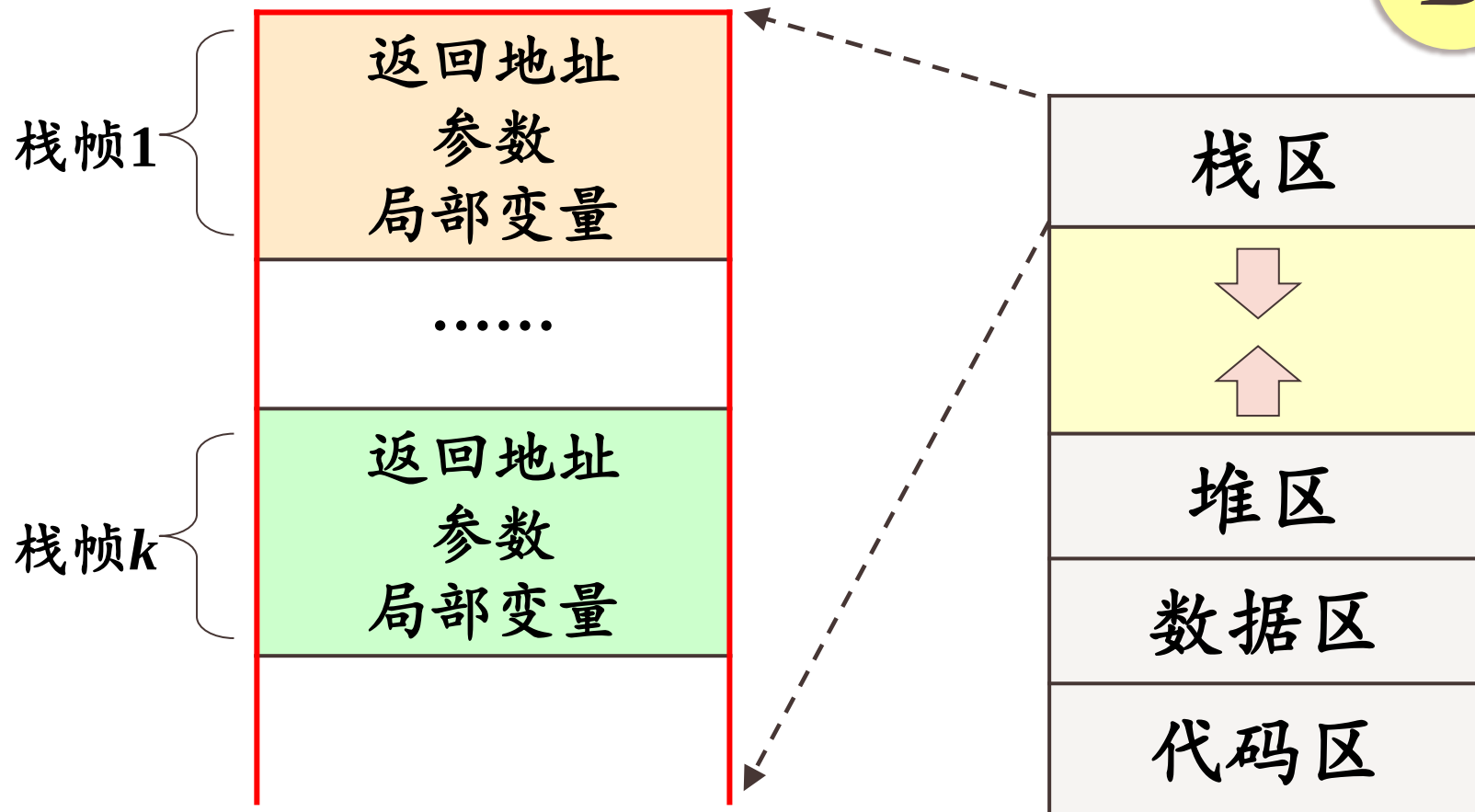
- 代码区：存放程序的可执行代码
- 数据区：存放全局变量、常量等
- 堆区：存放动态分配的空间
- 栈区：存放函数的局部变量、参数、返回地址等

高地址



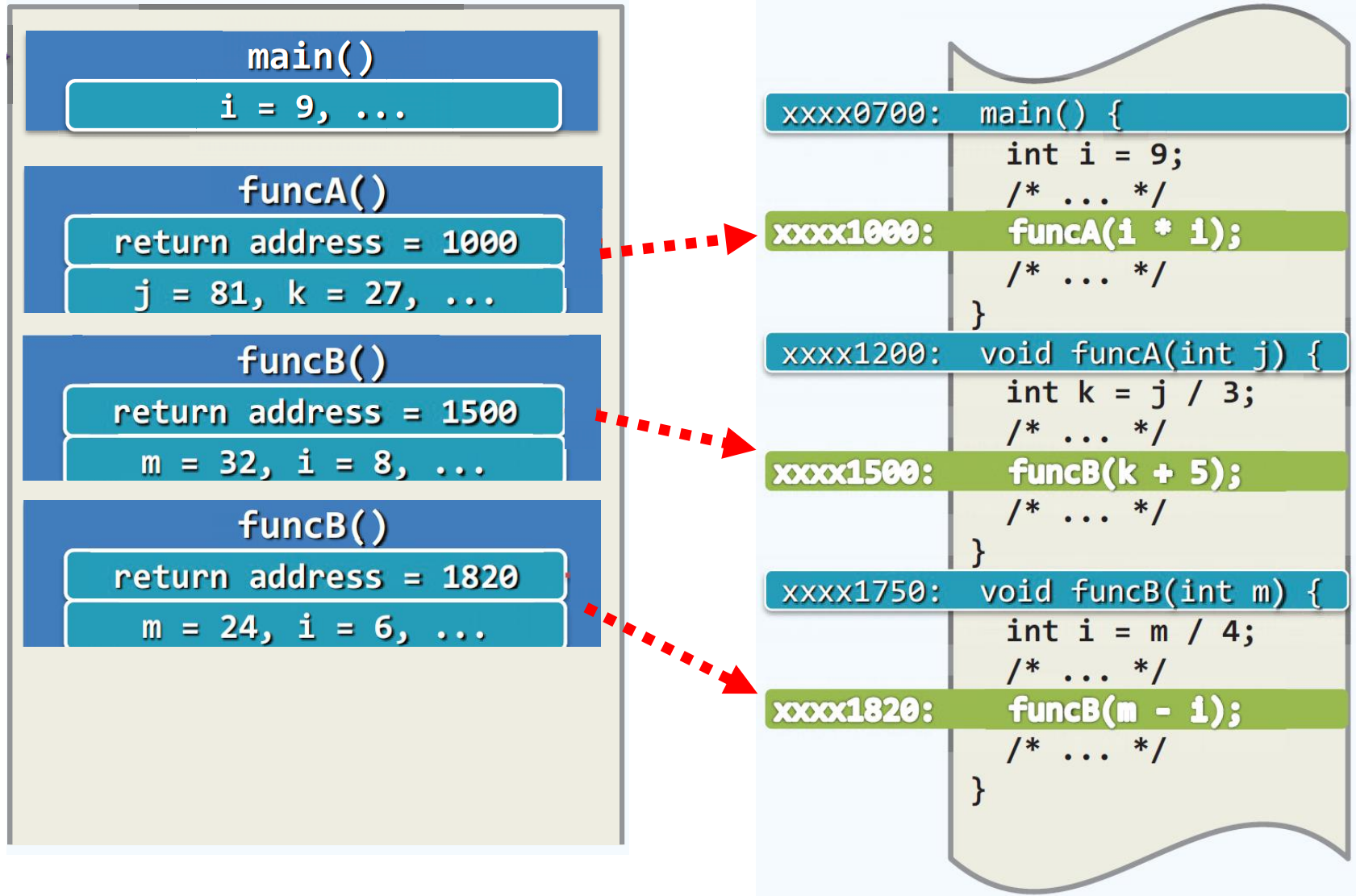
低地址

- ▶ 借助“栈区”可以跟踪属于同一程序的所有函数，记录它们间的调用关系，保证在每一调用执行完毕后可以准确返回。
- ▶ 一次函数调用所需要的信息表示为一个**栈帧**，包含**返回地址**、**参数**、**局部变量**等信息。栈帧存储在栈区。



函数**调用**时执行栈帧的**入栈**操作**保存**本次调用对应的信息。

函数**返回**时执行栈帧的**出栈**操作**恢复**上次调用对应的信息。



递归算法的空间复杂度

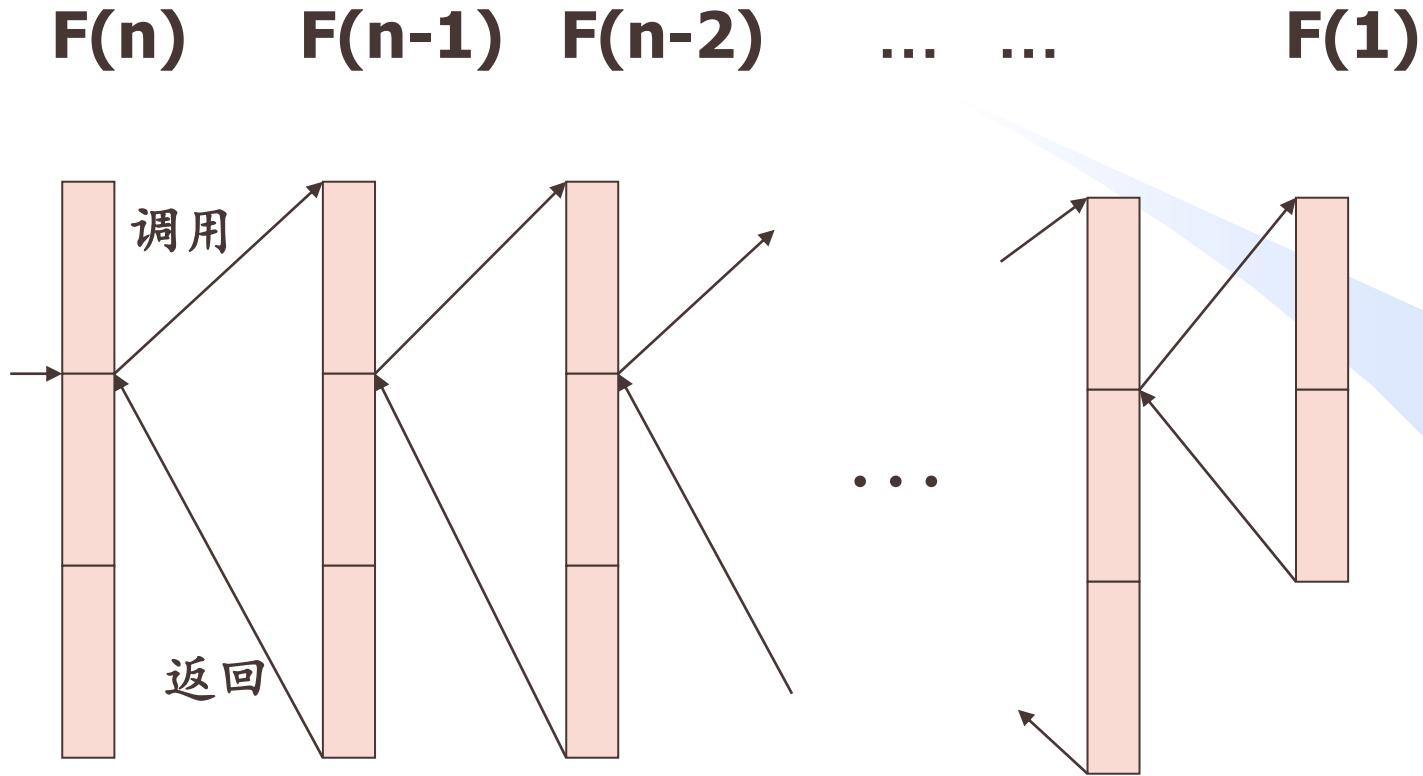
➤ 每一次递归调用都会在生产系统栈中产生一个栈帧，故递归函数的空间复杂度依赖于递归深度。

➤ 例：

```
int F(int n){  
    if(n<=1) return 1;  
    return n*F(n-1);  
}
```

空间复杂度
 $O(n)$

递归函数调用方式：



递归隐患：若递归过深，可能导致系统栈溢出。

递归算法的空间复杂度

➤ 例:

```
int F(int n){  
    if(n==1) return 1;  
    return n+F(n/2);  
}
```

空间复杂度
 $O(\log n)$

算法 BinarySelect (A, i, j, max, min) // 求数组A的最大最小元素

IF $i = j$ **THEN** ($\text{max} \leftarrow \text{min} \leftarrow A[i]$. **RETURN.**) //递归出口, 有1个元素

IF $i = j - 1$ **THEN** //递归出口, 有2个元素

(**IF** $A[i] < A[j]$ **THEN** ($\text{max} \leftarrow A[j]$. $\text{min} \leftarrow A[i]$.)

ELSE ($\text{max} \leftarrow A[i]$. $\text{min} \leftarrow A[j]$.)

RETURN.



)
 $\text{mid} \leftarrow \lfloor (i+j)/2 \rfloor$. //取中值

BinarySelect (A, i , mid , Lmax, Lmin) . //在左侧子数组递归查找

BinarySelect (A, $mid+1$, j , Rmax, Rmin) . //在右侧子数组递归查找

IF $\text{Lmax} > \text{Rmax}$ **THEN** $\text{max} \leftarrow \text{Lmax}$. //合并

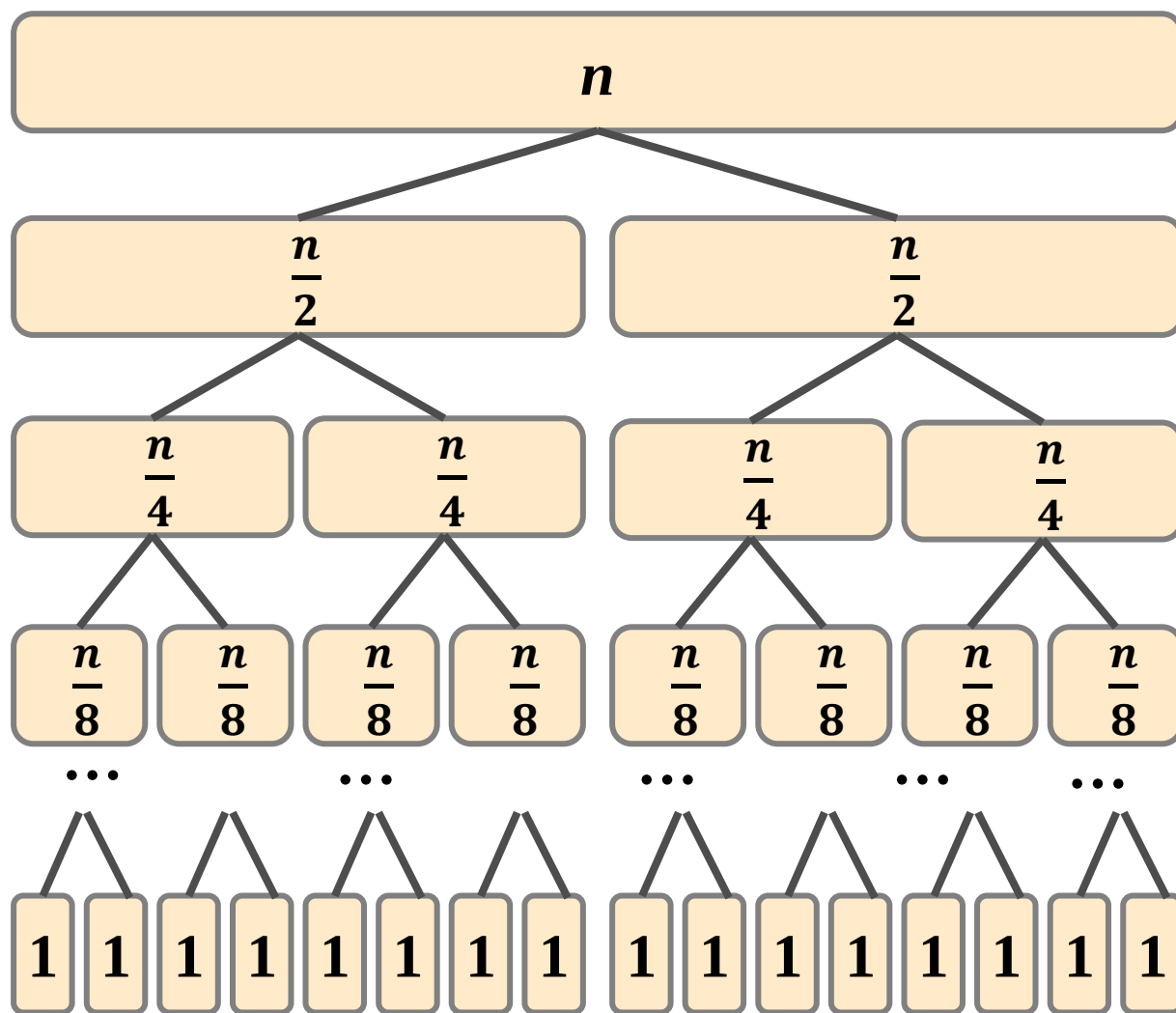
ELSE $\text{max} \leftarrow \text{Rmax}$.

IF $\text{Lmin} < \text{Rmin}$ **THEN** $\text{min} \leftarrow \text{Lmin}$.

ELSE $\text{min} \leftarrow \text{Rmin}$. ■

BinarySelect算法空间复杂度

B



每层递归都把数组分
成长度相等的两半

$\log n$

空间复杂度
 $O(\log n)$

诺亚·泰

根据穿越小说鼻祖《你们这些还魂尸》改编

PREDESTINATION

1月9日全国公映

自身产生自身的闭环烧脑递归

澳大利亚银屏机构 澳大利亚昆士兰银屏机构 出品 中国电影集团公司 进口 中国电影股份有限公司 发行 中国电影股份有限公司 译制

 中国移动  中国联通  中国电信  中国网通  中国宽带  中国教育  中国财经  中国体育  中国文化  中国新闻  中国娱乐  中国教育  中国财经  中国体育  中国文化  中国新闻  中国娱乐  中国教育  中国财经  中国体育  中国文化  中国新闻  中国娱乐  中国教育  中国财经  中国体育  中国文化  中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育 中国财经 中国体育 中国文化 中国新闻 中国娱乐 中国教育

L'ATLANTIQUE. SE
TROUVENT PRIS DANS DE
TERRIBLES CONDITIONS
CLIMATIQUES. ALORS QU'ILS
TENTENT D'ABORDER UN
AUTRE BATEAU, UNE
CATASTROPHE MARITIME
JAMAIS VUE SE
PRODUIT. DANS CE
CONTEXTE, UNE FEMME
MENTALEMENT DÉRANGÉE
VIT CETTE EXPÉRIENCE AU
TRAVERS DE SES TROIS
DIFFÉRENTES
PERSONNALITÉS.

MICHAEL DORMAN
(DAYBREAKERS)

RACHAEL CARPANI
(TRUE BLOOD)

LIAM HEMSWORTH
(PRÉDICTIONS)

PRIX MAD MOVIE
DE L'INÉDIT VIDÉO
GERARDMER 2011

DURÉE 99 Minutes

LANGUE & SOUS-TITRES

Français, Anglais

FORMAT
IMAGE 185 format respecté
compatible (16/9)

PISTE
SON français 5.1 Dolby Digital / 5.1 DTS.
anglais 5.1 Dolby Digital / 5.1 DTS.



TRIANGLE

恐怖游轮 2009

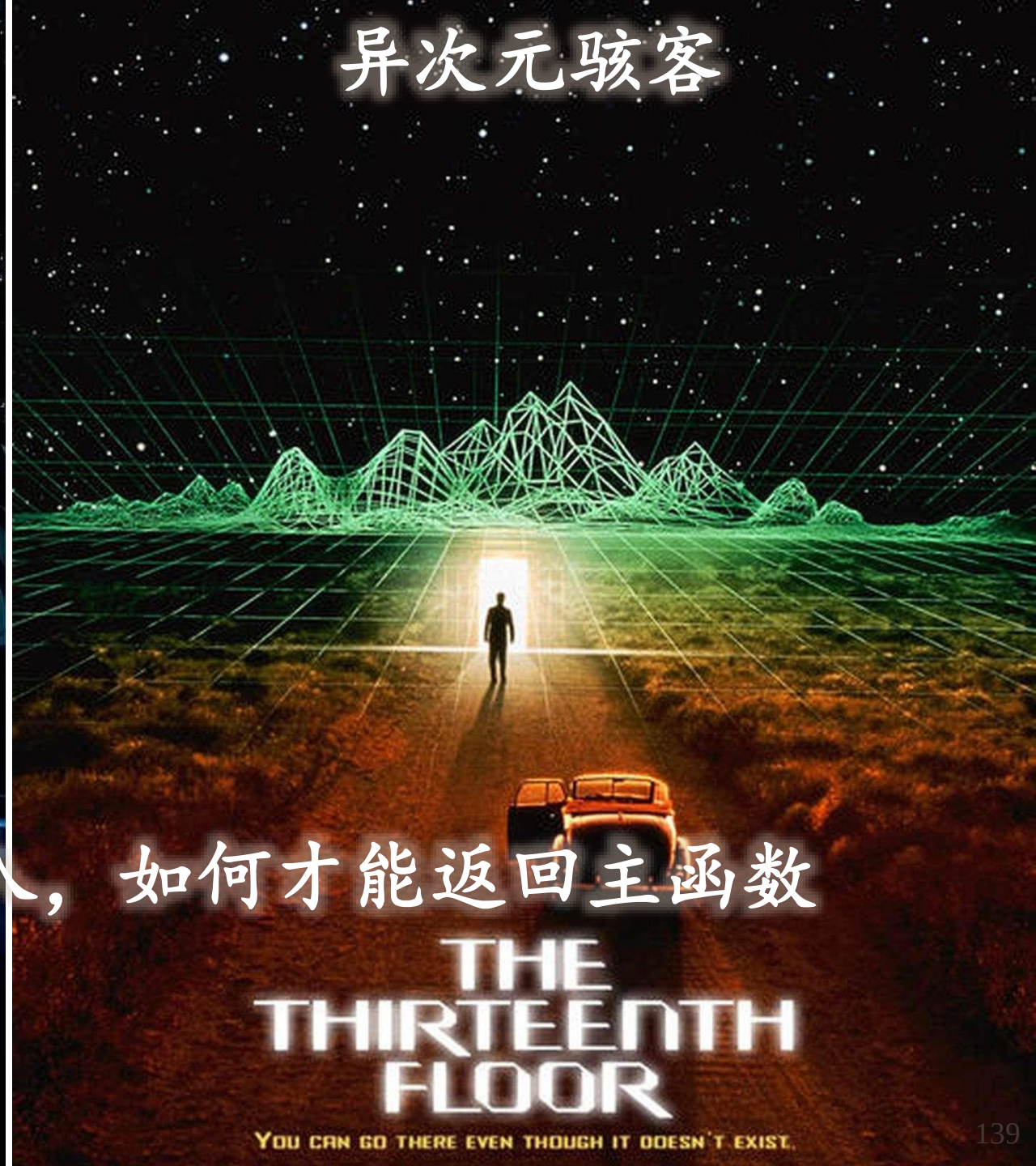
UN FILM DE CHRISTOPHER SMITH LE RÉALISATEUR DE CREEP ET DE SEVERANCE
TRIANGLE

递归函数栈帧大混合
如何才能终结一切

LAISSEZ LA FRAYEUR



递归调用层层深入，如何才能返回主函数

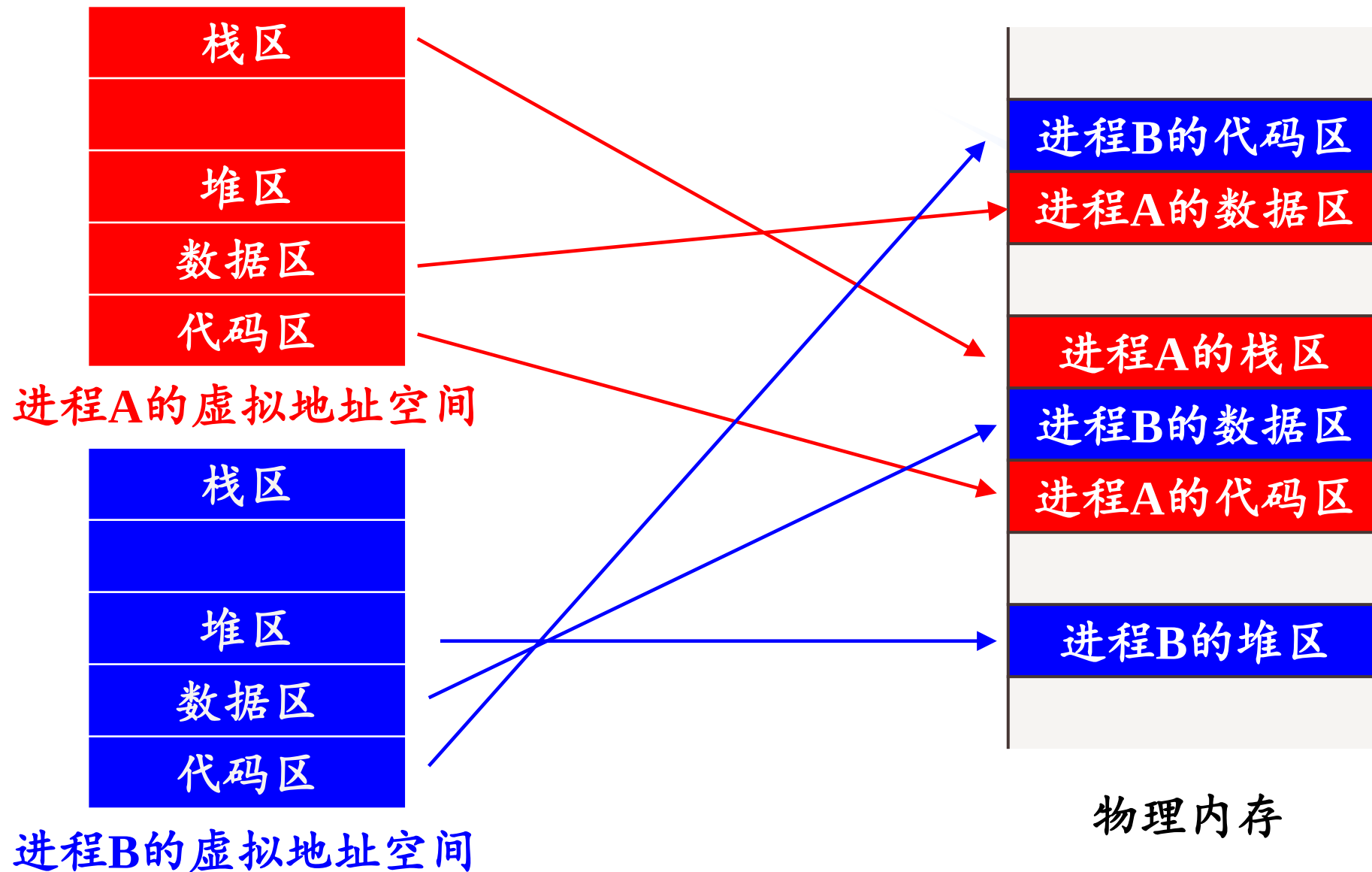


异次元骇客

THE
THIRTEENTH
FLOOR

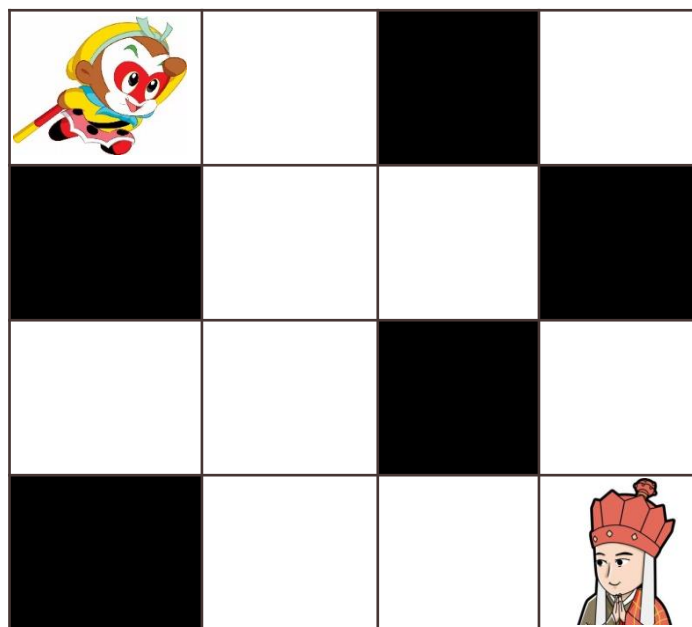
YOU CAN GO THERE EVEN THOUGH IT DOESN'T EXIST.

程序在内存里是什么样



回溯算法-迷宫寻径

- 给定 $n \times m$ 的迷宫，从起始位置 (sx, sy) 开始每步走一格，可移动至上、下、左、右任意一个相邻的格子，不能斜着走，有障碍物的格子不能走。
- 如何能走到目标位置 (tx, ty) ？输出所有可能的方案（保证起点和终点没有障碍物）。



0	0	1	0
1	0	0	1
0	0	1	0
1	0	0	0



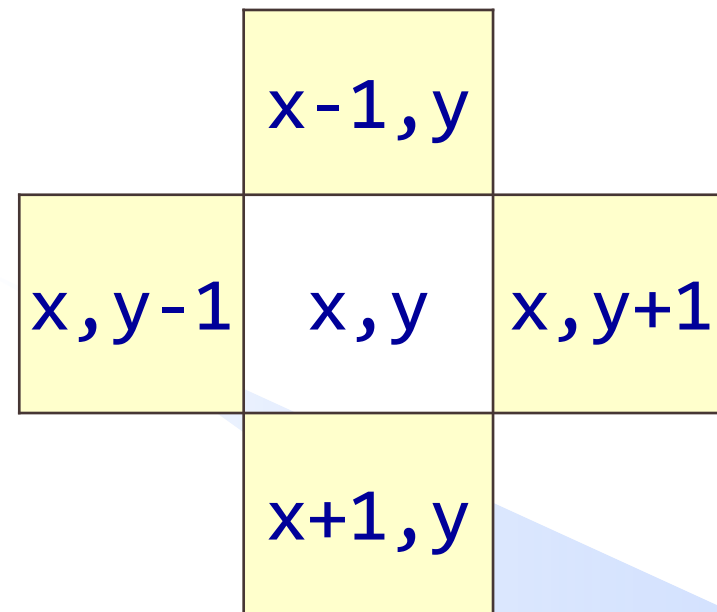
- 解：动作序列

类似题目：

[洛谷P1605](#)

[洛谷B3625](#)

动作



- 0—上: $dx[0]=-1$, $dy[0]=0$
- 1—右: $dx[1]=0$, $dy[1]=1$
- 2—下: $dx[2]=1$, $dy[2]=0$
- 3—左: $dx[3]=0$, $dy[3]=-1$

`int dx[4]={-1,0,1,0}; int dy[4]={0,1,0,-1};`

在 (x, y) 处执行动作 i , 到达的新位置 (nx, ny) 为

$nx = x + dx[i];$

$ny = y + dy[i];$

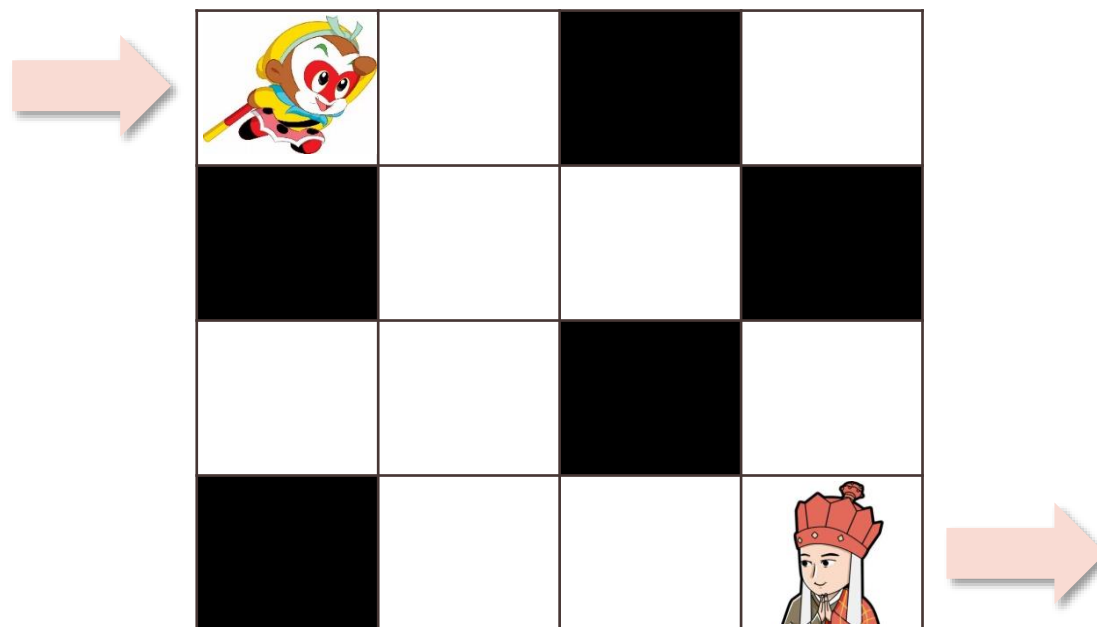
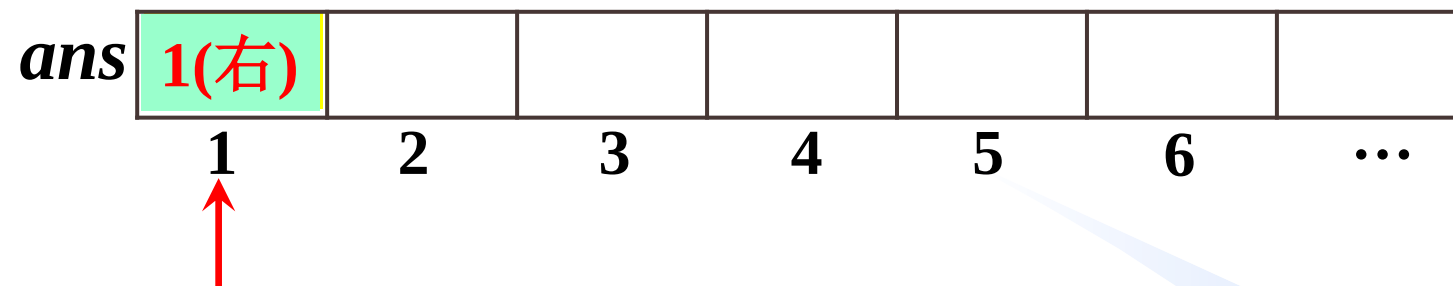
准备工作

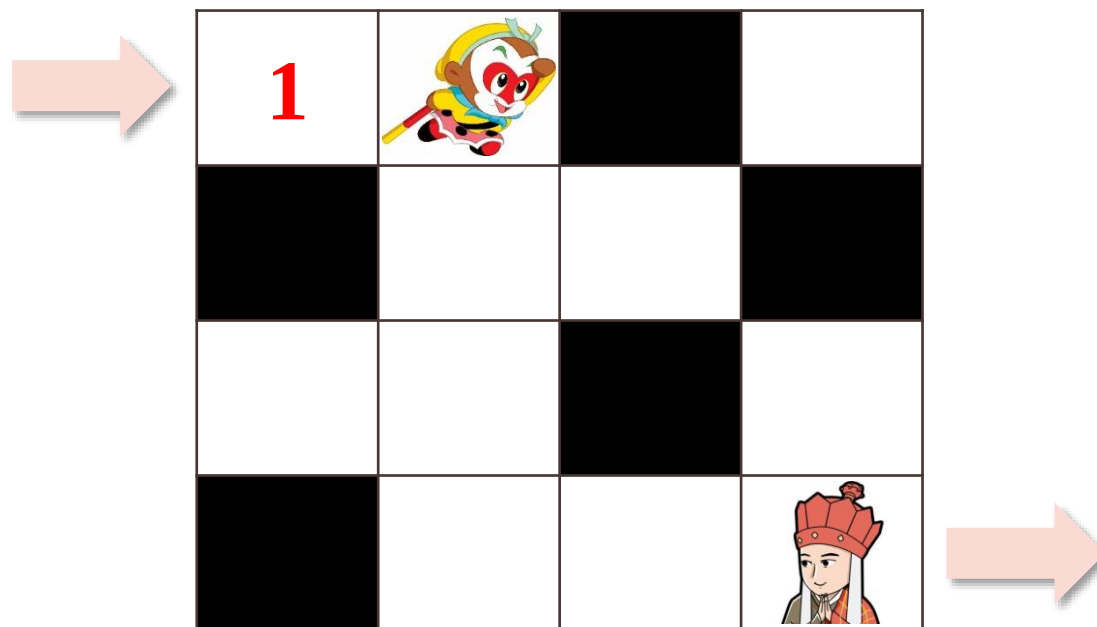
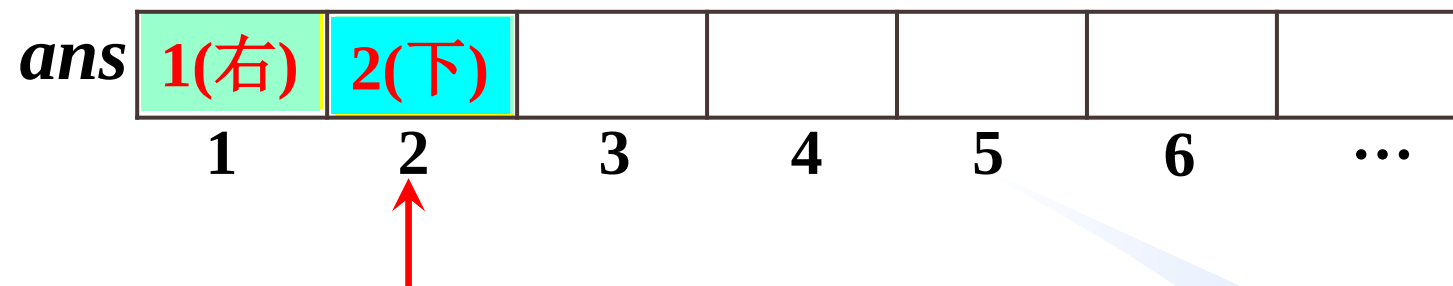
```

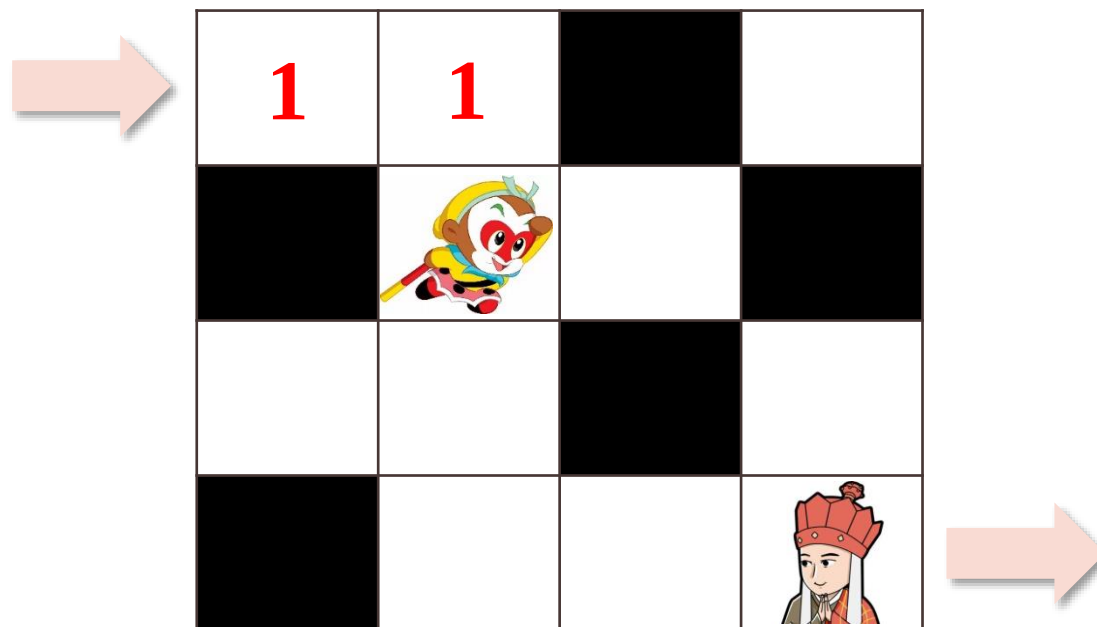
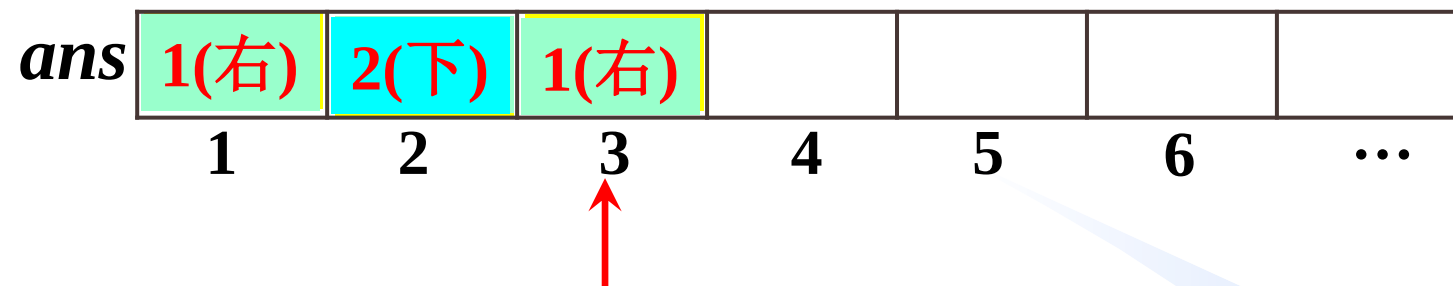
const int maxsize=110;
int map[maxsize][maxsize]; //地图：空位0，障碍 1
int vis[maxsize][maxsize]={0}; /*用于标记走过的格子，规模与
map相同:vis[i][j]=0表示点(i,j)没被访问过, vis[i][j]=1表示点(i,j)被访问过*/
int n,m; //迷宫的规模n行m列
bool feasible(int x,int y){ //迷宫第x行第y列的格子能否走
    if(x>=0 && x<n && y>=0 && y<m && vis[x][y]==0 && map[x][y]==0)
        return true;
    return false;
}
void Print(int ans[], int k){
    for(int i=1; i<k; i++)
        printf("%d ", ans[i]);
    printf("\n");
}

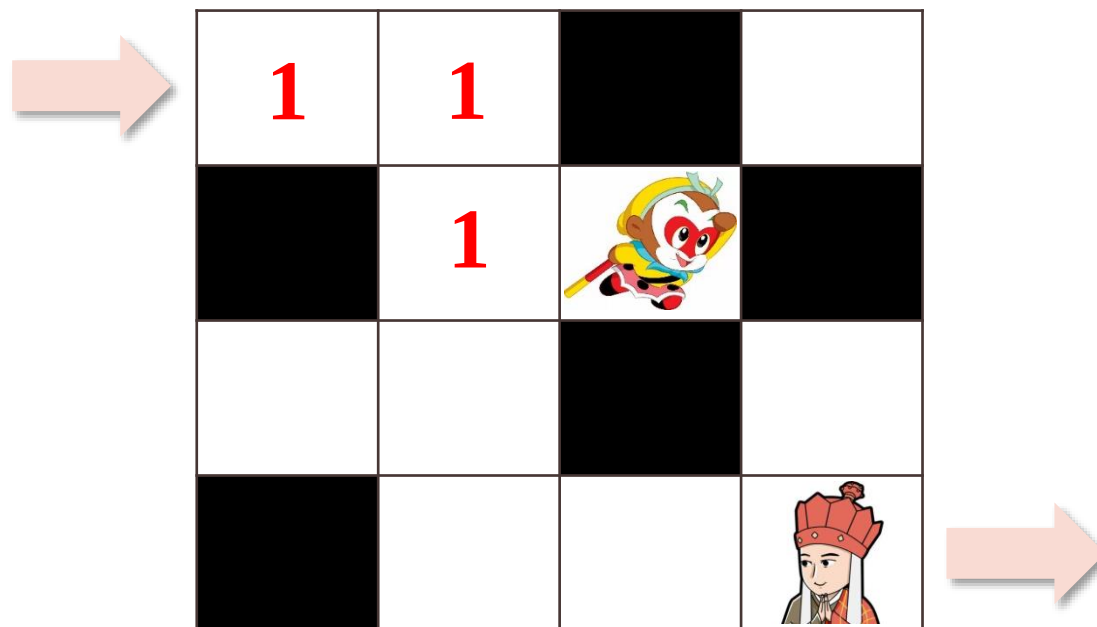
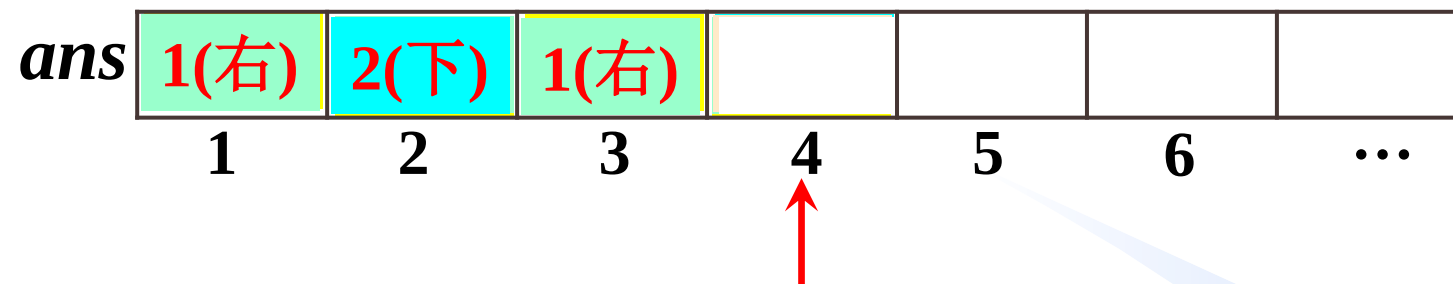
```

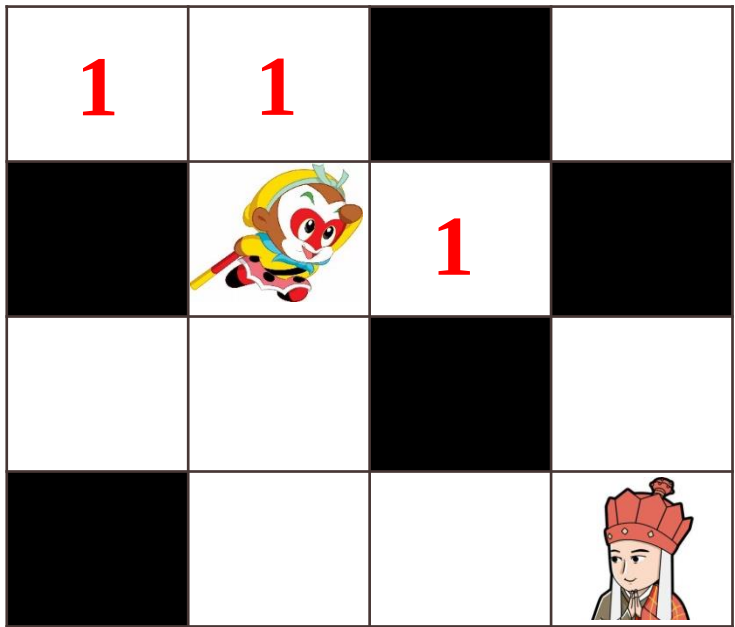
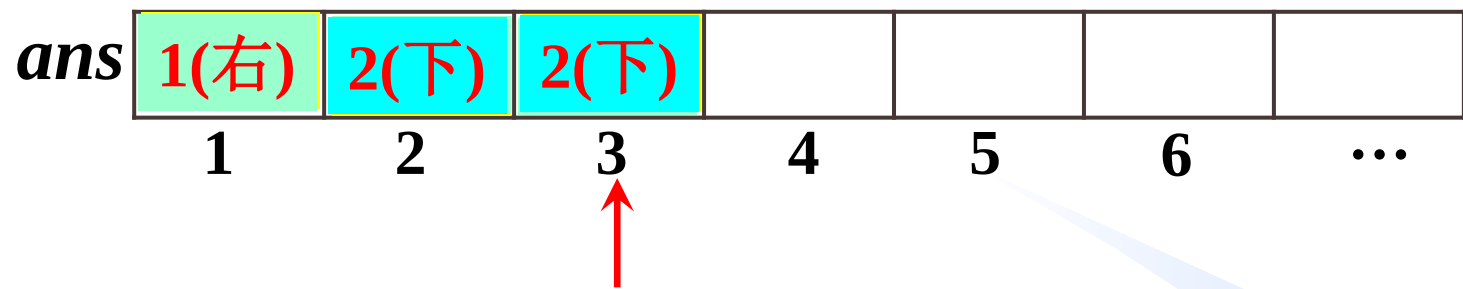
<i>sx, sy</i>			
			<i>tx, ty</i>



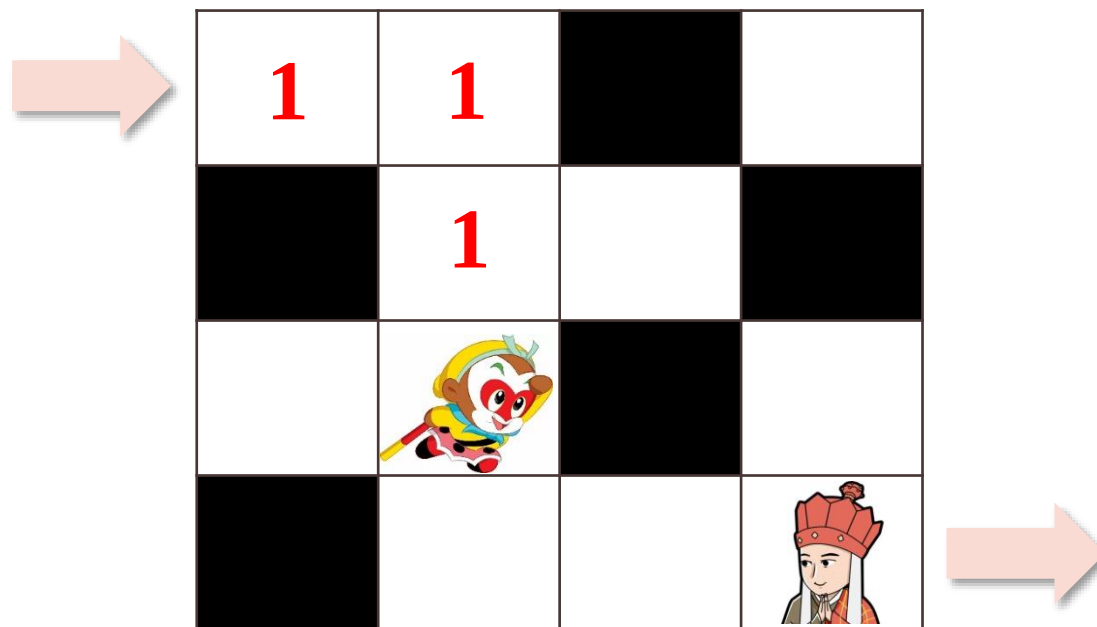
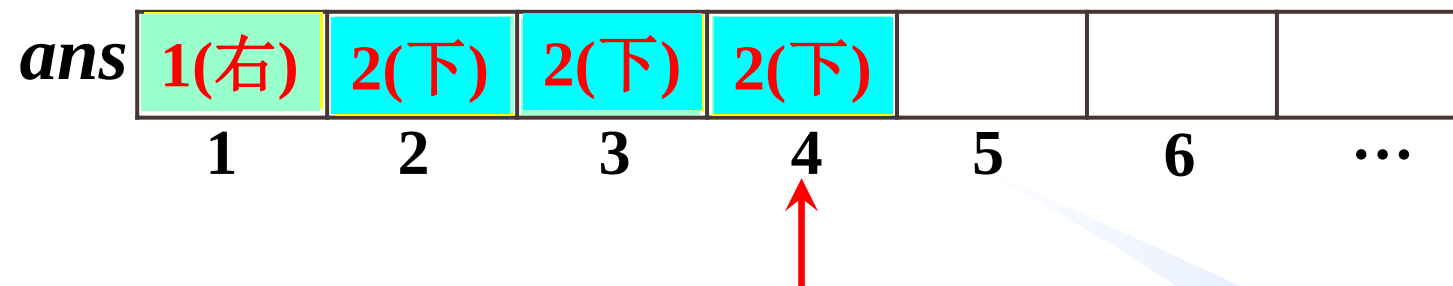


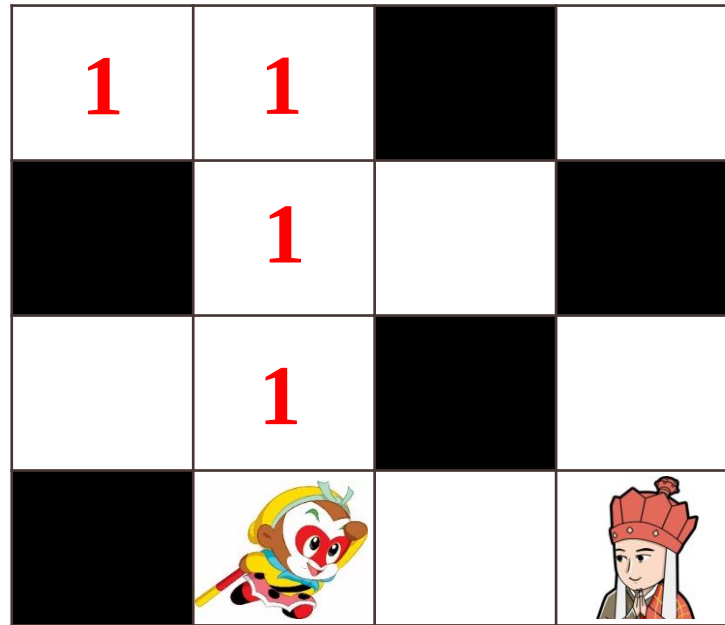
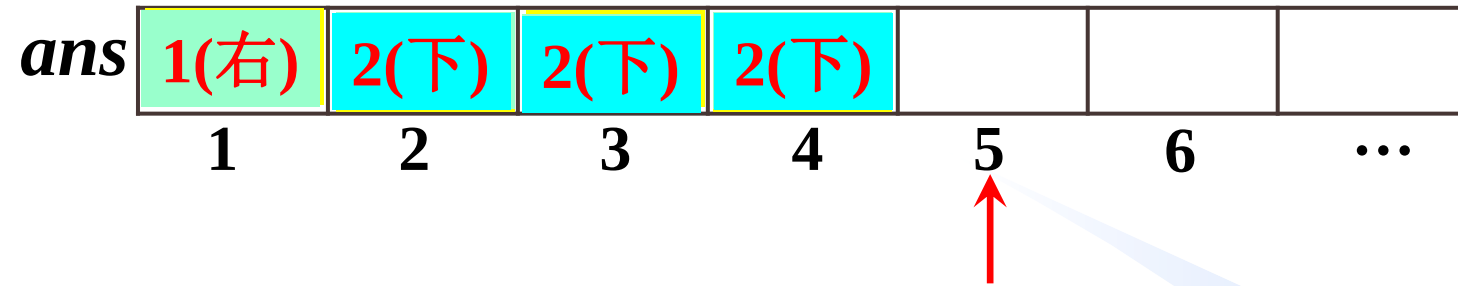






回溯



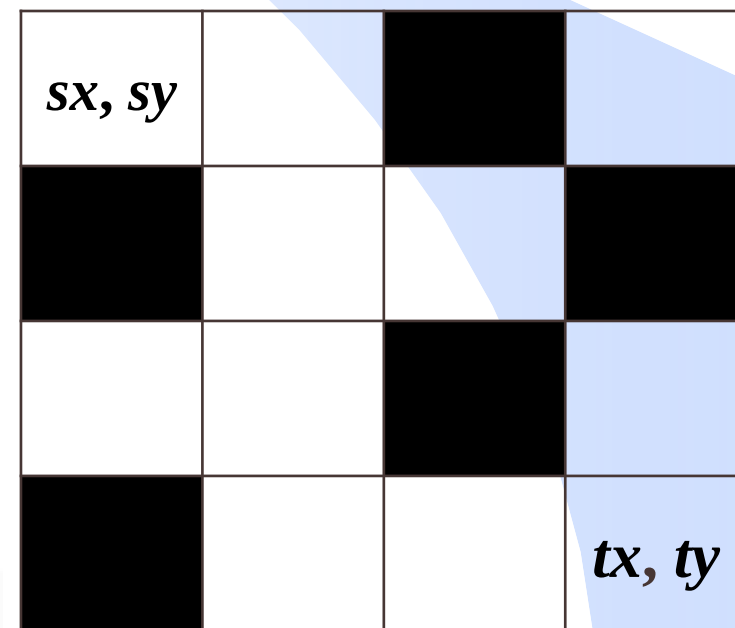
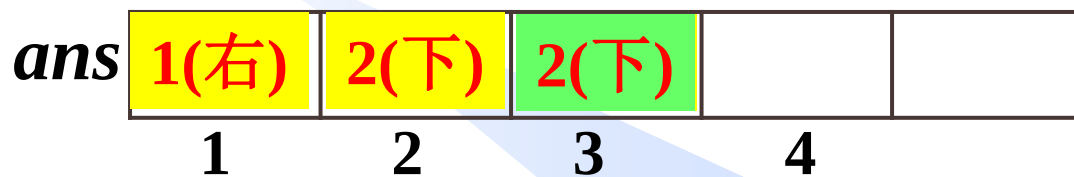



```

void find(int x,int y,int tx,int ty,int ans[],int k){
    //以(x,y)为起点搜索，填解数组的第k位ans[k]
    if(x==tx && y==ty) {Print(ans,k); return;}
    vis[x][y]=1; //访问点(x,y)
    for(int i=0; i<4; i++) {
        int nx=x+dx[i], ny=y+dy[i];
        if(feasible(nx,ny)){
            ans[k] = i;
            find(nx,ny,tx,ty,ans,k+1);
        }
    }
    vis[x][y]=0; //不在当前行进路线上的点vis值清0
}

```

初始调用
`find(sx,sy,tx,ty,ans,1)`



回溯算法：可行则进、不行则换、换不了则退

```
void find(int x,int y,int tx,int ty,int ans[],int k)
```

B

迷宫寻径
回溯法
递归过程

`find(0,0,tx,ty,ans,1)`

`ans[1]=右`

`find(0,1,tx,ty,ans,2)`

`ans[2]=下`

`find(1,1,tx,ty,ans,3)`

`ans[3]=右`

`ans[3]=下`

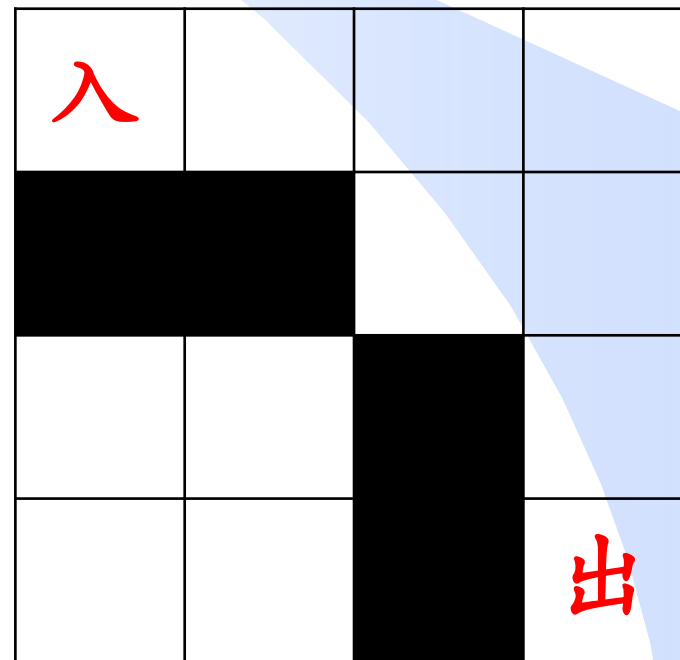
`find(1,2,tx,ty,ans,4)`

`find(2,1,tx,ty,ans,4)`

函数返回前是否有必要把vis[x][y]清0?

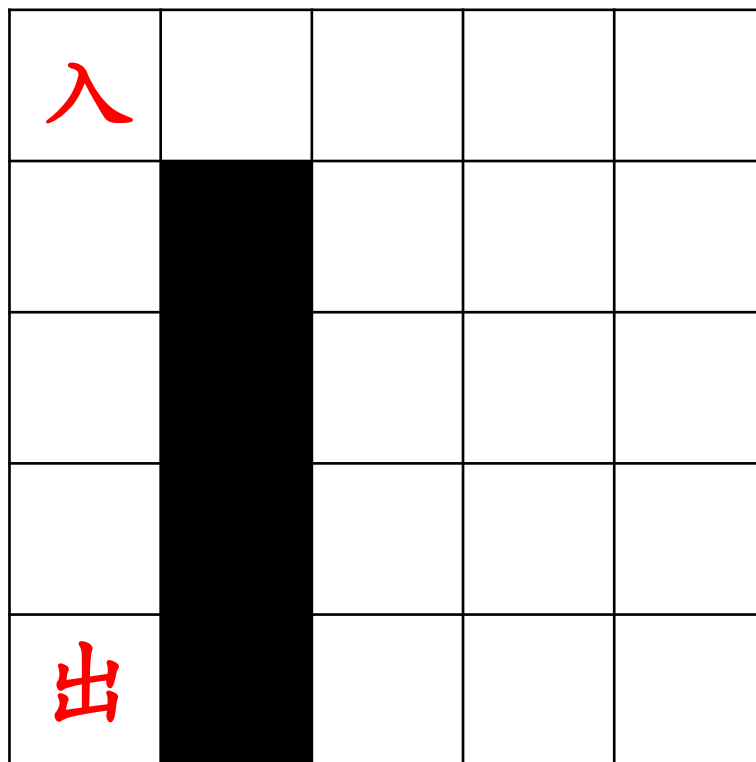
```
void find(int x,int y,int tx,int ty,int ans[],int k){
    if(x==tx && y==ty){Print(ans,k); return;}
    vis[x][y]=1; //访问点(x,y)
    for(int i=0; i<4; i++) {
        int nx=x+dx[i], ny=y+dy[i];
        if(feasible(nx,ny)){
            ans[k] = i;
            find(nx,ny,tx,ty,ans,k+1);
        }
    }
    vis[x][y]=0;
}
```

使不在当前行进路线上的点的vis值清0,
保证只有在当前行进路线上的点的vis
值为1, 以便找到所有路线



函数返回前是否有必要把vis[x][y]清0?

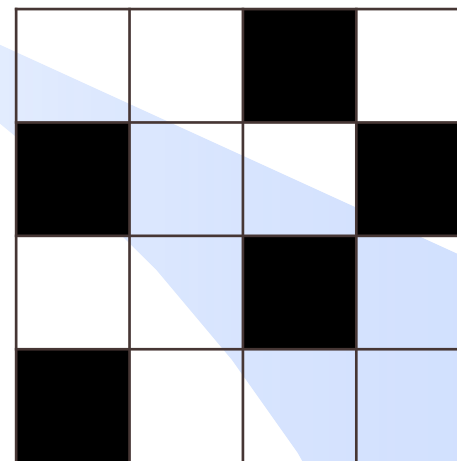
如果只需找一条路径或确定是否存在路径，vis值还有必要清0么？



- ✓ vis值清0：确保找到所有解
- ✓ vis值不清0：更快找到一个解

只找一个解

```
bool find(int x,int y,int tx,int ty,int ans[],int k){  
    //填解数组的第k位ans[k], 当前行进路线上的点的visit值置1  
    if(x==tx && y==ty){ Print(ans,k); return true;}  
    vis[x][y]=1; //访问点(x,y)  
    for(int i=0; i<4; i++) {  
        int nx=x+dx[i], ny=y+dy[i];  
        if(feasible(nx,ny)){  
            ans[k] = i;  
            if(find(nx,ny,tx,ty,ans,k+1)) return true;  
        }  
    }  
    return false;  
}
```



只要找到一个解，就不断向上
返回true;

回溯算法的基本框架

算法 $F(k)$ // 填解数组的第 k 位 $ans[k]$

IF ans 已是一个解 **THEN** (**PRINT**(ans). **RETURN.**) // 输出

FOR $ans[k]$ 的所有可能取值 i **DO**

(**IF** *feasible*(i) **THEN** // 如果 i 可行

($ans[k] \leftarrow i$.

$F(k+1)$. // 填 $ans[k+1]$

)

)

初始调用 $F(1)$

回溯：
可行则进
不行则换
换不了则退

回溯算法的基本框架2（只找一个解）

B

算法 $F(k)$ // 填解数组的第 k 位 $ans[k]$

IF ans 已是一个解 **THEN** (**PRINT**(ans). **RETURN TRUE.**)

FOR $ans[k]$ 的所有可能取值 i **DO**

(**IF** *feasible*(i) **THEN** // 如果 i 可行

($ans[k] \leftarrow i$.

IF $F(k+1) = \text{TRUE}$ **THEN RETURN TRUE.**

)

)

RETURN FALSE. ■

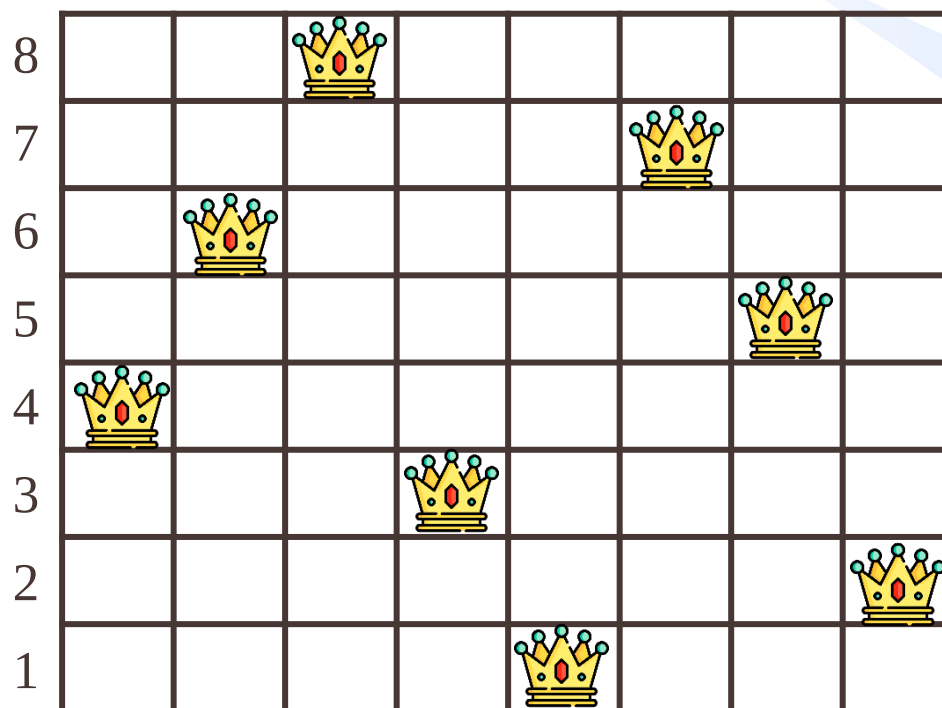
回溯：
可行则进
不行则换
换不了则退

初始调用 $F(1)$

八皇后问题

8 × 8棋盘，如何放置8个皇后，使得任意两个皇后都不在同一行、同一列、同一斜线。

$ans[k]$ 表示第 k 列的皇后放到第几行



ans	4	6	8	3	1	7	5	2
	1	2	3	4	5	6	7	8

类似题目
[HDU 2553](#)

共有92个解，数学家高斯找到76个

八皇后问题——穷（枚）举法

B

算法Nqueens(ans)

FOR ans[1]=1 TO 8 DO

FOR ans[2]=1 TO 8 DO

FOR ans[3]=1 TO 8 DO

FOR ans[4]=1 TO 8 DO

FOR ans[5]=1 TO 8 DO

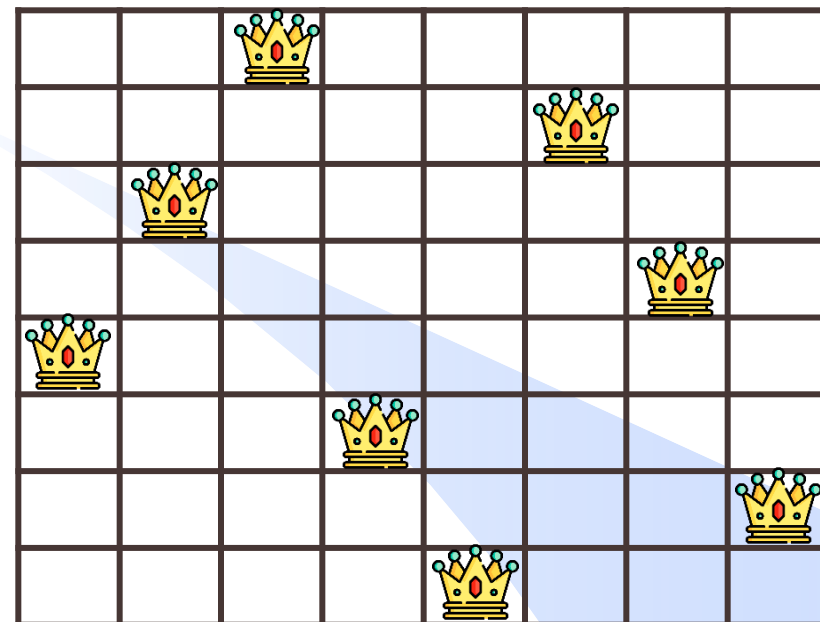
FOR ans[6]=1 TO 8 DO

FOR ans[7]=1 TO 8 DO

FOR ans[8]=1 TO 8 DO

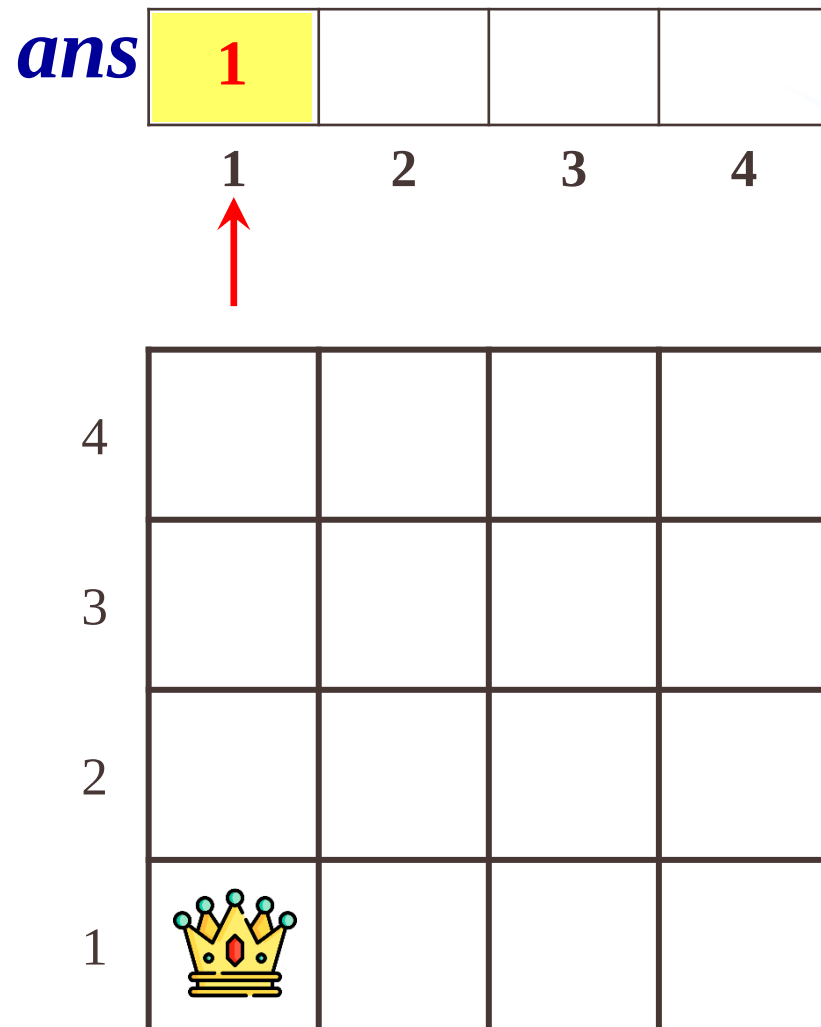
IF feasible(ans) THEN

PRINT(ans). ■

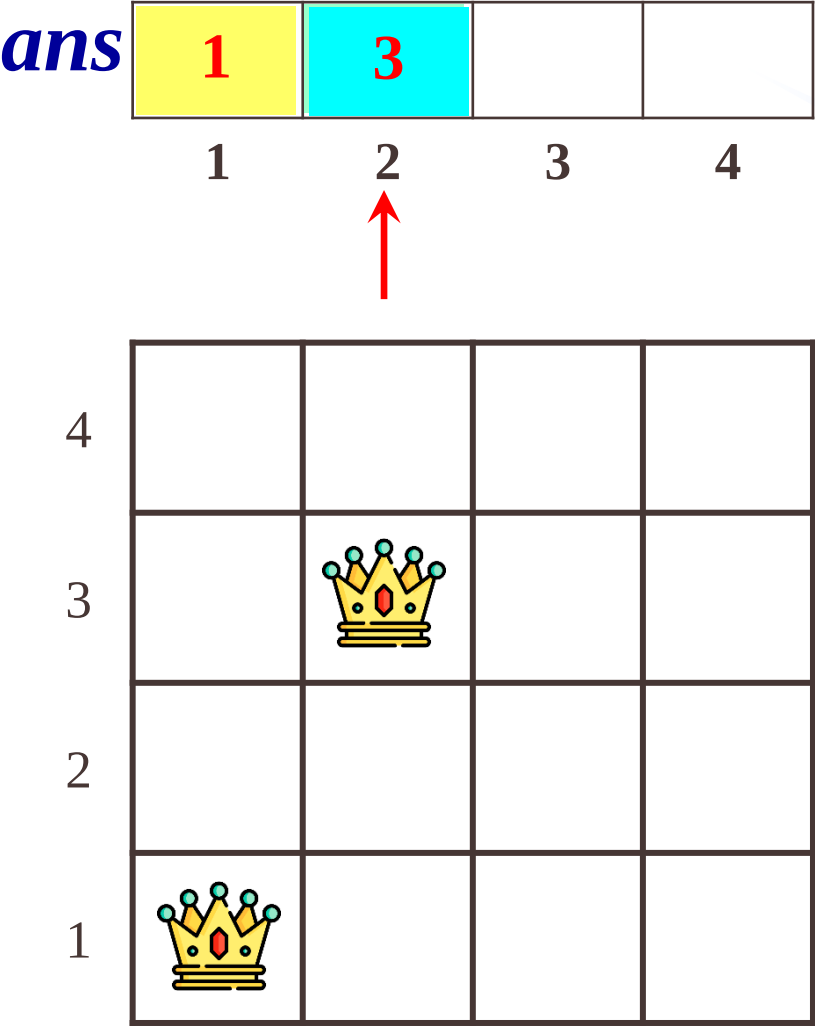


<i>ans</i>	4	6	8	3	1	7	5	2
	1	2	3	4	5	6	7	8

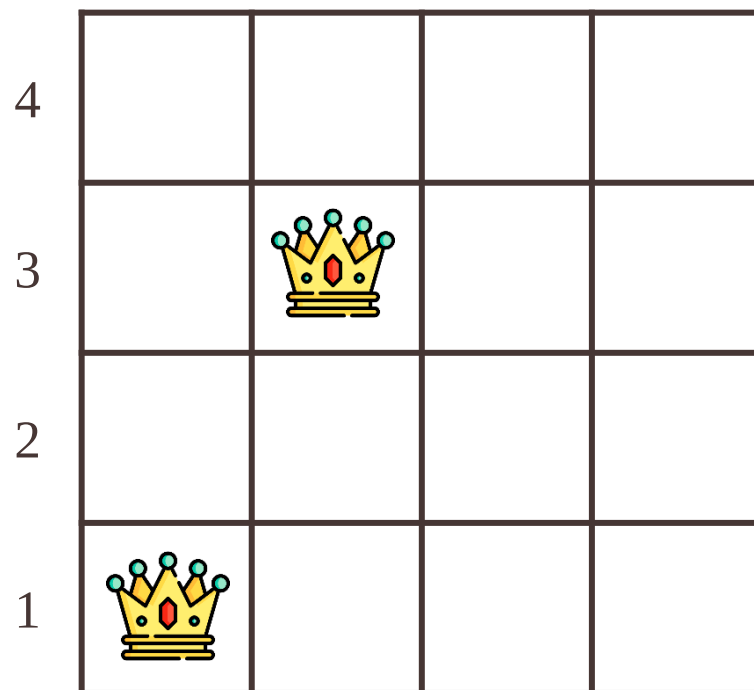
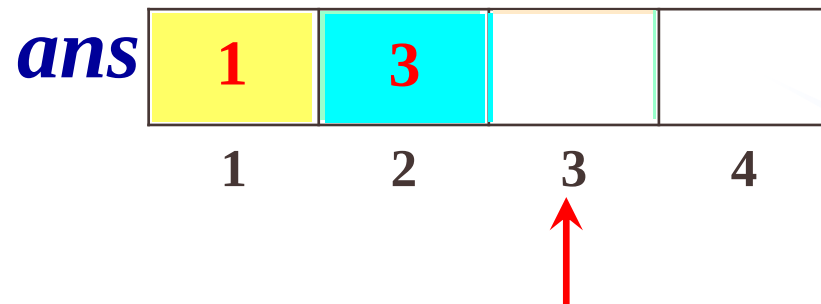
回溯法——以4皇后问题为例



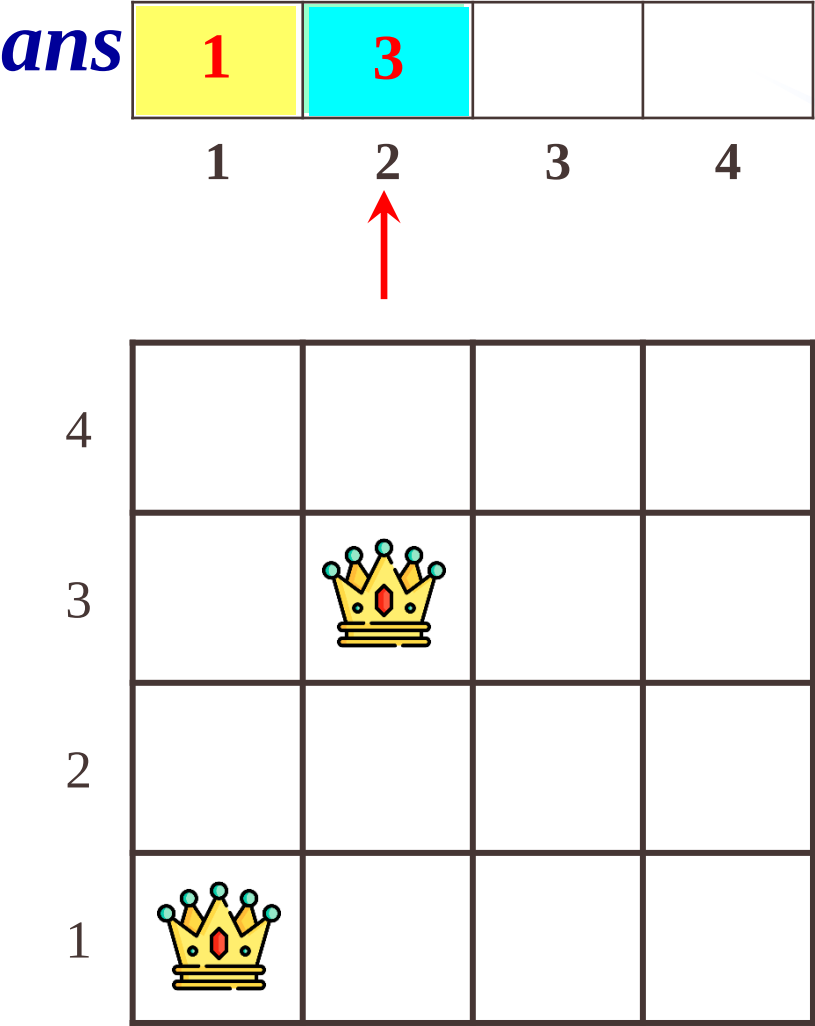
回溯法——以4皇后问题为例



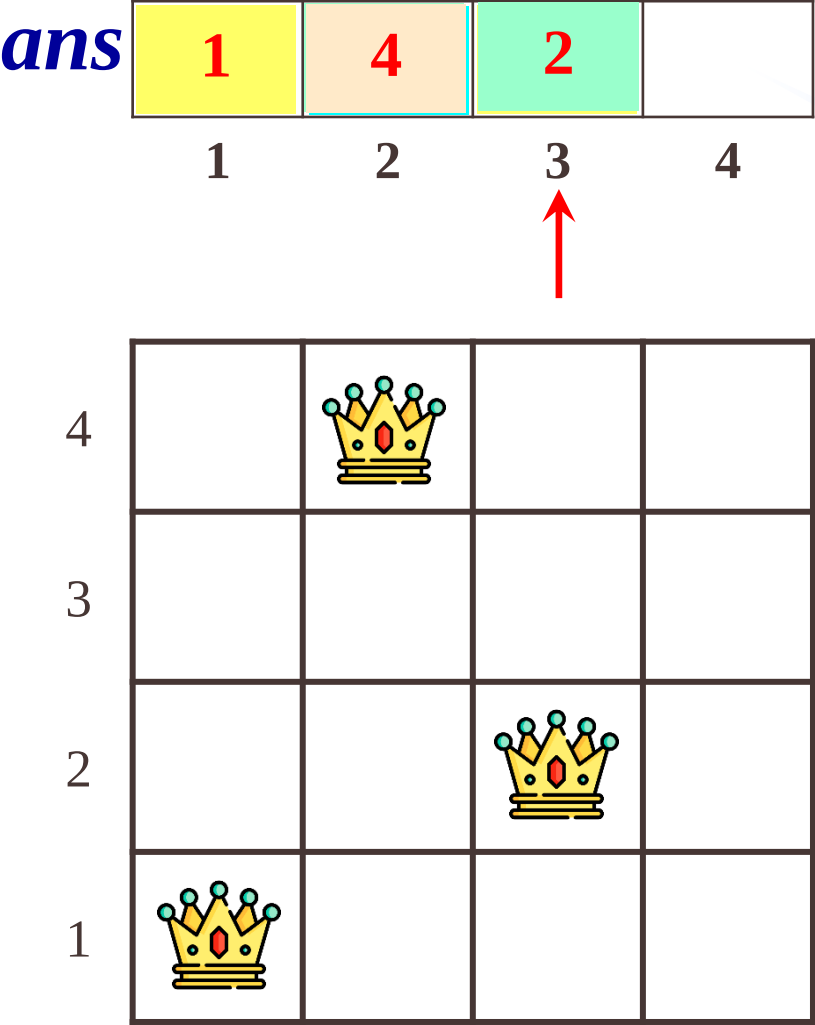
回溯法——以4皇后问题为例



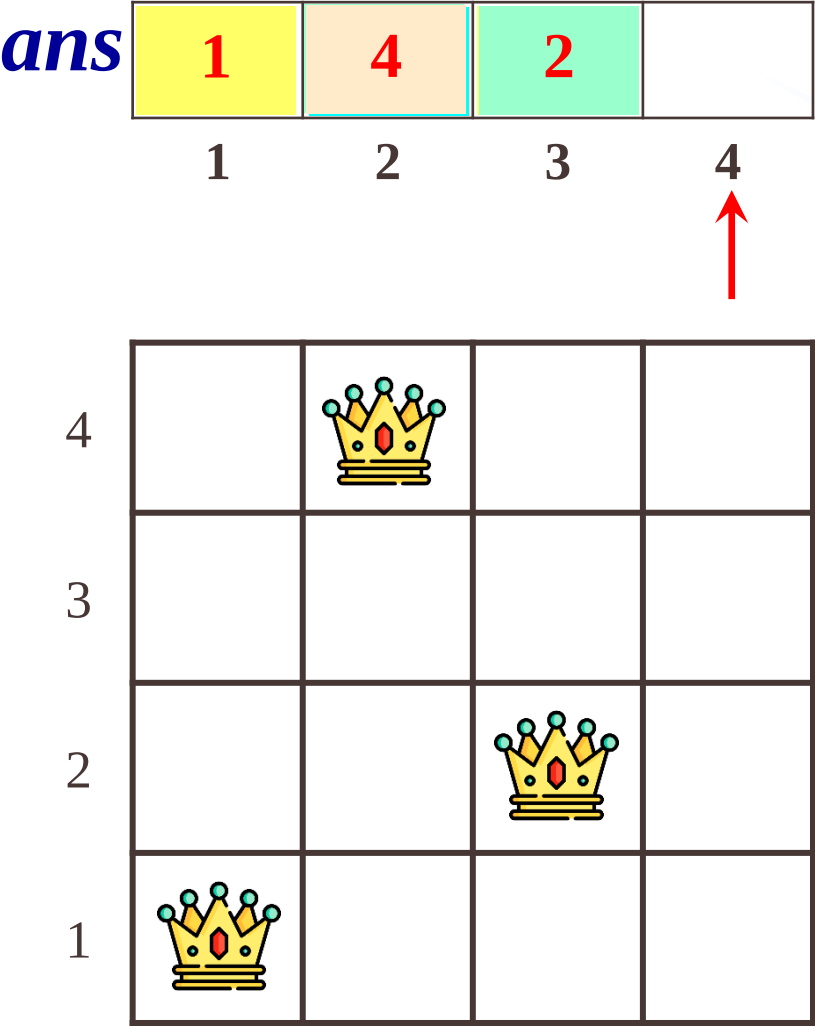
回溯法——以4皇后问题为例



回溯法——以4皇后问题为例



回溯法——以4皇后问题为例

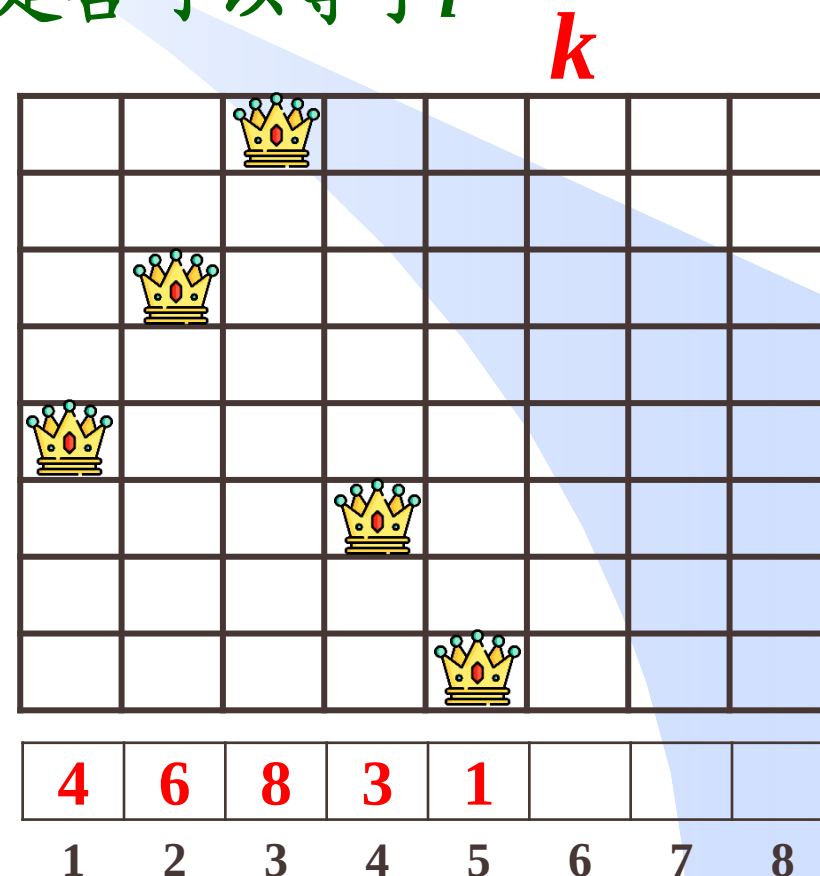


n皇后问题-回溯法

B

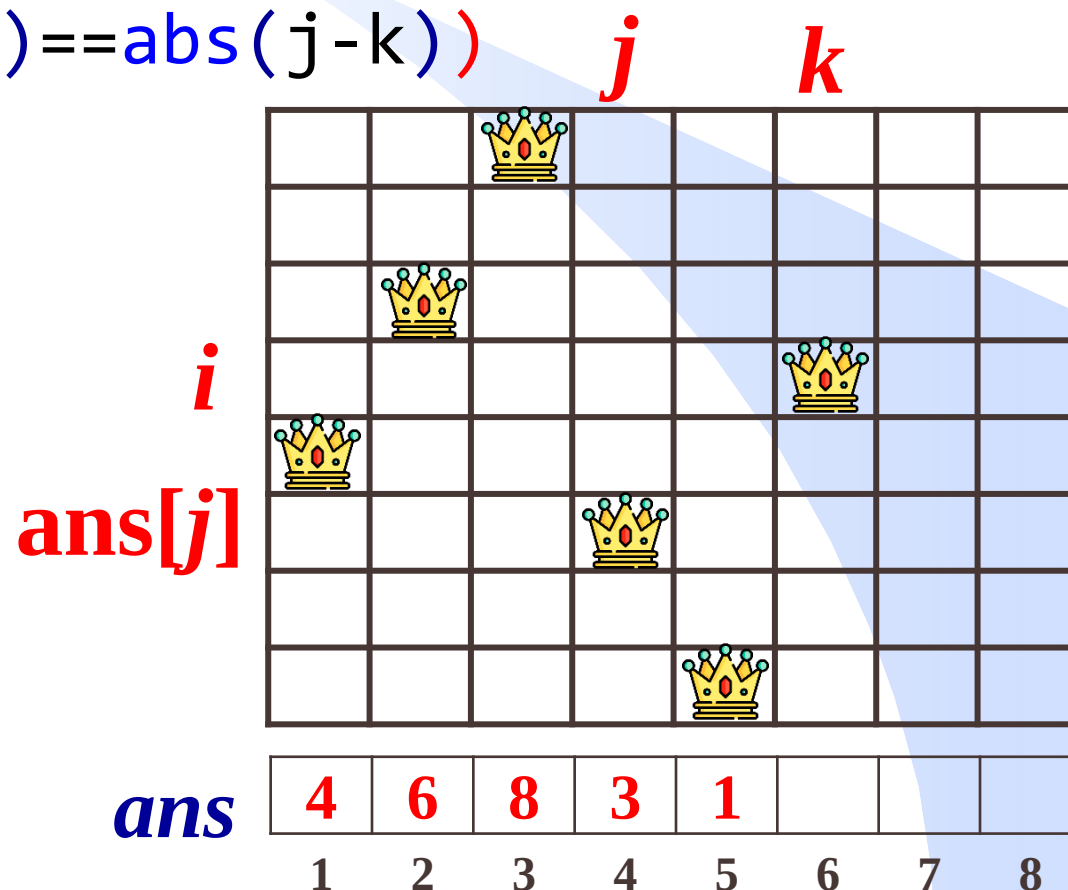
```
void Nqueens(int ans[], int n, int k) { //填解数组的第k位ans[k]
    if(k>n) { Print(ans, n); return; } //解数组的n位都填完了
    for(int i=1; i<=n; i++) {
        if(feasible(ans,k,i)) { //ans[k]是否可以等于i
            ans[k]=i;
            Nqueens(ans, n, k+1);
        }
    }
}
```

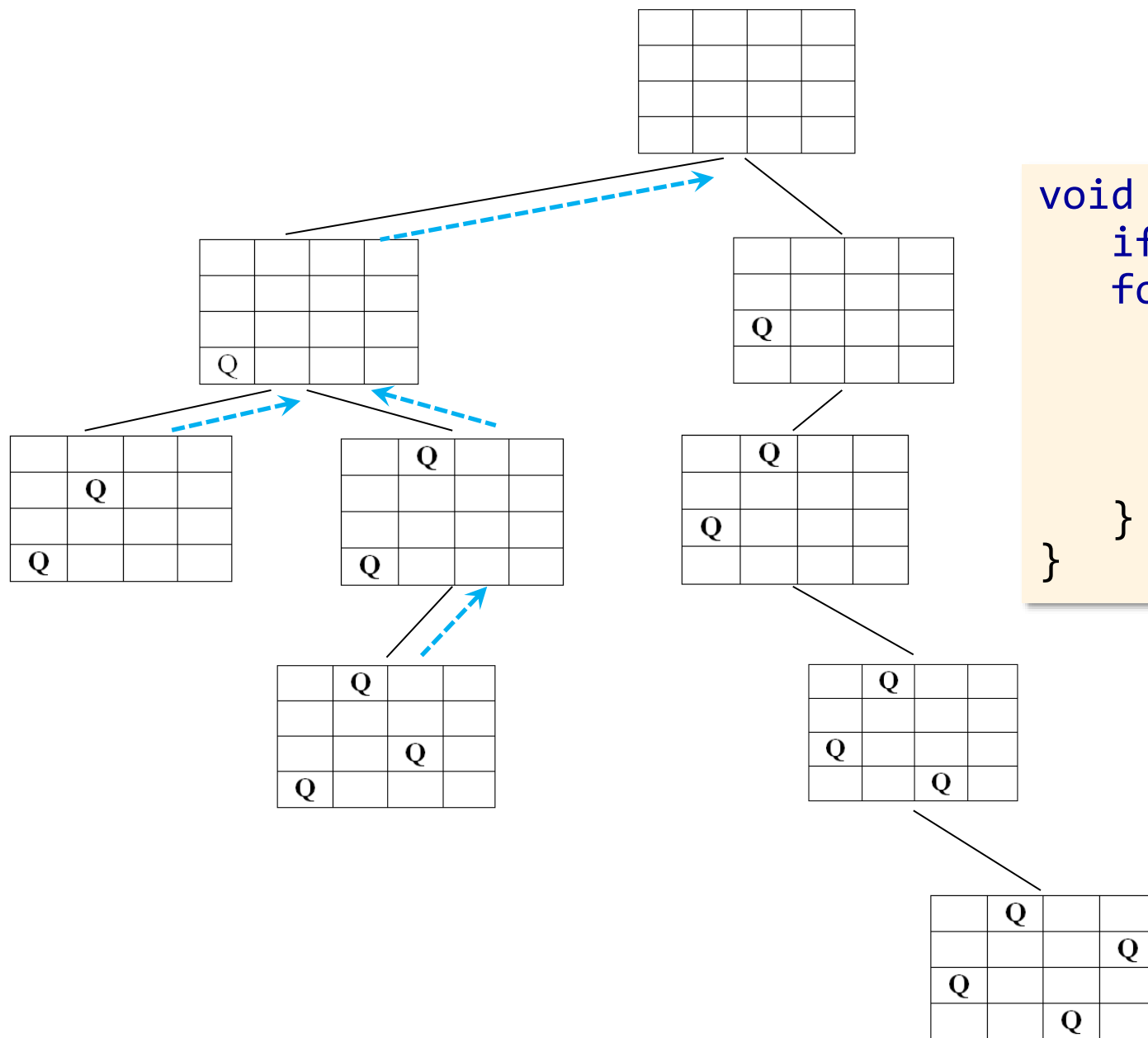
初始调用
Nqueens(ans, n, 1)



n皇后问题-回溯法

```
bool feasible(int ans[], int k, int i){
    //ans[k]是否可以等于i, 即第k列第i行是否可以放置皇后
    for(int j = 1; j <= k-1; j++){
        if(ans[j]==i || abs(ans[j]-i)==abs(j-k))
            return false;
    }
    return true;
}
```





```

void Nqueens(int ans[],int n,int k){
    if(k>n) {Print(ans,n);return;}
    for(int i=1; i<=n; i++) {
        if(feasible(ans,k,i)) {
            ans[k]=i;
            Nqueens(ans, n, k+1);
        }
    }
}

```

课下复盘
4皇后回溯法
递归过程示例



翅膀微扇 万物皆变

蝴蝶效应

回溯到人生岔路口重新选择，结果真的会更好？

快速幂

给定实数 a 和正整数 n ，编写算法求 a^n ，不允许调用库函数。

【字节跳动、腾讯、阿里、快手、微软面试题 [LeetCode50](#)】

```
double pow(double a, long long n){  
    double ans=1;  
    for(int i=0; i<n; i++)  
        ans *= a;  
    return ans;  
}
```



快速幂

给定实数 a 和正整数 n ，编写算法求 a^n ，不允许调用库函数。

【字节跳动、腾讯、阿里、快手、微软面试题 [LeetCode50](#)】

```
double pow(double a, long long n){  
    double ans=1;  
    for(int i=0; i<n; i++)  
        ans *= a;  
    return ans;  
}
```



时间复杂度 $O(n)$

快速幂

$$a^n = \begin{cases} a^{\frac{n}{2}} \times a^{\frac{n}{2}}, & n \text{ 是偶数} \\ a^{\lfloor \frac{n}{2} \rfloor} \times a^{\lfloor \frac{n}{2} \rfloor} \times a, & n \text{ 是奇数} \end{cases}$$

```
double pow(double a, long long n){  
    if (a==1 || n==0) return 1;  
    double b = pow(a, n/2);  
    double ans = b*b;  
    if (n % 2 == 1) ans *= a;  
    return ans;  
}
```

时间复杂度 $O(\log n)$

消除递归的方法

- 尾递归转循环：递归调用出现在函数末尾，可用循环结构代替递归。
- 利用栈模拟递归过程。具体问题具体分析，结合一定技巧。
- 递归转递推：动态规划（下章介绍）。
-

消除递归的方法1——尾递归转循环

➤ 尾递归：递归调用出现在函数末尾，可用循环结构代替递归。

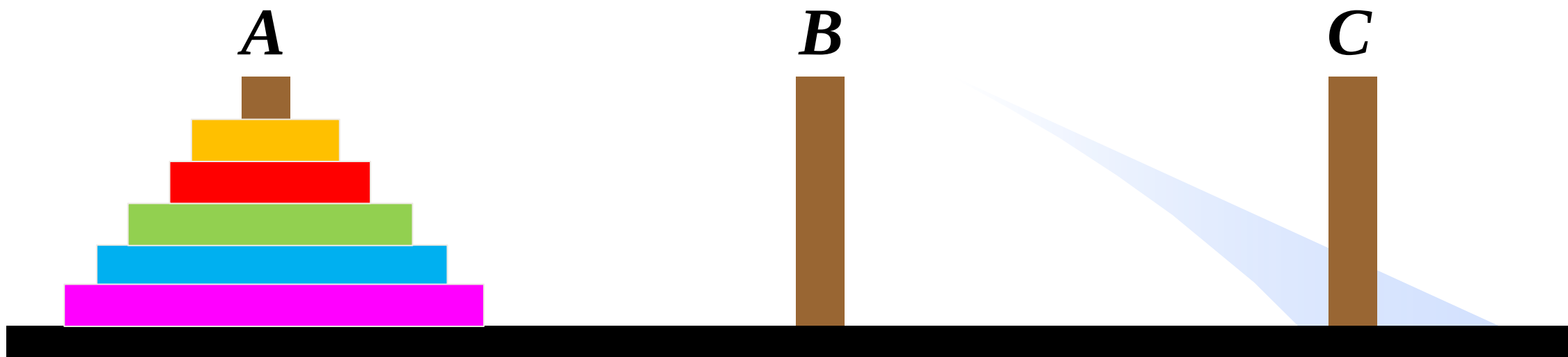
```
int f(int n, int sum){  
    if(n == 0) return sum;  
    sum += n;  
    return f(n-1, sum);  
}
```



```
int f(int n, int sum){  
    while(n>0){  
        sum += n;  
        n--;  
    }  
    return sum;  
}
```


消除递归的方法2——利用自定义栈模拟递归过程

B

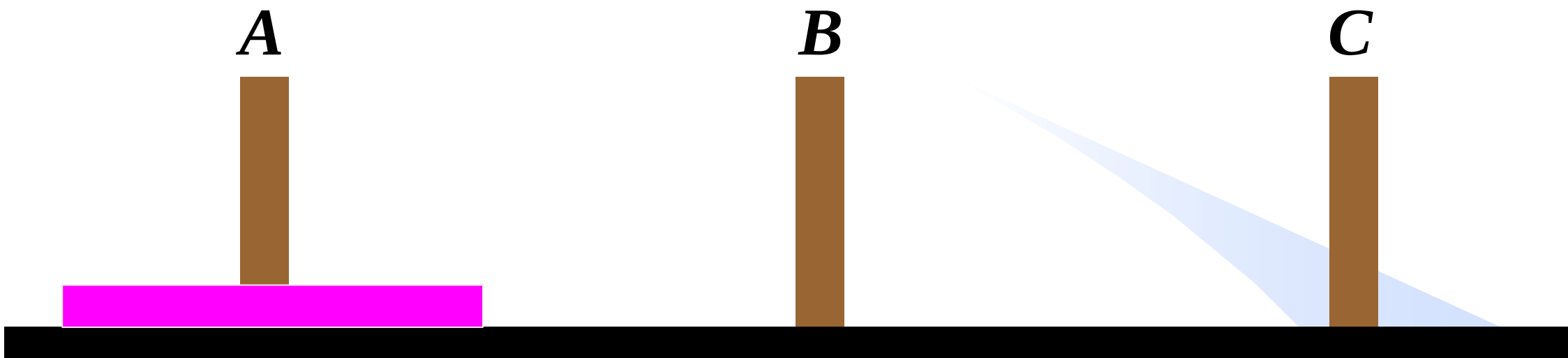


汉诺塔问题

借助B柱，将 n 个盘子从A柱移至C柱

```
void Hanoi(int n, char A, char B, char C)
```

汉诺塔问题

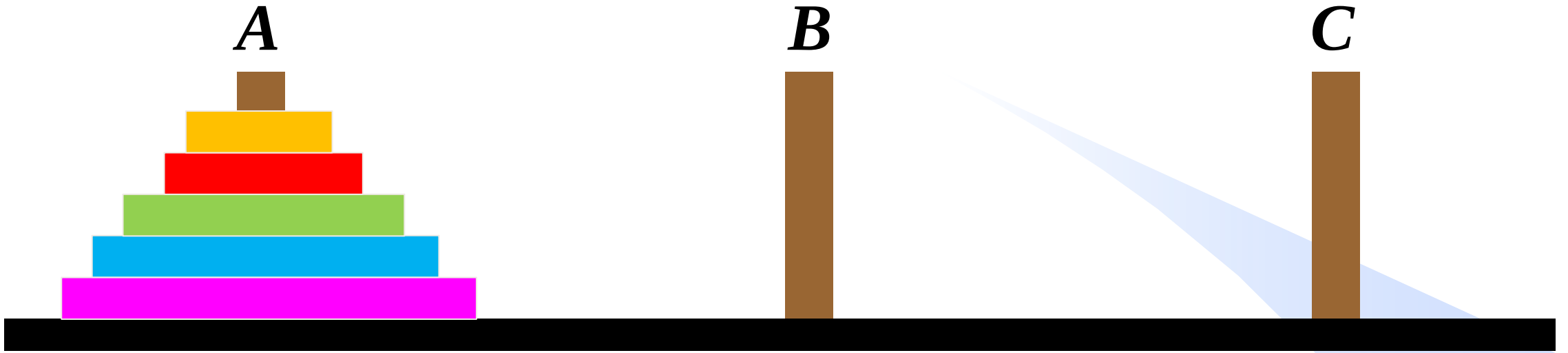


```
void Hanoi(int n, char A, char B, char C) {  
    if(n == 1) move(A, C);
```

若只有1个盘子，则直接
将其从A柱移至C柱

```
void move(char x, char y) {  
    printf("Move disk from %c to %c\n", x, y);  
}
```

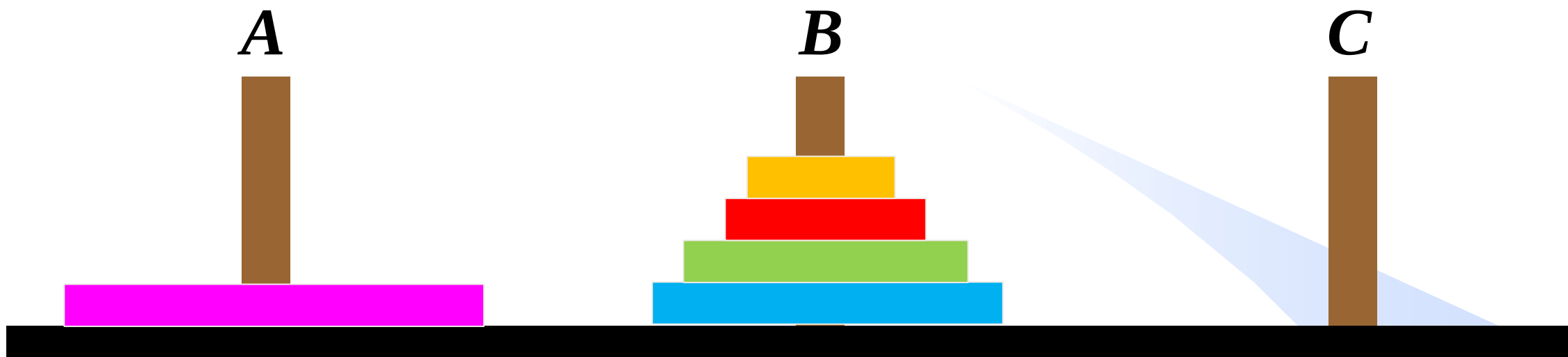
汉诺塔问题



```
void Hanoi(int n, char A, char B, char C) {  
    if(n == 1) move(A, C);  
    else {
```

若有多个盘子

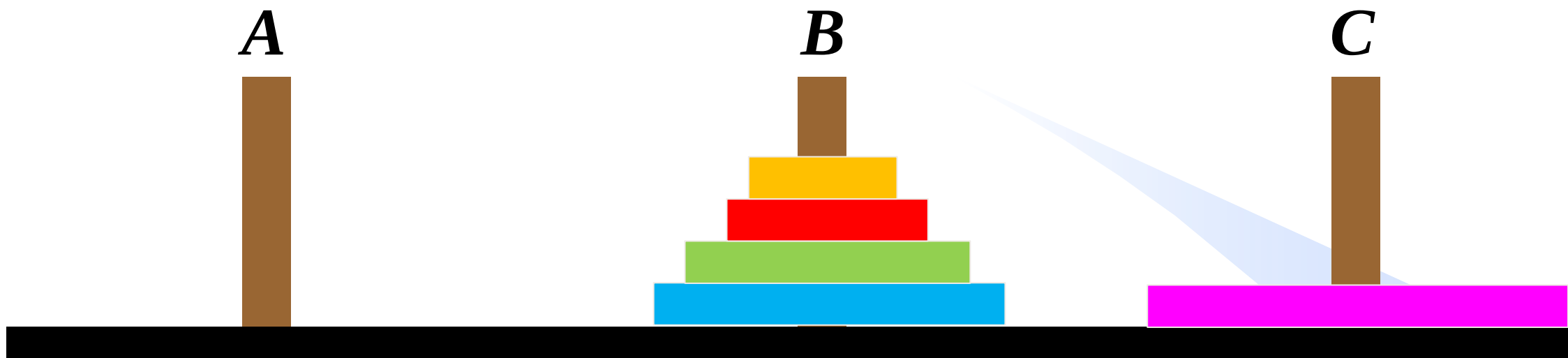
汉诺塔问题



```
void Hanoi(int n, char A, char B, char C) {  
    if(n == 1) move(A, C);  
    else {  
        Hanoi(n-1, A, C, B);
```

将 $n-1$ 个盘子
由A柱经C柱移至B柱

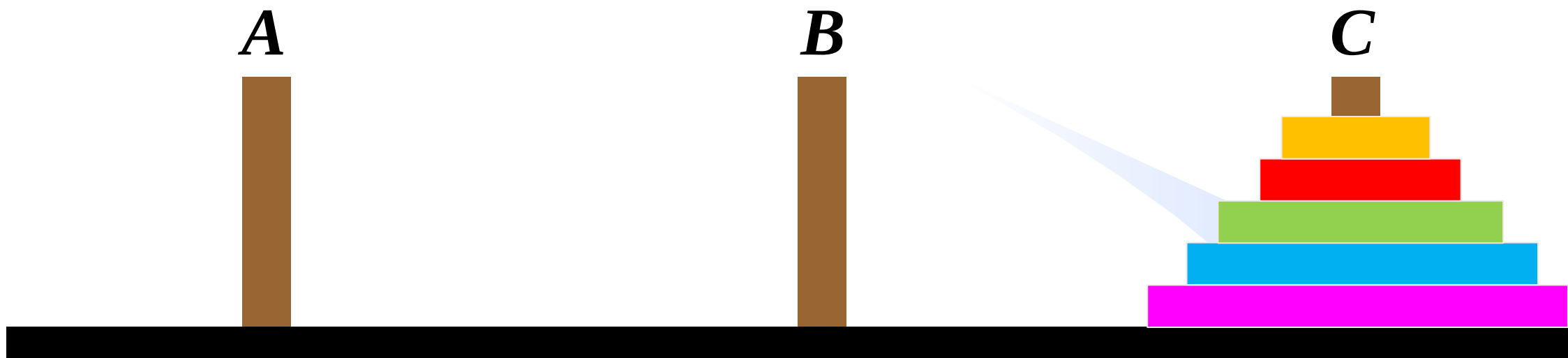
汉诺塔问题



```
void Hanoi(int n, char A, char B, char C) {  
    if(n == 1) move(A, C);  
    else {  
        Hanoi(n-1, A, C, B);  
        move(A, C);  
    }  
}
```

将A柱剩下的1个盘子
移至C柱

汉诺塔问题



```
void Hanoi(int n, char A, char B, char C) {  
    if(n == 1) move(A, C);  
    else {  
        Hanoi(n-1, A, C, B);  
        move(A, C);  
        Hanoi(n-1, B, A, C);  
    }  
}
```

将 $n-1$ 个盘子
由B柱经A柱移至C柱

求解汉诺塔问题的递归算法

```
void Hanoi(int n, char A, char B, char C) {  
    //将 n 个圆盘由A柱经B柱移至 C柱  
    if (n == 1) move(A, C); //将 A 柱唯一圆盘移至 C 柱  
    else {  
        Hanoi(n-1, A, C, B); //将n-1个圆盘由A柱经C柱移至B柱  
        move(A, C);  
        Hanoi(n-1, B, A, C); //将n-1个圆盘由B柱经A柱移至C柱  
    }  
}  
  
void move(char x, char y) {  
    printf("Move disk from %c to %c\n", x, y);  
}
```

课下思考

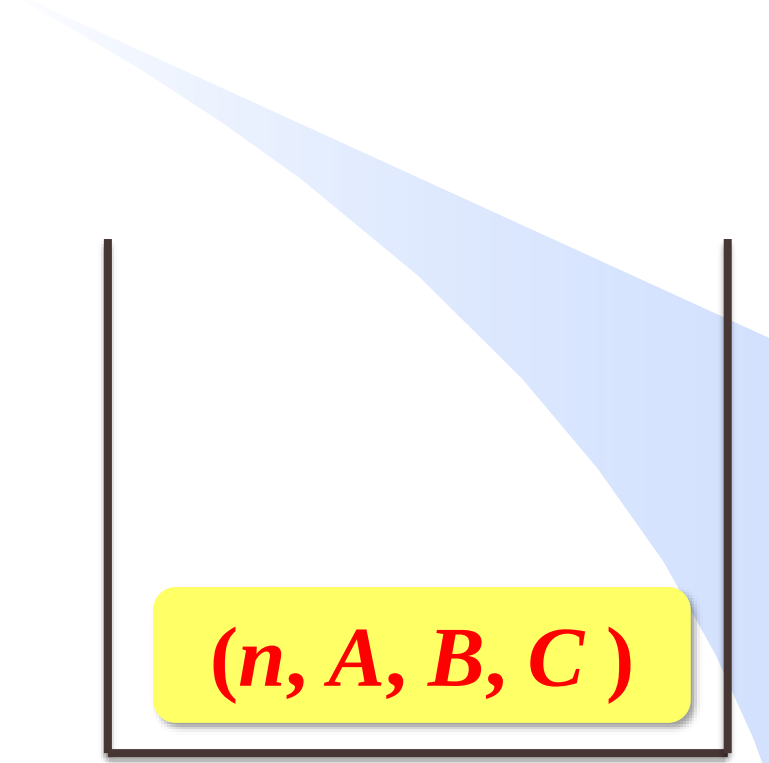
时间复杂度和空间复杂度

利用自定义栈模拟递归过程

采用栈模拟递归过程中子任务的产生和处理过程：

- 产生新的子任务对应入栈操作；
- 处理子任务对应出栈操作；

```
void Hanoi(int n, char A, char B, char C){  
    if (n==1) move(A, C);  
    else {  
        Hanoi(n-1, A, C, B);    ①  
        Hanoi(1, A, B, C);      ②  
        Hanoi(n-1, B, A, C);    ③  
    }  
}
```



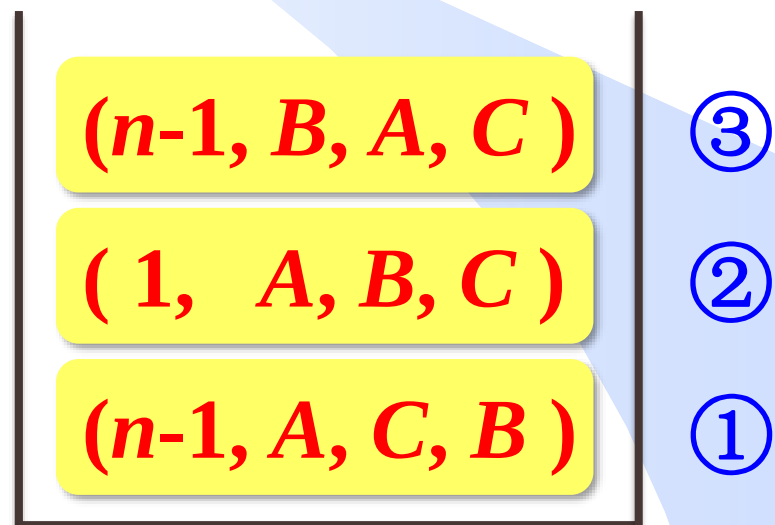
(n, A, B, C)

栈 保存四元组 (n, A, B, C)

利用自定义栈模拟递归过程

按子任务的处理顺序进栈？

```
void Hanoi(int n, char A, char B, char C){
    if (n==1) move(A, C);
    else {
        Hanoi(n-1, A, C, B);    ①
        Hanoi(1, A, B, C);      ②
        Hanoi(n-1, B, A, C);    ③
    }
}
```



栈 保存四元组 (n, A, B, C)

利用自定义栈模拟递归过程

按处理子任务的逆序进栈

- 先处理的子任务后入栈;
- 后处理的子任务先入栈。

```
void Hanoi(int n, char A, char B, char C){  
    if (n==1) move(A, C);  
    else {  
        Hanoi(n-1, A, C, B);  
        Hanoi(1, A, B, C);  
        Hanoi(n-1, B, A, C);  
    }  
}
```

①

②

③

$(n-1, A, C, B)$

$(1, A, B, C)$

$(n-1, B, A, C)$

①

②

③

栈 保存四元组 (n, A, B, C)





```
struct Quad { int n; char A, B, C; }; //四元组
```

```
class HnoiStack{
```

```
private:
```

```
    Quad H[MaxSize];
```

```
    int top = -1;
```

```
public:
```

```
    bool empty() { return top == -1; }
```

```
    bool full() { return top == MaxSize - 1; }
```

```
    void push(int n, char A, char B, char C) {
```

```
        assert(!full()); top++;
```

```
        H[top].n = n; H[top].A = A; H[top].B = B; H[top].C = C;
```

```
    }
```

```
    void pop(int &n, char &A, char &B, char &C) {
```

```
        assert(!empty());
```

```
        n = H[top].n; A = H[top].A; B = H[top].B; C = H[top].C;
```

```
        top--;
```

```
    }
```

```
};
```

(n, A, B, C)

Hanoi塔问题的非递归算法

```

void Hanoi_NonRec(int n, char A, char B, char C){
    HnoiStack s;
    s.push(n, A, B, C);
    while (!s.empty()) {
        int n; char A, B, C;
        s.pop(n, A, B, C);
        if(n == 1) move(A, C);
        else {
            s.push(n-1, B, A, C);
            s.push(1, A, B, C);
            s.push(n-1, A, C, B);
        }
    }
}

```

```

void Hanoi(int n, char A, char B, char C){
    if(n == 1) move(A, C);
    else {
        Hanoi(n-1, A, C, B);
        Hanoi(1, A, B, C);
        Hanoi(n-1, B, A, C);
    }
}

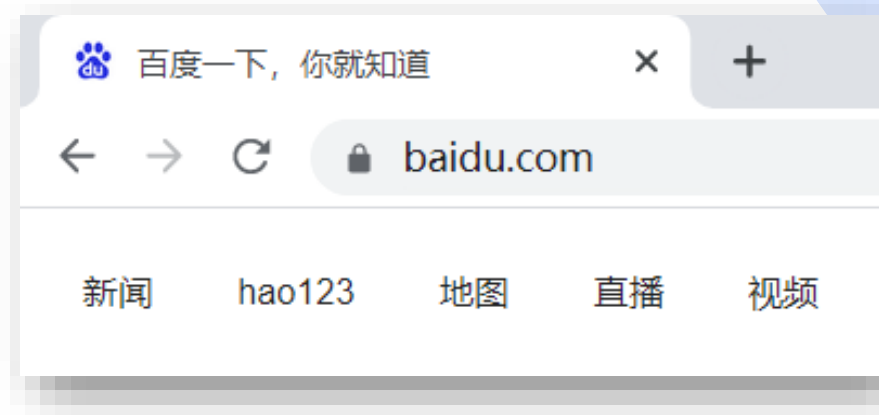
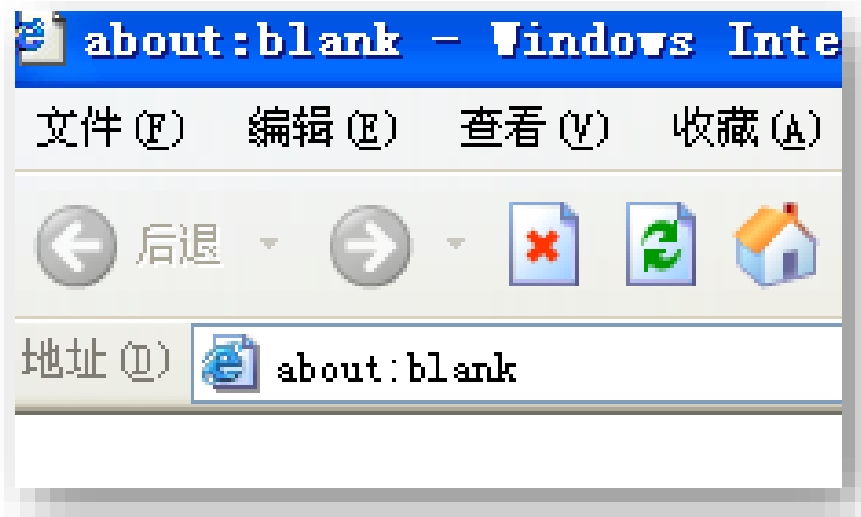
```

(n-1, A, C, B)
(1, A, B, C)
(n-1, B, A, C)

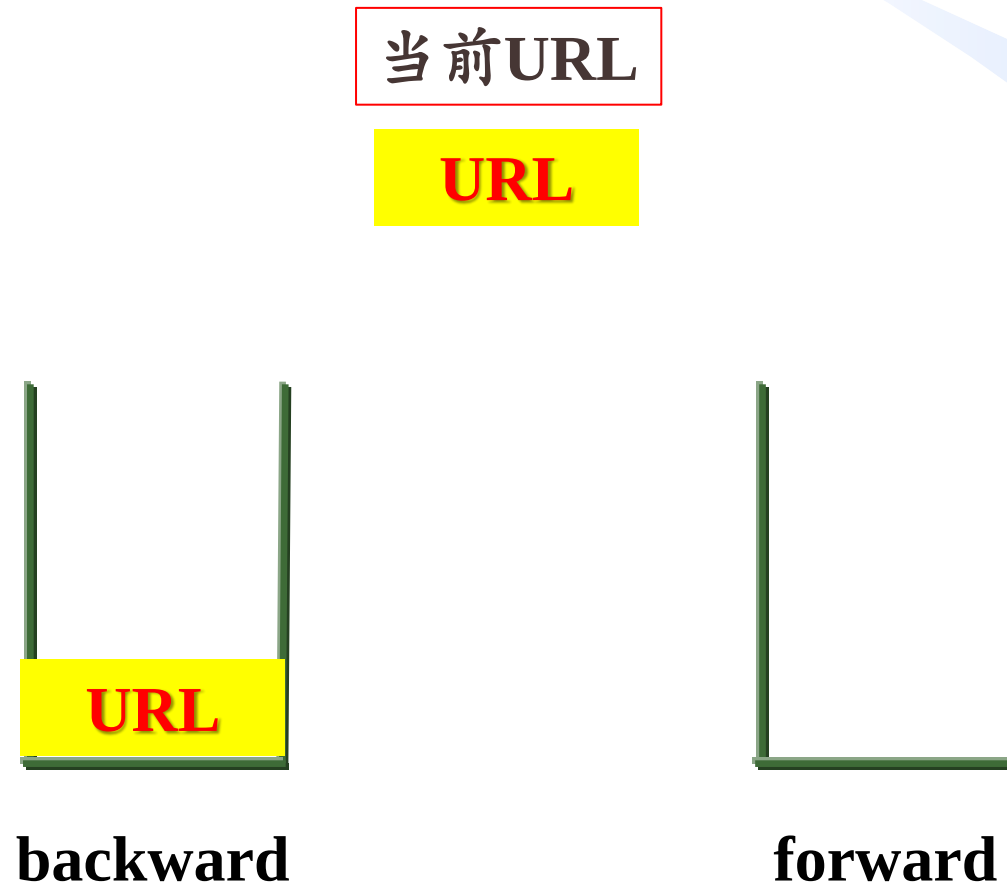
栈的其他应用举例——网络浏览器的前进/后退功能

C

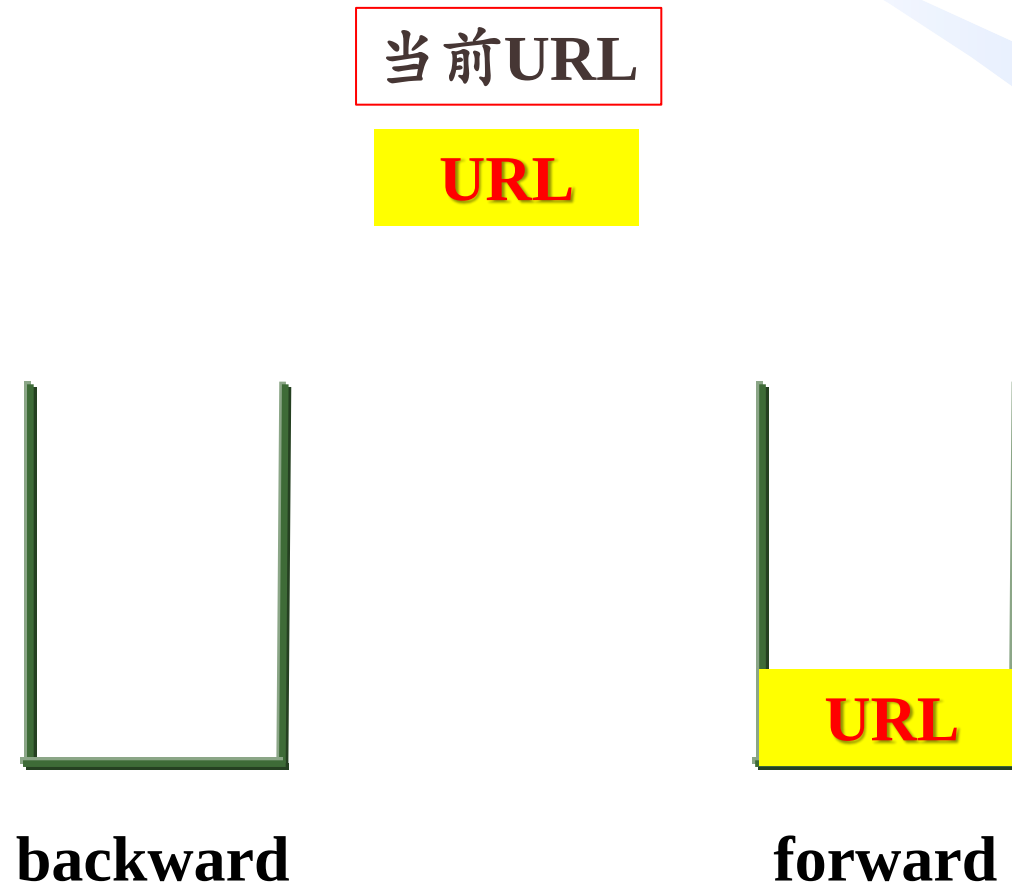
- 在用户最近访问的网页中进行“前进”和“后退”是web浏览器的常用功能。
- 实现该功能的一种方式是使用两个栈（backward栈和forward栈）来存储用户访问的网址，用户的不同操作对应的具体实现方法如下：



后退 (BACK) : 将当前页面压入forward栈; 从backward栈中弹出一个页面, 作为当前页面。如果backward栈为空, 则该命令被忽略。



前进（FORWARD）：将当前页面压入backward栈；从forward栈中弹出一个页面，作为当前页面。如果forward栈为空，则该命令被忽略。

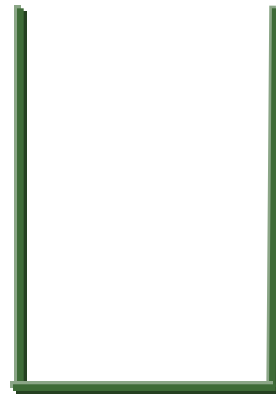


- 访问（VISIT）：将当前页面压入backward栈，并将此次新访问的网页作为当前页面，清空forward栈。

当前URL

[POJ 1028](#)

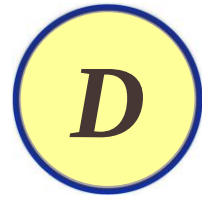
用户新访问的URL



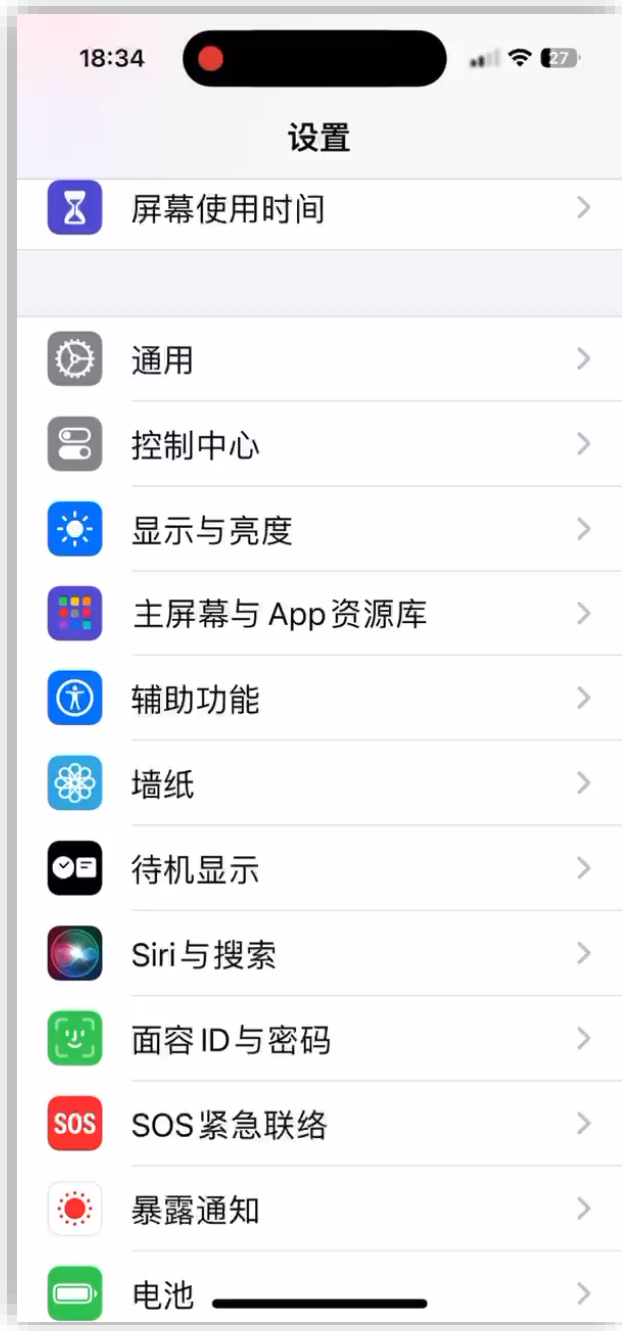
backward



forward



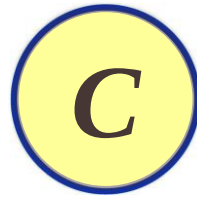
栈的其他应用举例



第3个界面

第2个界面

第1个界面



C++ STL中的栈 `stack<T>`

成员函数	作用
<code>bool empty()</code>	判断栈空
<code>int size()</code>	返回栈中元素个数
<code>void push(T x)</code>	将x压栈
<code>void pop()</code>	弹栈
<code>T top()</code>	返回栈顶元素

例：

```
stack<int> stk; //创建一个栈，其元素为int型
stk.push(5);
int x = stk.top();
stk.pop();
```



自愿性质OJ练习题

- ✓ [LeetCode 150](#) (后缀表达式求值)
- ✓ [洛谷P1449](#) (后缀表达式求值)
- ✓ [openjudgeP0440](#) (含小数的中缀表达式求值)
- ✓ [LeetCode 50](#) (快速幂, 注意本题n可能是负数, 若n是负数, -n会超出int范围)
- ✓ [HDU 2553](#) (N皇后问题)
- ✓ [洛谷P1605](#) (回溯法走迷宫)
- ✓ [洛谷B3625](#) (回溯法走迷宫)
- ✓ [HDU 1702](#) (栈和队列)
- ✓ [POJ 1028](#) (栈实现网页浏览器的前进后退)
- ✓ [POJ 1363](#) (栈混洗的甄别)
- ✓ [openjudgeP0450](#) (栈混洗的甄别)